

CONTENTS

<i>Contributors</i>	<i>vii</i>
<i>Acknowledgments</i>	<i>xx</i>
<i>Preface</i>	<i>xxi</i>
<i>Introduction</i>	<i>xxiii</i>

I	Declarations and Access Control	I
	Java Refresher	2
	Identifiers & JavaBeans (Objectives 1.3 and 1.4)	4
	Legal Identifiers	5
	Sun's Java Code Conventions	6
	JavaBeans Standards	8
	Declare Classes (Exam Objective 1.1)	10
	Source File Declaration Rules	11
	Class Declarations and Modifiers	12
	Exercise I-1: Creating an Abstract Superclass and Concrete Subclass	18
	Declare Interfaces (Exam Objectives 1.1 and 1.2)	19
	Declaring an Interface	19
	Declaring Interface Constants	22
	Declare Class Members (Objectives 1.3 and 1.4)	24
	Access Modifiers	24
	Nonaccess Member Modifiers	39
	Constructor Declarations	47
	Variable Declarations	49
	Declaring Enums	60
	✓ Two-Minute Drill	68
Q&A	Self Test	74
	Self Test Answers	79

2	Object Orientation	85
	Encapsulation (Exam Objective 5.1)	86
	Inheritance, Is-A, Has-A (Exam Objective 5.5)	90
	IS-A	94
	HAS-A	96
	Polymorphism (Exam Objective 5.2)	98
	Overriding / Overloading (Exam Objectives 1.5 and 5.4)	103
	Overridden Methods	103
	Overloaded Methods	109
	Reference Variable Casting (Objective 5.2)	116
	Implementing an Interface (Exam Objective 1.2)	120
	Legal Return Types (Exam Objective 1.5)	126
	Return Type Declarations	126
	Returning a Value	128
	Constructors and Instantiation	
	(Exam Objectives 1.6, 5.3, and 5.4)	130
	Determine Whether a Default Constructor	
	Will Be Created	135
	Overloaded Constructors	139
	Statics (Exam Objective 1.3)	145
	Static Variables and Methods	145
	Coupling and Cohesion (Exam Objective 5.1)	151
	✓ Two-Minute Drill	157
	Q&A Self Test	162
	Self Test Answers	171
3	Assignments	183
	Stack and Heap—Quick Review	184
	Literals, Assignments, and Variables	
	(Exam Objectives 1.3 and 7.6)	186
	Literal Values for All Primitive Types	186
	Assignment Operators	190
	Exercise 3-1: Casting Primitives	195
	Using a Variable or Array Element That Is Uninitialized	
	and Unassigned	203
	Local (Stack, Automatic) Primitives and Objects	207

Passing Variables into Methods (Objective 7.3)	213
Passing Object Reference Variables	213
Does Java Use Pass-By-Value Semantics?	214
Passing Primitive Variables	215
Array Declaration, Construction, and Initialization	
(Exam Objective 1.3)	219
Declaring an Array	219
Constructing an Array	220
Initializing an Array	224
Initialization Blocks	234
Using Wrapper Classes and Boxing (Exam Objective 3.1)	237
An Overview of the Wrapper Classes	238
Creating Wrapper Objects	239
Using Wrapper Conversion Utilities	240
Autoboxing	244
Overloading (Exam Objectives 1.5 and 5.4)	247
Garbage Collection (Exam Objective 7.4)	254
Overview of Memory Management and	
Garbage Collection	254
Overview of Java's Garbage Collector	255
Writing Code That Explicitly Makes Objects Eligible	
for Collection	257
Exercise 3-2: Garbage Collection Experiment	262
✓ Two-Minute Drill	265
Q&A Self Test	269
Self Test Answers	277

4 Operators 287

Java Operators (Exam Objective 7.6)	288
Assignment Operators	288
Relational Operators	290
instanceof Comparison	295
Arithmetic Operators	298
Conditional Operator	304
Logical Operators	305
✓ Two-Minute Drill	311
Q&A Self Test	313
Self Test Answers	319

5	Flow Control, Exceptions, and Assertions	327
	if and switch Statements (Exam Objective 2.1)	328
	if-else Branching	329
	switch Statements	334
	Exercise 5-1: Creating a switch-case Statement	342
	Loops and Iterators (Exam Objective 2.2)	343
	Using while Loops	343
	Using do Loops	344
	Using for Loops	345
	Using break and continue	352
	Unlabeled Statements	353
	Labeled Statements	354
	Exercise 5-2: Creating a Labeled while Loop	356
	Handling Exceptions (Exam Objectives 2.4 and 2.5)	356
	Catching an Exception Using try and catch	357
	Using finally	359
	Propagating Uncaught Exceptions	362
	Exercise 5-3: Propagating and Catching	
	an Exception	364
	Defining Exceptions	365
	Exception Hierarchy	366
	Handling an Entire Class Hierarchy of Exceptions	368
	Exception Matching	369
	Exception Declaration and the Public Interface	371
	Rethrowing the Same Exception	376
	Exercise 5-4: Creating an Exception	377
	Common Exceptions and Errors(Exam Objective 2.6)	378
	Working with the Assertion Mechanism (Exam Objective 2.3)	383
	Assertions Overview	384
	Enabling Assertions	387
	Using Assertions Appropriately	391
	✓ Two-Minute Drill	397
Q&A	Self Test	401
	Self Test Answers	411

6	Strings, I/O, Formatting, and Parsing	425
	String, StringBuilder, and StringBuffer (Exam Objective 3.1)	426
	The String Class	426
	Important Facts About Strings and Memory	433
	Important Methods in the String Class	434
	The StringBuffer and StringBuilder Classes	438
	Important Methods in the StringBuffer and StringBuilder Classes	440
	File Navigation and I/O (Exam Objective 3.2)	443
	The java.io.Console Class	457
	Serialization (Exam Objective 3.3)	459
	Dates, Numbers, and Currency (Exam Objective 3.4)	473
	Working with Dates, Numbers, and Currencies	474
	Parsing, Tokenizing, and Formatting (Exam Objective 3.5)	487
	A Search Tutorial	488
	Locating Data via Pattern Matching	498
	Tokenizing	501
	Formatting with printf() and format()	506
	✓ Two-Minute Drill	511
	Q&A Self Test	515
	Self Test Answers	526
7	Generics and Collections	541
	Overriding hashCode() and equals() (Objective 6.2)	542
	Overriding equals()	544
	Overriding hashCode()	549
	Collections (Exam Objective 6.1)	556
	So What Do You Do with a Collection?	556
	List Interface	561
	Set Interface	562
	Map Interface	563
	Queue Interface	564
	Using the Collections Framework (Objectives 6.3 and 6.5)	566
	ArrayList Basics	567
	Autoboxing with Collections	568
	Sorting Collections and Arrays	568
	Navigating (Searching) TreeSets and TreeMaps	586
	Other Navigation Methods	587
	Backed Collections	589

Generic Types (Objectives 6.3 and 6.4)	595
Generics and Legacy Code	600
Mixing Generic and Non-generic Collections	601
Polymorphism and Generics	607
Generic Methods	609
Generic Declarations	622
✓ Two-Minute Drill	631
Q&A Self Test	636
Self Test Answers	647
8 Inner Classes	661
Inner Classes	663
Coding a "Regular" Inner Class	664
Referencing the Inner or Outer Instance from Within	
the Inner Class	668
Method-Local Inner Classes	670
What a Method-Local Inner Object Can and Can't Do	671
Anonymous Inner Classes	673
Plain-Old Anonymous Inner Classes, Flavor One	673
Plain-Old Anonymous Inner Classes, Flavor Two	677
Argument-Defined Anonymous Inner Classes	678
Static Nested Classes	680
Instantiating and Using Static Nested Classes	681
✓ Two-Minute Drill	683
Q&A Self Test	685
Self Test Answers	692
9 Threads	701
Defining, Instantiating, and Starting Threads (Objective 4.1)	702
Defining a Thread	705
Instantiating a Thread	706
Starting a Thread	709
Thread States and Transitions (Objective 4.2)	718
Thread States	718
Preventing Thread Execution	720
Sleeping	721
Exercise 9-1: Creating a Thread and	
Putting It to Sleep	723
Thread Priorities and yield()	724

Synchronizing Code (Objective 4.3)	728
Synchronization and Locks	735
Exercise 9-2: Synchronizing a Block of Code	738
Thread Deadlock	745
Thread Interaction (Objective 4.4)	746
Using notifyAll() When Many Threads	
May Be Waiting	752
✓ Two-Minute Drill	758
Q&A Self Test	761
Self Test Answers	772
Exercise Answers	787
10 Development	789
Using the javac and java Commands	
(Exam Objectives 7.1, 7.2, and 7.5)	790
Compiling with javac	790
Launching Applications with java	793
Searching for Other Classes	796
JAR Files (Objective 7.5)	802
JAR Files and Searching	803
Using Static Imports (Exam Objective 7.1)	806
Static Imports	806
✓ Two-Minute Drill	809
Q&A Self Test	811
Self Test Answers	820
A About the CD	831
System Requirements	832
Installing and Running Master Exam	832
Master Exam	832
Electronic Book	833
Help	833
Removing Installation(s)	833
Technical Support	833
LearnKey Technical Support	833
Index	835

ACKNOWLEDGMENTS

Kathy and Bert would like to thank the following people:

- All the incredibly hard-working folks at McGraw-Hill: Tim Green, Jim Kussow, Jody McKenzie, Madhu Bhardwaj, and Jennifer Housh for all their help, and for being so responsive and patient—well, okay, not all that patient—but so professional and the nicest group of people you could hope to work with.
- To our saviors Solveig Haugland and Midori Batten, for coming to our rescue when we were really in a bind!
- Some of the software professionals and friends who helped us in the early days: Tom Bender, Peter Loerincs, Craig Matthews, Dave Gustafson, Leonard Coyne, Morgan Porter, and Mike Kavenaugh.
- The wonderful and talented Certification team at Sun Educational Services, primarily the most persistent get-it-done person we know, Evelyn Cartagena.
- Our great friends and gurus, Bryan Basham, Kathy Collina, and Simon Roberts.
- To Eden and Skyler, for being horrified that adults—out of school— would study this hard for an exam.
- To the JavaRanch Trail Boss Paul Wheaton, for running the best Java community site on the Web, and to all the generous and patient JavaRanch moderators.
- To all the past and present Sun Ed Java instructors for helping to make learning Java a fun experience including (to name only a few): Alan Petersen, Jean Tordella, Georgianna Meagher, Anthony Orapallo, Jacqueline Jones, James Cubeta, Teri Cubeta, Rob Weingruber, John Nyquist, Asok Perumainar, Steve Stelting, Kimberly Bobrow, Keith Ratliff, and the most caring and inspiring Java guy on the planet, Jari Paukku.
- To Darren and Mary, thanks for keeping us sane and for helping us with our new furry friends Andi, Kara, Birta, Sola, Draumur, and Tjara.
- Finally, to Eric and Beth Freeman for your continued inspiration.

PREFACE

This book's primary objective is to help you prepare for and pass Sun Microsystem's SCJP certification for Java 6 or Java 5. The Java 6 and Java 5 exams are almost identical in scope, and they are both much broader than their predecessor, the Java 1.4 exam. For the remainder of this book we'll typically reference the Java 6 exam, but remember that other than the addition of the `System.Console` class and `Navigable` collections, the Java 5 and Java 6 exams are identical in scope. We recommend that you take the Java 6 exam and not the Java 5 exam, but if you do decide to take the Java 5 exam, this book is still appropriate. The new exam's objectives touch on many of the more commonly used of Java's APIs. The key word here is "touch." The exam's creators intended that passing the exam will demonstrate that the candidate understands the *basics* of APIs such as those for file I/O and regular expressions. This book follows closely both the breadth and the depth of the real exam. For instance, after reading this book, you probably won't emerge as a regex guru, but if you study the material, and do well on the self tests, you'll have a basic understanding of regex, and you'll do well on the exam. After completing this book, you should feel confident that you have thoroughly reviewed all of the objectives that Sun has established for the exam.

In This Book

This book is organized to optimize your learning of the topics covered by the SCJP Java 6 exam. Whenever possible, we've organized the chapters to parallel the Sun objectives, but sometimes we'll mix up objectives or partially repeat them in order to present topics in an order better suited to learning the material.

In addition to fully covering the SCJP Java 6 exam, we have also included on the CD eight chapters that cover important aspects of Sun's SCJD exam.

In Every Chapter

We've created a set of chapter components that call your attention to important items, reinforce important points, and provide helpful exam-taking hints. Take a look at what you'll find in every chapter:

- Every chapter begins with the **Certification Objectives**—what you need to know in order to pass the section on the exam dealing with the chapter topic.



The Certification Objective headings identify the objectives within the chapter, so you'll always know an objective when you see it!

- **Exam Watch** notes call attention to information about, and potential pitfalls in, the exam. Since we were on the team that created the exam, we know what you're about to go through!

- **On the Job** callouts discuss practical aspects of certification topics that might not occur on the exam, but that will be useful in the real world.

- **Exercises** are interspersed throughout the chapters. They help you master skills that are likely to be an area of focus on the exam. Don't just read through the exercises; they are hands-on practice that you should be comfortable completing. Learning by doing is an effective way to increase your competency with a product.

- **From the Classroom** sidebars describe the issues that come up most often in the training classroom setting. These sidebars give you a valuable perspective into certification- and product-related topics. They point out common mistakes and address questions that have arisen from classroom discussions.

- The **Certification Summary** is a succinct review of the chapter and a restatement of salient points regarding the exam.



- The **Two-Minute Drill** at the end of every chapter is a checklist of the main points of the chapter. It can be used for last-minute review.

Q&A

- The **Self Test** offers questions similar to those found on the certification exam, including multiple choice, and pseudo drag-and-drop questions. The answers to these questions, as well as explanations of the answers, can be found at the end of every chapter. By taking the Self Test after completing each chapter, you'll reinforce what you've learned from that chapter, while becoming familiar with the structure of the exam questions.

INTRODUCTION

Organization

This book is organized in such a way as to serve as an in-depth review for the Sun Certified Programmer for both the Java 6 and Java 5 exams, for experienced Java professionals and those in the early stages of experience with Java technologies. Each chapter covers at least one major aspect of the exam, with an emphasis on the "why" as well as the "how to" of programming in the Java language. The CD included with the book also includes an in-depth review of the essential ingredients for a successful assessment of a project submitted for the Sun Certified Java Developer exam.

What This Book Is Not

You will not find a beginner's guide to learning Java in this book. All 800 pages of this book are dedicated solely to helping you pass the exams. If you are brand new to Java, we suggest you spend a little time learning the basics. You shouldn't start with this book until you know how to write, compile, and run simple Java programs. We do not, however, assume any level of prior knowledge of the individual topics covered. In other words, for any given topic (driven exclusively by the actual exam objectives), we start with the assumption that you are new to that topic. So we assume you're new to the individual topics, but we assume that you are not new to Java.

We also do not pretend to be both preparing you for the exam and simultaneously making you a complete Java being. This is a certification exam study guide, and it's very clear about its mission. That's not to say that preparing for the exam won't help you become a better Java programmer! On the contrary, even the most experienced Java developers often claim that having to prepare for the certification exam made them far more knowledgeable and well-rounded programmers than they would have been without the exam-driven studying.

On the CD

For more information on the CD-ROM, please see Appendix A.

Some Pointers

Once you've finished reading this book, set aside some time to do a thorough review. You might want to return to the book several times and make use of all the methods it offers for reviewing the material:

1. Re-read all the Two-Minute Drills, or have someone quiz you. You also can use the drills as a way to do a quick cram before the exam. You might want to make some flash cards out of 3×5 index cards that have the Two-Minute Drill material on them.
2. Re-read all the Exam Watch notes. Remember that these notes are written by authors who helped create the exam. They know what you should expect—and what you should be on the lookout for.
3. Re-take the Self Tests. Taking the tests right after you've read the chapter is a good idea, because the questions help reinforce what you've just learned. However, it's an even better idea to go back later and do all the questions in the book in one sitting. Pretend that you're taking the live exam. (Whenever you take the self tests mark your answers on a separate piece of paper. That way, you can run through the questions as many times as you need to until you feel comfortable with the material.)
4. Complete the Exercises. The exercises are designed to cover exam topics, and there's no better way to get to know this material than by practicing. Be sure you understand why you are performing each step in each exercise. If there is something you are not clear on, re-read that section in the chapter.
5. Write lots of Java code. We'll repeat this advice several times. When we wrote this book, we wrote hundreds of small Java programs to help us do our research. We have heard from hundreds of candidates who have passed the exam, and in almost every case the candidates who scored extremely well on the exam wrote lots of code during their studies. Experiment with the code samples in the book, create horrendous lists of compiler errors—put away your IDE, crank up the command line, and write code!

Introduction to the Material in the Book

The Sun Certified Java Programmer (SCJP) exam is considered one of the hardest in the IT industry, and we can tell you from experience that a large chunk of exam candidates go in to the test unprepared. As programmers, we tend to learn only what we need to complete our current project, given the insane deadlines we're usually under.

But this exam attempts to prove your complete understanding of the Java language, not just the parts of it you've become familiar with in your work.

Experience alone will rarely get you through this exam with a passing mark, because even the things you think you know might work just a little different than you imagined. It isn't enough to be able to get your code to work correctly; you must understand the core fundamentals in a deep way, and with enough breadth to cover virtually anything that could crop up in the course of using the language.

The Sun Certified Developer Exam (covered in chapters that are contained on the CD) is unique to the IT certification realm, because it actually evaluates your skill as a developer rather than simply your knowledge of the language or tools. Becoming a Certified Java Developer is, by definition, a development experience.

Who Cares About Certification?

Employers do. Headhunters do. Programmers do. Sun's programmer exam has been considered the fastest-growing certification in the IT world, and the number of candidates taking the exam continues to grow each year. Passing this exam proves three important things to a current or prospective employer: you're smart; you know how to study and prepare for a challenging test; and, most of all, you know the Java language. If an employer has a choice between a candidate who has passed the exam and one who hasn't, the employer knows that the certified programmer does not have to take time to learn the Java language.

But does it mean that you can actually develop software in Java? Not necessarily, but it's a good head start. To really demonstrate your ability to develop (as opposed to just your knowledge of the language), you should consider pursuing the Developer Exam, where you're given an assignment to build a program, start to finish, and submit it for an assessor to evaluate and score.

Sun's Certification Program

Currently there are eight Java certification exams (although several of them might have more than one live version). The Associate exam, the Programmer exam, and the Developer exam are all associated with the Java Standard Edition. The Web Component exam, the Business Component exam, the Web Services exam, and the Enterprise Architect exam are all associated with the Java Enterprise Edition. The Mobile Application exam is associated with the Java Micro Edition.

The Associate, Programmer, Web Component, Business Component, Web Services, and Mobile Application exams are exclusively multiple-choice and drag-and-drop exams taken at a testing center, while the Developer and Architect exams also involve submitting a project.

The Associate Exam (CX-310-019)

Sun Certified Java Associate (SCJA)

The Associate exam is for candidates just entering an application development or a software project management career using Java technologies. This exam tests basic knowledge of object-oriented concepts, the basics of UML, the basics of the Java programming language, and general knowledge of Java Platforms and Technologies. This exam has no prerequisites.

The Programmer Exams (CX-310-065)

Sun Certified Java Programmer (SCJP) for Java 6

The Programmer exam is designed to test your knowledge of the Java programming language itself. It requires detailed knowledge of language syntax, core concepts, and a number of common application programming interfaces (APIs). This exam also tests intermediate knowledge of object-oriented design concepts. It does not test any issues related to architecture, and it does not ask why one approach is better than another, but rather it asks whether the given approach works in a particular situation. This exam has no prerequisites. As of May, 2008, two older versions of this exam are still available, the 1.4 and the 5.0.

The Developer Exam (CX-310-252A, CX-310-027)

Sun Certified Java Developer (SCJD)

The Developer exam picks up where the Programmer exam leaves off. Passing the Programmer exam is required before you can start the Developer exam. The Developer exam requires you to develop an actual program and then defend your design decisions. It is designed to test your understanding of why certain approaches are better than others in certain circumstances, and to prove your ability to follow a specification and implement a correct, functioning, and user-friendly program.

The Developer exam consists of two pieces: a project assignment and a follow-up essay exam. Candidates have an unlimited amount of time to complete the project, but once the project is submitted, the candidate then must go to a testing center and complete a short follow-up essay exam, designed primarily to validate and verify that it was you who designed and built the project.

The Web Component Developer Exam (CX-310-083)

Sun Certified Web Component Developer for Java EE Platform (SCWCD)

The web developer exam is for those who are using Java servlet and JSP (Java Server Pages) technologies to build Web applications. It's based on the Servlet and JSP specifications defined in the Java Enterprise Edition (Java EE). This exam requires that the candidate is a Sun Certified Java Programmer.

The Business Component Developer Exam (CX-310-091)

Sun Certified Business Component Developer for Java EE Platform (SCBCD)

The business component developer exam is for those candidates who are using Java EJB technology to build business-tier applications. The exam is based on the EJB specification defined in the Java Enterprise Edition (Java EE). This exam requires that the candidate is a Sun Certified Java Programmer.

The Web Services Developer Exam (CX-310-220)

Sun Certified Developer for Web Services for Java EE Platform (SCDJWS)

The web services exam is for those candidates who are building applications using Java EE and Java Web Services Developer Pack technologies. This exam requires that the candidate is a Sun Certified Java Programmer.

The Architect Exam (CX-310-052, CX-310-301A, CX-310-062)

Sun Certified Enterprise Architect for J2EE Technology (SCEA)

This certification is for enterprise architects, and thus does not require that the candidate pass the Programmer exam. The Architect exam is in three pieces: a knowledge-based multiple-choice exam, an architectural design assignment, and a follow-up essay exam. You must successfully pass the multiple-choice exam before registering and receiving the design assignment.

The Mobile Exam (CX-310-110)

Sun Certified Mobile Application Developer for Java ME (SCMAD)

The mobile application developer exam is for candidates creating applications for cell phones or other Java enabled devices. The exam covers the Java Technology for Wireless Industry (JTWI) specification, the Wireless Messaging API, and Mobile Media APIs. This exam requires that the candidate is an SCJP.

Taking the Programmer's Exam

In a perfect world, you would be assessed for your true knowledge of a subject, not simply how you respond to a series of test questions. But life isn't perfect, and it just isn't practical to evaluate everyone's knowledge on a one-to-one basis.

For the majority of its certifications, Sun evaluates candidates using a computer-based testing service operated by Sylvan Prometric. This service is quite popular in the industry, and it is used for a number of vendor certification programs, including Novell's CNE and Microsoft's MCSE. Thanks to Sylvan Prometric's large number of facilities, exams can be administered worldwide, generally in the same town as a prospective candidate.

For the most part, Sylvan Prometric exams work similarly from vendor to vendor. However, there is an important fact to know about Sun's exams: they use the traditional Sylvan Prometric test format, not the newer adaptive format. This gives the candidate an advantage, since the traditional format allows answers to be reviewed and revised during the test.

exam

Watch

Many experienced test takers do not go back and change answers unless they have a good reason to do so. Only change an answer when you feel you may have misread or misinterpreted the question the first time. Nervousness may make you second-guess every answer and talk yourself out of a correct one.

To discourage simple memorization, Sun exams present a potentially different set of questions to different candidates. In the development of the exam, hundreds of questions are compiled and refined using beta testers. From this large collection, questions are pulled together from each objective and assembled into many different versions of the exam.

Each Sun exam has a specific number of questions (the Programmer's exam contains 72 questions) and test duration (210 minutes for the Programmer's exam) is designed to be generous. The time remaining is always displayed in the corner of the testing screen, along with the number of remaining questions. If time expires during an exam, the test terminates, and incomplete answers are counted as incorrect.

At the end of the exam, your test is immediately graded, and the results are displayed on the screen. Scores for each subject area are also provided, but the system will not indicate which specific questions were missed. A report is automatically printed at the proctor's desk for your files. The test score is electronically transmitted back to Sun.

exam

Watch

When you find yourself stumped answering multiple-choice questions, use your scratch paper to write down the two or three answers you consider the strongest, then underline the answer you feel is most likely correct. Here is an example of what your scratch paper might look like when you've gone through the test once:

21. B or C

33. A or C

This is extremely helpful when you mark the question and continue on. You can then return to the question and immediately pick up your thought process where you left off. Use this technique to avoid having to re-read and re-think questions. You will also need to use your scratch paper during complex, text-based scenario questions to create visual images to better understand the question. This technique is especially helpful if you are a visual learner.

Question Format

Sun's Java exams pose questions in either multiple-choice or drag-and-drop formats.

Multiple Choice Questions

In earlier versions of the exam, when you encountered a multiple-choice question you were not told how many answers were correct, but with each version of the exam, the questions have become more difficult, so today each multiple-choice question tells you how many answers to choose. The Self Test questions at the end of each chapter are closely matched to the format, wording, and difficulty of the real exam questions, with two exceptions:

- Whenever we can, our questions will NOT tell you how many correct answers exist (we will say "Choose all that apply"). We do this to help you master the material. Some savvy test-takers can eliminate wrong answers when the number of correct answers is known. It's also possible, if you know how many answers are correct, to choose the most plausible answers. Our job is to toughen you up for the real exam!
- The real exam typically numbers lines of code in a question. Sometimes we do not number lines of code—mostly so that we have the space to add comments at key places. On the real exam, when a code listing starts with line 1, it means that you're looking at an entire source file. If a code listing starts at a line number greater than 1, that means you're looking at a partial source file. When looking at a partial source file, assume that the code you can't see is correct. (For instance, unless explicitly stated, you can assume that a partial source file will have the correct import and package statements.)

Drag-and-Drop Questions

Although many of the other Sun Java certification exams have been using drag-and-drop questions for several years, this is the first version of the SCJP exam that includes drag-and-drop questions. As we discussed earlier, the exam questions you receive are randomized, but you should expect that about 20–25% of the questions you encounter will be drag-and-drop style.

Drag-and-drop questions typically consist of three components:

- **A scenario** A short description of the task you are meant to complete.
- **A partially completed task** A code listing, a table, or a directory tree. The partially completed task will contain empty slots, which are indicated with (typically yellow) boxes. These boxes need to be filled to complete the task.
- **A set of possible "fragment" answers** You will click on fragments (typically blue boxes) and drag-and-drop them into the correct empty slots. The question's scenario will tell you whether you can reuse fragments.

Most drag-and-drop questions will have anywhere from 4 to 10 empty slots to fill, and typically a few more fragments than are needed (usually some fragments are left unused). Drag-and-drop questions are often the most complex on the exam, and the number of possible answer combinations makes them almost impossible to guess.

exam

Watch

In regards to drag-and-drop questions, there is a huge problem with the testing software at many of the Prometric centers world-wide. In general, the testing software allows you to review questions you've already answered as often as you'd like.

In the case of drag-and-drop questions, however, many candidates have reported that if they choose to review a question, the software will erase their previous answer! BE CAREFUL! Until this problem is corrected, we recommend that you keep a list of which questions are drag and drop, so that you won't review one unintentionally. Another good idea is to write down your drag-and-drop answers so that if one gets erased it will be less painful to recreate the answer.

This brings us to another issue that some candidates have reported. The testing center is supposed to provide you with sufficient writing implements so that you can work problems out "on paper." In some cases, the centers have provided inadequate markers and dry-erase boards which are too small and cumbersome to use effectively. We recommend that you call ahead and verify that you will be supplied with actual pencils and at least several sheets of blank paper.

Tips on Taking the Exam

There are 72 questions on the 310-065 (Java 6) exam. You will need to get at least 47 of them correct to pass—around 65%. You are given over three hours to complete the exam. This information is subject to change. Always check with Sun before taking the exam, at www.suned.sun.com.

You are allowed to answer questions in any order, and you can go back and check your answers after you've gone through the test. There are no penalties for wrong answers, so it's better to at least attempt an answer than to not give one at all.

A good strategy for taking the exam is to go through once and answer all the questions that come to you quickly. You can then go back and do the others. Answering one question might jog your memory for how to answer a previous one.

Be very careful on the code examples. Check for syntax errors first: count curly braces, semicolons, and parenthesis and then make sure there are as many left ones as right ones. Look for capitalization errors and other such syntax problems before trying to figure out what the code does.

Many of the questions on the exam will hinge on subtleties of syntax. You will need to have a thorough knowledge of the Java language in order to succeed.

Tips on Studying for the Exam

First and foremost, give yourself plenty of time to study. Java is a complex programming language, and you can't expect to cram what you need to know into a single study session. It is a field best learned over time, by studying a subject and then applying your knowledge. Build yourself a study schedule and stick to it, but be reasonable about the pressure you put on yourself, especially if you're studying in addition to your regular duties at work.

One easy technique to use in studying for certification exams is the 15-minutes-per-day effort. Simply study for a minimum of 15 minutes every day. It is a small but significant commitment. If you have a day where you just can't focus, then give up at 15 minutes. If you have a day where it flows completely for you, study longer. As long as you have more of the "flow days," your chances of succeeding are excellent.

We strongly recommend you use flash cards when preparing for the Programmer's exam. A flash card is simply a 3 x 5 or 4 x 6 index card with a question on the front, and the answer on the back. You construct these cards yourself as you go through a chapter, capturing any topic you think might need more memorization or practice time. You can drill yourself with them by reading the question, thinking through the answer, and then turning the card over to see if you're correct. Or you can get another person to help you by holding up the card with the question facing you, and then verifying your answer. Most of our students have found these to be tremendously helpful, especially because they're so portable that while you're in study mode, you can take them everywhere. Best not to use them while driving, though, except at red lights. We've taken ours everywhere—the doctor's office, restaurants, theaters, you name it.

Certification study groups are another excellent resource, and you won't find a larger or more willing community than on the JavaRanch.com Big Moose Saloon certification forums. If you have a question from this book, or any other mock exam question you may have stumbled upon, posting a question in a certification forum will get you an answer, in nearly all cases, within a day—usually, within a few hours. You'll find us (the authors) there several times a week, helping those just starting out on their exam preparation journey. (You won't actually think of it as anything as pleasant-sounding as a "journey" by the time you're ready to take the exam.)

Finally, we recommend that you write a lot of little Java programs! During the course of writing this book we wrote hundreds of small programs, and if you listen to what the most successful candidates say (you know, those guys who got 98%), they almost always report that they wrote a lot of code.

Scheduling Your Exam

The Sun exams are purchased directly from Sun, but are scheduled through Sylvan Prometric. For locations outside the United States, your local number can be found on Sylvan's Web site at <http://www.2test.com>. Sylvan representatives can schedule your exam, but they don't have information about the certification programs. Questions about certifications should be directed to Sun's Worldwide Training department. These representatives are familiar enough with the exams to find them by name, but it's best if you have the exam number handy when you call. You wouldn't want to be scheduled and charged for the wrong exam.

Exams can be scheduled up to a year in advance, although it's really not necessary. Generally, scheduling a week or two ahead is sufficient to reserve the day and time you prefer. When scheduling, operators will search for testing centers in your area. For convenience, they can also tell which testing centers you've used before.

When registering for the exam, you will be asked for your ID number. This number is used to track your exam results back to Sun. It's important that you use the same ID number each time you register, so that Sun can follow your progress. Address information provided when you first register is also used by Sun to ship certificates and other related material. In the United States, your Social Security Number is commonly used as your ID number. However, Sylvan can assign you a unique ID number if you prefer not to use your Social Security Number.

Arriving at the Exam

As with any test, you'll be tempted to cram the night before. Resist that temptation. You should know the material by this point, and if you're groggy in the morning, you won't remember what you studied anyway. Get a good night's sleep.

Arrive early for your exam; it gives you time to relax and review key facts. Take the opportunity to review your notes. If you get burned out on studying, you can usually start your exam a few minutes early. We don't recommend arriving late. Your test could be cancelled, or you might not have enough time to complete the exam.

When you arrive at the testing center, you'll need to sign in with the exam administrator. In order to sign in, you need to provide two forms of identification. Acceptable forms include government-issued IDs (for example, passport or driver's license), credit cards, and company ID badges. One form of ID must include a photograph. They just want to be sure that you don't send your brilliant Java guru next-door-neighbor-who-you've-paid to take the exam for you.

Aside from a brain full of facts, you don't need to bring anything else to the exam room. In fact, your brain is about all you're allowed to take into the exam!

All the tests are closed-book, meaning you don't get to bring any reference materials with you. You're also not allowed to take any notes out of the exam room. The test administrator will provide you with paper and a pencil. Some testing centers may provide a small marker board instead (we recommend that you don't settle for a whiteboard). We do recommend that you bring a water bottle. Three hours is a long time to keep your brain active, and it functions much better when well hydrated.

Leave your pager and telephone in the car, or turn them off. They only add stress to the situation, since they are not allowed in the exam room, and can sometimes still be heard if they ring outside of the room. Purses, books, and other materials must be left with the administrator before entering the exam.

Once in the testing room, the exam administrator logs onto your exam, and you have to verify that your ID number and the exam number are correct. If this is the first time you've taken a Sun test, you can select a brief tutorial of the exam software. Before the test begins, you will be provided with facts about the exam, including the duration, the number of questions, and the score required for passing. The odds are good that you will be asked to fill out a brief survey before the exam actually begins. This survey will ask you about your level of Java experience. The time you spend on the survey is NOT deducted from your actual test time—nor do you get more time if you fill out the survey quickly. Also remember that the questions you get on the exam will NOT change depending on how you answer the survey questions. Once you're done with the survey, the real clock starts ticking and the fun begins.

The testing software is Windows-based, but you won't have access to the main desktop or any of the accessories. The exam is presented in full screen, with a single question per screen. Navigation buttons allow you to move forward and backward between questions. In the upper-right corner of the screen, counters show the number of questions and time remaining. Most important, there is a Mark check box in the upper-left corner of the screen—this will prove to be a critical tool, as explained in the next section.

Test-Taking Techniques

Without a plan of attack, candidates can become overwhelmed by the exam or become side-tracked and run out of time. For the most part, if you are comfortable with the material, the allotted time is more than enough to complete the exam. The trick is to keep the time from slipping away during any one particular problem.

Your obvious goal is to answer the questions correctly and quickly, but other factors can distract you. Here are some tips for taking the exam more efficiently.

Size Up the Challenge

First, take a quick pass through all the questions in the exam. "Cherry-pick" the easy questions, answering them on the spot. Briefly read each question, noticing the type of question and the subject. As a guideline, try to spend less than 25 percent of your testing time in this pass.

This step lets you assess the scope and complexity of the exam, and it helps you determine how to pace your time. It also gives you an idea of where to find potential answers to some of the questions. Sometimes the wording of one question might lend clues or jog your thoughts for another question.

If you're not entirely confident in your answer to a question, answer it anyway, but check the Mark box to flag it for later review. In the event that you run out of time, at least you've provided a "first guess" answer, rather than leaving it blank.

Second, go back through the entire test, using the insight you gained from the first go-through. For example, if the entire test looks difficult, you'll know better than to spend more than a minute or two on each question. Create a pacing with small milestones—for example, "I need to answer 10 questions every 25 minutes."

At this stage, it's probably a good idea to skip past the time-consuming questions, marking them for the next pass. Try to finish this phase before you're 50–60 percent through the testing time.

Third, go back through all the questions you marked for review, using the Review Marked button in the question review screen. This step includes taking a second look at all the questions you were unsure of in previous passes, as well as tackling the time-consuming ones you deferred until now. Chisel away at this group of questions until you've answered them all.

If you're more comfortable with a previously marked question, unmark the Review Marked button now. Otherwise, leave it marked. Work your way through the time-consuming questions now, especially those requiring manual calculations. Unmark them when you're satisfied with the answer.

By the end of this step, you've answered every question in the test, despite having reservations about some of your answers. If you run out of time in the next step, at least you won't lose points for lack of an answer. You're in great shape if you still have 10–20 percent of your time remaining.

Review Your Answers

Now you're cruising! You've answered all the questions, and you're ready to do a quality check. Take yet another pass (yes, one more) through the entire test

(although you'll probably want to skip a review of the drag-and-drop questions!), briefly re-reading each question and your answer.

Carefully look over the questions again to check for "trick" questions. Be particularly wary of those that include a choice of "Does not compile." Be alert for last-minute clues. You're pretty familiar with nearly every question at this point, and you may find a few clues that you missed before.

The Grand Finale

When you're confident with all your answers, finish the exam by submitting it for grading. After what will seem like the longest 10 seconds of your life, the testing software will respond with your score. This is usually displayed as a bar graph, showing the minimum passing score, your score, and a PASS/FAIL indicator.

If you're curious, you can review the statistics of your score at this time. Answers to specific questions are not presented; rather, questions are lumped into categories, and results are tallied for each category. This detail is also on a report that has been automatically printed at the exam administrator's desk.

As you leave, you'll need to leave your scratch paper behind or return it to the administrator. (Some testing centers track the number of sheets you've been given, so be sure to return them all.) In exchange, you'll receive a copy of the test report.

This report will be embossed with the testing center's seal, and you should keep it in a safe place. Normally, the results are automatically transmitted to Sun, but occasionally you might need the paper report to prove that you passed the exam.

In a few weeks, Sun will send you a package in the mail containing a nice paper certificate, a lapel pin, and a letter. You may also be sent instructions for how to obtain artwork for a logo that you can use on personal business cards.

Re-Testing

If you don't pass the exam, don't be discouraged. Try to have a good attitude about the experience, and get ready to try again. Consider yourself a little more educated. You know the format of the test a little better, and the report shows which areas you need to strengthen.

If you bounce back quickly, you'll probably remember several of the questions you might have missed. This will help you focus your study efforts in the right area.

Ultimately, remember that Sun certifications are valuable because they're hard to get. After all, if anyone could get one, what value would it have? In the end, it takes a good attitude and a lot of studying, but you can do it!



I

Declarations and Access Control

CERTIFICATION OBJECTIVES

- Declare Classes & Interfaces
- Develop Interfaces & Abstract Classes
- Use Primitives, Arrays, Enums, & Legal Identifiers
- Use Static Methods, JavaBeans Naming, & Var-Args
- ✓ Two-Minute Drill
- Q&A Self Test

We assume that because you're planning on becoming certified, you already know the basics of Java. If you're completely new to the language, this chapter—and the rest of the book—will be confusing; so be sure you know at least the basics of the language before diving into this book. That said, we're starting with a brief, high-level refresher to put you back in the Java mood, in case you've been away for awhile.

Java Refresher

A Java program is mostly a collection of *objects* talking to other objects by invoking each other's *methods*. Every object is of a certain *type*, and that type is defined by a *class* or an *interface*. Most Java programs use a collection of objects of many different types.

- **Class** A template that describes the kinds of state and behavior that objects of its type support.
- **Object** At runtime, when the Java Virtual Machine (JVM) encounters the new keyword, it will use the appropriate class to make an object which is an instance of that class. That object will have its own state, and access to all of the behaviors defined by its class.
- **State (instance variables)** Each object (instance of a class) will have its own unique set of instance variables as defined in the class. Collectively, the values assigned to an object's instance variables make up the object's state.
- **Behavior (methods)** When a programmer creates a class, she creates methods for that class. Methods are where the class' logic is stored. Methods are where the real work gets done. They are where algorithms get executed, and data gets manipulated.

Identifiers and Keywords

All the Java components we just talked about—classes, variables, and methods—need names. In Java these names are called *identifiers*, and, as you might expect, there are rules for what constitutes a legal Java identifier. Beyond what's *legal*,

though, Java programmers (and Sun) have created *conventions* for naming methods, variables, and classes.

Like all programming languages, Java has a set of built-in *keywords*. These keywords must *not* be used as identifiers. Later in this chapter we'll review the details of these naming rules, conventions, and the Java keywords.

Inheritance

Central to Java and other object-oriented (OO) languages is the concept of *inheritance*, which allows code defined in one class to be reused in other classes. In Java, you can define a general (more abstract) superclass, and then extend it with more specific subclasses. The superclass knows nothing of the classes that inherit from it, but all of the subclasses that inherit from the superclass must explicitly declare the inheritance relationship. A subclass that inherits from a superclass is automatically given accessible instance variables and methods defined by the superclass, but is also free to override superclass methods to define more specific behavior.

For example, a *Car* superclass class could define general methods common to all automobiles, but a *Ferrari* subclass could override the `accelerate()` method.

Interfaces

A powerful companion to inheritance is the use of interfaces. Interfaces are like a 100-percent abstract superclass that defines the methods a subclass must support, but not *how* they must be supported. In other words, an `Animal` interface might declare that all `Animal` implementation classes have an `eat()` method, but the `Animal` interface doesn't supply any logic for the `eat()` method. That means it's up to the classes that implement the `Animal` interface to define the actual code for how that particular `Animal` type behaves when its `eat()` method is invoked.

Finding Other Classes

As we'll see later in the book, it's a good idea to make your classes *cohesive*. That means that every class should have a focused set of responsibilities. For instance, if you were creating a zoo simulation program, you'd want to represent aardvarks with one class, and zoo visitors with a different class. In addition, you might have a `Zookeeper` class, and a `Popcorn` vendor class. The point is that you don't want a class that has both *Aardvark* and *Popcorn* behaviors (more on that in Chapter 2).

Even a simple Java program uses objects from many different classes: some that you created, and some built by others (such as Sun's Java API classes). Java organizes classes into *packages*, and uses *import* statements to give programmers a consistent

way to manage naming of, and access to, classes they need. The exam covers a lot of concepts related to packages and class access; we'll explore the details in this—and later—chapters.

CERTIFICATION OBJECTIVE

Identifiers & JavaBeans (Objectives 1.3 and 1.4)

1.3 Develop code that declares, initializes, and uses primitives, arrays, enums, and objects as static, instance, and local variables. Also, use legal identifiers for variable names.

1.4 Develop code that declares both static and non-static methods, and—if appropriate—use method names that adhere to the JavaBeans naming standards. Also develop code that declares and uses a variable-length argument list.

Remember that when we list one or more Certification Objectives in the book, as we just did, it means that the following section covers at least some part of that objective. Some objectives will be covered in several different chapters, so you'll see the same objective in more than one place in the book. For example, this section covers declarations, identifiers, and JavaBeans naming, but *using* the things you declare is covered primarily in later chapters.

So, we'll start with Java identifiers. The three aspects of Java identifiers that we cover here are

- **Legal Identifiers** The rules the compiler uses to determine whether a name is legal.
- **Sun's Java Code Conventions** Sun's recommendations for naming classes, variables, and methods. We typically adhere to these standards throughout the book, except when we're trying to show you how a tricky exam question might be coded. You won't be asked questions about the Java Code Conventions, but we strongly recommend that programmers use them.
- **JavaBeans Naming Standards** The naming requirements of the JavaBeans specification. You don't need to study the JavaBeans spec for the exam, but you do need to know a few basic JavaBeans naming rules we cover in this chapter.

Legal Identifiers

Technically, legal identifiers must be composed of only Unicode characters, numbers, currency symbols, and connecting characters (like underscores). The exam doesn't dive into the details of which ranges of the Unicode character set are considered to qualify as letters and digits. So, for example, you won't need to know that Tibetan digits range from `\u0420` to `\u0f29`. Here are the rules you *do* need to know:

- Identifiers must start with a letter, a currency character (\$), or a connecting character such as the underscore (_). Identifiers cannot start with a number!
- After the first character, identifiers can contain any combination of letters, currency characters, connecting characters, or numbers.
- In practice, there is no limit to the number of characters an identifier can contain.
- You can't use a Java keyword as an identifier. Table 1-1 lists all of the Java keywords including one new one for 5.0, `enum`.
- Identifiers in Java are case-sensitive; `foo` and `FOO` are two different identifiers.

Examples of legal and illegal identifiers follow, first some legal identifiers:

```
int _a;
int $c;
int _____2_w;
int _$;
int this_is_a_very_detailed_name_for_an_identifier;
```

The following are illegal (it's your job to recognize why):

```
int :b;
int -d;
int e#;
int .f;
int 7g;
```

TABLE 1-1 Complete List of Java Keywords (assert added in 1.4, enum added in 1.5)

abstract	boolean	break	byte	case	catch
char	class	const	continue	default	do
double	else	extends	final	finally	float
for	goto	if	implements	import	instanceof
int	interface	long	native	new	package
private	protected	public	return	short	static
strictfp	super	switch	synchronized	this	throw
throws	transient	try	void	volatile	while
assert	enum				

Sun's Java Code Conventions

Sun estimates that over the lifetime of a standard piece of code, 20 percent of the effort will go into the original creation and testing of the code, and 80 percent of the effort will go into the subsequent maintenance and enhancement of the code. Agreeing on, and coding to, a set of code standards helps to reduce the effort involved in testing, maintaining, and enhancing any piece of code. Sun has created a set of coding standards for Java, and published those standards in a document cleverly titled "Java Code Conventions," which you can find at java.sun.com. It's a great document, short and easy to read and we recommend it highly.

That said, you'll find that many of the questions in the exam don't follow the code conventions, because of the limitations in the test engine that is used to deliver the exam internationally. One of the great things about the Sun certifications is that the exams are administered uniformly throughout the world. In order to achieve that, the code listings that you'll see in the real exam are often quite cramped, and do not follow Sun's code standards. In order to toughen you up for the exam, we'll often present code listings that have a similarly cramped look and feel, often indenting our code only two spaces as opposed to the Sun standard of four.

We'll also jam our curly braces together unnaturally, and sometimes put several statements on the same line...ouch! For example:

```

1. class Wombat implements Runnable {
2.     private int i;
3.     public synchronized void run() {
4.         if (i%5 != 0) { i++; }
5.         for(int x=0; x<5; x++, i++)

```

```

6.      { if (x > 1) Thread.yield(); }
7.      System.out.print(i + " ");
8.    }
9.    public static void main(String[] args) {
10.        Wombat n = new Wombat();
11.        for(int x=100; x>0; --x) { new Thread(n).start(); }
12.    } }

```

Consider yourself forewarned—you'll see lots of code listings, mock questions, and real exam questions that are this sick and twisted. Nobody wants you to write your code like this. Not your employer, not your coworkers, not us, not Sun, and not the exam creation team! Code like this was created only so that complex concepts could be tested within a universal testing tool. The one standard that is followed as much as possible in the real exam are the naming standards. Here are the naming standards that Sun recommends, and that we use in the exam and in most of the book:

- **Classes and interfaces** The first letter should be capitalized, and if several words are linked together to form the name, the first letter of the inner words should be uppercase (a format that's sometimes called "camelCase"). For classes, the names should typically be nouns. For example:

```

Dog
Account
PrintWriter

```

For interfaces, the names should typically be adjectives like

```

Runnable
Serializable

```

- **Methods** The first letter should be lowercase, and then normal camelCase rules should be used. In addition, the names should typically be verb-noun pairs. For example:

```

getBalance
doCalculation
setCustomerName

```

- **Variables** Like methods, the camelCase format should be used, starting with a lowercase letter. Sun recommends short, meaningful names, which sounds good to us. Some examples:

```
buttonWidth  
accountBalance  
myString
```

- **Constants** Java constants are created by marking variables `static` and `final`. They should be named using uppercase letters with underscore characters as separators:

```
MIN_HEIGHT
```

JavaBeans Standards

The JavaBeans spec is intended to help Java developers create Java components that can be easily used by other Java developers in a visual Integrated Development Environment (IDE) tool (like Eclipse or NetBeans). As a Java programmer, you want to be able to use components from the Java API, but it would be great if you could also buy the Java component you want from "Beans 'R Us," that software company down the street. And once you've found the components, you'd like to be able to access them through a development tool in such a way that you don't have to write all your code from scratch. By using naming rules, the JavaBeans spec helps guarantee that tools can recognize and use components built by different developers. The JavaBeans API is quite involved, but you'll need to study only a few basics for the exam.

First, JavaBeans are Java classes that have *properties*. For our purposes, think of properties as `private` instance variables. Since they're `private`, the only way they can be accessed from outside of their class is through *methods* in the class. The methods that change a property's value are called *setter* methods, and the methods that retrieve a property's value are called *getter* methods. The JavaBean naming rules that you'll need to know for the exam are the following:

JavaBean Property Naming Rules

- If the property is not a boolean, the getter method's prefix must be *get*. For example, `getSize()` is a valid JavaBeans getter name for a property named "size." Keep in mind that you do not need to have a variable named *size*

(although some IDEs expect it). The name of the property is *inferred* from the getters and setters, not through any variables in your class. What you return from `getSize()` is up to you.

- If the property is a boolean, the getter method's prefix is either `get` or `is`. For example, `getStopped()` or `isStopped()` are both valid JavaBeans names for a boolean property.
- The setter method's prefix must be `set`. For example, `setSize()` is the valid JavaBean name for a property named `size`.
- To complete the name of a getter or setter method, change the first letter of the property name to uppercase, and then append it to the appropriate prefix (`get`, `is`, or `set`).
- Setter method signatures must be marked `public`, with a `void` return type and an argument that represents the property type.
- Getter method signatures must be marked `public`, take no arguments, and have a return type that matches the argument type of the setter method for that property.

Second, the JavaBean spec supports *events*, which allow components to notify each other when something happens. The event model is often used in GUI applications when an event like a mouse click is multicast to many other objects that may have things to do when the mouse click occurs. The objects that receive the information that an event occurred are called *listeners*. For the exam, you need to know that the methods that are used to add or remove listeners from an event must also follow JavaBean naming standards:

JavaBean Listener Naming Rules

- Listener method names used to "register" a listener with an event source must use the prefix `add`, followed by the listener type. For example, `addActionListener()` is a valid name for a method that an event source will have to allow others to register for Action events.
- Listener method names used to remove ("unregister") a listener must use the prefix `remove`, followed by the listener type (using the same rules as the registration `add` method).
- The type of listener to be added or removed must be passed as the argument to the method.
- Listener method names must end with the word "Listener".

Examples of valid JavaBean method signatures are

```
public void setMyValue(int v)
public int getMyValue()
public boolean isMyStatus()
public void addMyListener(MyListener m)
public void removeMyListener(MyListener m)
```

Examples of *invalid* JavaBean method signatures are

```
void setCustomerName(String s)           // must be public
public void modifyMyValue(int v)         // can't use 'modify'
public void addXListener(MyListener m)   // listener type mismatch
```

exam

Watch

The objective says you have to know legal identifiers only for variable names, but the rules are the same for ALL Java components. So remember that a legal identifier for a variable is also a legal identifier for a method or a class. However, you need to distinguish between legal identifiers and naming conventions, such as the JavaBeans standards, that indicate how a Java component should be named. In other words, you must be able to recognize that an identifier is legal even if it doesn't conform to naming standards. If the exam question is asking about naming conventions—not just whether an identifier will compile—JavaBeans will be mentioned explicitly.

CERTIFICATION OBJECTIVE

Declare Classes (Exam Objective 1.1)

1.1 Develop code that declares classes (including abstract and all forms of nested classes), interfaces, and enums, and includes the appropriate use of package and import statements (including static imports).

When you write code in Java, you're writing classes or interfaces. Within those classes, as you know, are variables and methods (plus a few other things). How you declare your classes, methods, and variables dramatically affects your code's behavior. For example, a `public` method can be accessed from code running anywhere in your application. Mark that method `private`, though, and it vanishes from everyone's radar (except the class in which it was declared). For this objective, we'll study the ways in which you can declare and modify (or not) a class. You'll find that we cover modifiers in an extreme level of detail, and though we know you're already familiar with them, we're starting from the very beginning. Most Java programmers think they know how all the modifiers work, but on closer study often find out that they don't (at least not to the degree needed for the exam). Subtle distinctions are everywhere, so you need to be absolutely certain you're completely solid on everything in this section's objectives before taking the exam.

Source File Declaration Rules

Before we dig into class declarations, let's do a quick review of the rules associated with declaring classes, `import` statements, and `package` statements in a source file:

- There can be only one `public` class per source code file.
- Comments can appear at the beginning or end of any line in the source code file; they are independent of any of the positioning rules discussed here.
- If there is a `public` class in a file, the name of the file must match the name of the `public` class. For example, a class declared as `public class Dog { }` must be in a source code file named `Dog.java`.
- If the class is part of a package, the `package` statement must be the first line in the source code file, before any `import` statements that may be present.
- If there are `import` statements, they must go *between* the `package` statement (if there is one) and the class declaration. If there isn't a `package` statement, then the `import` statement(s) must be the first line(s) in the source code file. If there are no `package` or `import` statements, the class declaration must be the first line in the source code file.
- `import` and `package` statements apply to *all* classes within a source code file. In other words, there's no way to declare multiple classes in a file and have them in different packages, or use different imports.
- A file can have more than one nonpublic class.

- Files with no public classes can have a name that does not match any of the classes in the file.

In Chapter 10 we'll go into a lot more detail about the rules involved with declaring and using imports, packages, and a feature new to Java 5, static imports.

Class Declarations and Modifiers

Although nested (often called inner) classes are on the exam, we'll save nested class declarations for Chapter 8. You're going to love that chapter. No, really. Seriously. The following code is a bare-bones class declaration:

```
class MyClass { }
```

This code compiles just fine, but you can also add modifiers before the class declaration. Modifiers fall into two categories:

- Access modifiers: `public`, `protected`, `private`.
- Non-access modifiers (including `strictfp`, `final`, and `abstract`).

We'll look at access modifiers first, so you'll learn how to restrict or allow access to a class you create. Access control in Java is a little tricky because there are four access *controls* (levels of access) but only three access *modifiers*. The fourth access control level (called *default* or *package* access) is what you get when you don't use any of the three access modifiers. In other words, *every* class, method, and instance variable you declare has an access *control*, whether you explicitly type one or not. Although all four access *controls* (which means all three *modifiers*) work for most method and variable declarations, a class can be declared with only `public` or *default* access; the other two access control levels don't make sense for a class, as you'll see.



Java is a package-centric language; the developers assumed that for good organization and name scoping, you would put all your classes into packages. They were right, and you should. Imagine this nightmare: Three different programmers, in the same company but working on different parts of a project, write a class named Utilities. If those three Utilities classes have

not been declared in any explicit package, and are in the classpath, you won't have any way to tell the compiler or JVM which of the three you're trying to reference. Sun recommends that developers use reverse domain names, appended with division and/or project names. For example, if your domain name is `geeksanonymous.com`, and you're working on the client code for the `TwelvePointOSteps` program, you would name your package something like `com.geeksanonymous.steps.client`. That would essentially change the name of your class to `com.geeksanonymous.steps.client.Utilities`. You might still have name collisions within your company, if you don't come up with your own naming schemes, but you're guaranteed not to collide with classes developed outside your company (assuming they follow Sun's naming convention, and if they don't, well, Really Bad Things could happen).

Class Access

What does it mean to access a class? When we say code from one class (class A) has access to another class (class B), it means class A can do one of three things:

- Create an *instance* of class B.
- Extend class B (in other words, become a subclass of class B).
- Access certain methods and variables within class B, depending on the access control of those methods and variables.

In effect, access means *visibility*. If class A can't *see* class B, the access level of the methods and variables within class B won't matter; class A won't have any way to access those methods and variables.

Default Access A class with default access has *no* modifier preceding it in the declaration! It's the access control you get when you don't type a modifier in the class declaration. Think of *default* access as *package*-level access, because a class with default access can be seen only by classes within the same package. For example, if class A and class B are in different packages, and class A has default access, class B won't be able to create an instance of class A, or even declare a variable or return type of class A. In fact, class B has to pretend that class A doesn't even exist, or the compiler will complain. Look at the following source file:

14 Chapter 1: Declarations and Access Control

```
package cert;  
class Beverage { }
```

Now look at the second source file:

```
package exam.stuff;  
import cert.Beverage;  
class Tea extends Beverage { }
```

As you can see, the superclass (Beverage) is in a different package from the subclass (Tea). The `import` statement at the top of the Tea file is trying (fingers crossed) to import the Beverage class. The Beverage file compiles fine, but when we try to compile the Tea file we get something like:

```
Can't access class cert.Beverage. Class or interface must be  
public, in same package, or an accessible member class.  
import cert.Beverage;
```

Tea won't compile because its superclass, Beverage, has default access and is in a different package. You can do one of two things to make this work. You could put both classes in the same package, or you could declare Beverage as `public`, as the next section describes.

When you see a question with complex logic, be sure to look at the access modifiers first. That way, if you spot an access violation (for example, a class in package A trying to access a default class in package B), you'll know the code won't compile so you don't have to bother working through the logic. It's not as if you don't have anything better to do with your time while taking the exam. Just choose the "Compilation fails" answer and zoom on to the next question.

Public Access A class declaration with the `public` keyword gives all classes from all packages access to the `public` class. In other words, *all* classes in the Java Universe (JU) have access to a `public` class. Don't forget, though, that if a `public` class you're trying to use is in a different package from the class you're writing, you'll still need to import the `public` class.

In the example from the preceding section, we may not want to place the subclass in the same package as the superclass. To make the code work, we need to add the keyword `public` in front of the superclass (Beverage) declaration, as follows:

```
package cert;  
public class Beverage { }
```

This changes the `Beverage` class so it will be visible to all classes in all packages. The class can now be instantiated from all other classes, and any class is now free to subclass (extend from) it—unless, that is, the class is also marked with the nonaccess modifier `final`. Read on.

Other (Nonaccess) Class Modifiers

You can modify a class declaration using the keyword `final`, `abstract`, or `strictfp`. These modifiers are in addition to whatever access control is on the class, so you could, for example, declare a class as both `public` and `final`. But you can't always mix nonaccess modifiers. You're free to use `strictfp` in combination with `final`, for example, but you must never, ever, ever mark a class as both `final` and `abstract`. You'll see why in the next two sections.

You won't need to know how `strictfp` works, so we're focusing only on modifying a class as `final` or `abstract`. For the exam, you need to know only that `strictfp` is a keyword and can be used to modify a class or a method, but never a variable. Marking a class as `strictfp` means that any method code in the class will conform to the IEEE 754 standard rules for floating points. Without that modifier, floating points used in the methods might behave in a platform-dependent way. If you don't declare a class as `strictfp`, you can still get `strictfp` behavior on a method-by-method basis, by declaring a method as `strictfp`. If you don't know the IEEE 754 standard, now's not the time to learn it. You have, as we say, bigger fish to fry.

Final Classes When used in a class declaration, the `final` keyword means the class can't be subclassed. In other words, no other class can ever extend (inherit from) a `final` class, and any attempts to do so will give you a compiler error.

So why would you ever mark a class `final`? After all, doesn't that violate the whole object-oriented (OO) notion of inheritance? You should make a `final` class only if you need an absolute guarantee that none of the methods in that class will ever be overridden. If you're deeply dependent on the implementations of certain methods, then using `final` gives you the security that nobody can change the implementation out from under you.

You'll notice many classes in the Java core libraries are `final`. For example, the `String` class cannot be subclassed. Imagine the havoc if you couldn't guarantee how a `String` object would work on any given system your application is running on! If

programmers were free to extend the `String` class (and thus substitute their new `String` subclass instances where `java.lang.String` instances are expected), civilization—as we know it—could collapse. So use `final` for safety, but only when you're certain that your `final` class has indeed said all that ever needs to be said in its methods. Marking a class `final` means, in essence, your class can't ever be improved upon, or even specialized, by another programmer.

A benefit of having nonfinal classes is this scenario: Imagine you find a problem with a method in a class you're using, but you don't have the source code. So you can't modify the source to improve the method, but you can extend the class and override the method in your new subclass, and substitute the subclass everywhere the original superclass is expected. If the class is `final`, though, then you're stuck.

Let's modify our `Beverage` example by placing the keyword `final` in the declaration:

```
package cert;
public final class Beverage {
    public void importantMethod() { }
}
```

Now, if we try to compile the `Tea` subclass:

```
package exam.stuff;
import cert.Beverage;
class Tea extends Beverage { }
```

We get an error something like

```
Can't subclass final classes: class
cert.Beverage class Tea extends Beverage{
1 error
```

In practice, you'll almost never make a `final` class. A `final` class obliterates a key benefit of OO—extensibility. So unless you have a serious safety or security issue, assume that some day another programmer will need to extend your class. If you don't, the next programmer forced to maintain your code will hunt you down and <insert really scary thing>.

Abstract Classes An abstract class can never be instantiated. Its sole purpose, mission in life, *raison d'être*, is to be extended (subclassed). (Note, however, that you can compile and execute an abstract class, as long as you don't try

to make an instance of it.) Why make a class if you can't make objects out of it? Because the class might be just too, well, *abstract*. For example, imagine you have a class `Car` that has generic methods common to all vehicles. But you don't want anyone actually creating a generic, abstract `Car` object. How would they initialize its state? What color would it be? How many seats? Horsepower? All-wheel drive? Or more importantly, how would it behave? In other words, how would the methods be implemented?

No, you need programmers to instantiate actual car types such as `BMWBoxster` and `SubaruOutback`. We'll bet the Boxster owner will tell you his car does things the Subaru can do "only in its dreams." Take a look at the following abstract class:

```
abstract class Car {
    private double price;
    private String model;
    private String year;
    public abstract void goFast();
    public abstract void goUpHill();
    public abstract void impressNeighbors();
    // Additional, important, and serious code goes here
}
```

The preceding code will compile fine. However, if you try to instantiate a `Car` in another body of code, you'll get a compiler error something like this:

```
AnotherClass.java:7: class Car is an abstract
class. It can't be instantiated.
    Car x = new Car();
1 error
```

Notice that the methods marked `abstract` end in a semicolon rather than curly braces.

Look for questions with a method declaration that ends with a semicolon, rather than curly braces. If the method is in a class—as opposed to an interface—then both the method and the class must be marked `abstract`. You might get a question that asks how you could fix a code sample that includes a method ending in a semicolon, but without an `abstract` modifier on the class or method. In that case, you could either mark the method and class `abstract`, or change the semicolon to code (like a curly brace pair). Remember, if you change a method from `abstract` to `nonabstract`, don't forget to change the semicolon at the end of the method declaration into a curly brace pair!

We'll look at abstract methods in more detail later in this objective, but always remember that if even a single method is `abstract`, the whole class must be declared `abstract`. One abstract method spoils the whole bunch. You can, however, put nonabstract methods in an `abstract` class. For example, you might have methods with implementations that shouldn't change from `Car` type to `Car` type, such as `getColor()` or `setPrice()`. By putting nonabstract methods in an `abstract` class, you give all concrete subclasses (concrete just means not `abstract`) inherited method implementations. The good news there is that concrete subclasses get to inherit functionality, and need to implement only the methods that define subclass-specific behavior.

(By the way, if you think we misused *raison d'être* earlier, don't send an e-mail. We'd like to see *you* work it into a programmer certification book.)

Coding with abstract class types (including interfaces, discussed later in this chapter) lets you take advantage of polymorphism, and gives you the greatest degree of flexibility and extensibility. You'll learn more about polymorphism in Chapter 2.

You can't mark a class as both `abstract` and `final`. They have nearly opposite meanings. An `abstract` class must be subclassed, whereas a `final` class must not be subclassed. If you see this combination of `abstract` and `final` modifiers, used for a class or method declaration, the code will not compile.

EXERCISE 1-1

Creating an Abstract Superclass and Concrete Subclass

The following exercise will test your knowledge of `public`, `default`, `final`, and `abstract` classes. Create an `abstract` superclass named `Fruit` and a concrete subclass named `Apple`. The superclass should belong to a package called `food` and the subclass can belong to the default package (meaning it isn't put into a package explicitly). Make the superclass `public` and give the subclass default access.

1. Create the superclass as follows:

```
package food;
public abstract class Fruit{ /* any code you want */}
```

2. Create the subclass in a separate file as follows:

```
import food.Fruit;
class Apple extends Fruit{ /* any code you want */}
```

3. Create a directory called `food` off the directory in your class path setting.
 4. Attempt to compile the two files. If you want to use the `Apple` class, make sure you place the `Fruit.class` file in the `food` subdirectory.
-

CERTIFICATION OBJECTIVE

Declare Interfaces (Exam Objectives I.1 and I.2)

1.1 Develop code that declares classes (including abstract and all forms of nested classes), interfaces, and enums, and includes the appropriate use of package and import statements (including static imports).

1.2 Develop code that declares an interface. Develop code that implements or extends one or more interfaces. Develop code that declares an abstract class. Develop code that extends an abstract class.

Declaring an Interface

When you create an interface, you're defining a contract for *what* a class can do, without saying anything about *how* the class will do it. An interface is a contract. You could write an interface `Bounceable`, for example, that says in effect, "This is the `Bounceable` interface. Any class type that implements this interface must agree to write the code for the `bounce()` and `setBounceFactor()` methods."

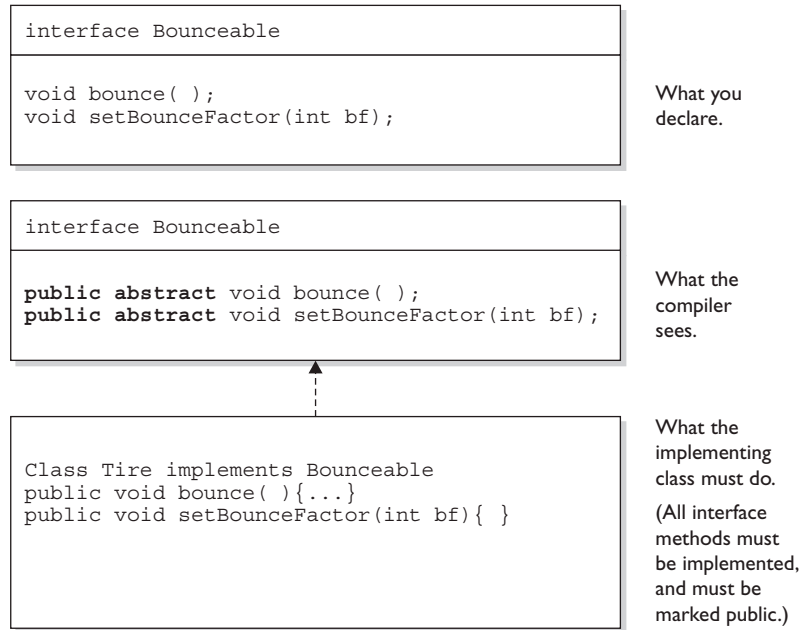
By defining an interface for `Bounceable`, any class that wants to be treated as a `Bounceable` thing can simply implement the `Bounceable` interface and provide code for the interface's two methods.

Interfaces can be implemented by any class, from any inheritance tree. This lets you take radically different classes and give them a common characteristic. For example, you might want both a `Ball` and a `Tire` to have bounce behavior, but `Ball` and `Tire` don't share any inheritance relationship; `Ball` extends `Toy` while `Tire` extends only `java.lang.Object`. But by making both `Ball` and `Tire` implement `Bounceable`, you're saying that `Ball` and `Tire` can be treated as, "Things that can bounce," which in Java translates to "Things on which you can invoke the

bounce() and setBounceFactor() methods." Figure 1-1 illustrates the relationship between interfaces and classes.

FIGURE 1-1

The Relationship between interfaces and classes



Think of an interface as a 100-percent abstract class. Like an abstract class, an interface defines abstract methods that take the following form:

```
abstract void bounce(); // Ends with a semicolon rather than
                        // curly braces
```

But while an abstract class can define both abstract and non-abstract methods, an interface can have only abstract methods. Another way interfaces differ from abstract classes is that interfaces have very little flexibility in how the methods and variables defined in the interface are declared. These rules are strict:

- All interface methods are implicitly `public` and `abstract`. In other words, you do not need to actually type the `public` or `abstract` modifiers in the method declaration, but the method is still always `public` and `abstract`.
- All variables defined in an interface must be `public`, `static`, and `final`—in other words, interfaces can declare only constants, not instance variables.

- Interface methods must not be `static`.
- Because interface methods are abstract, they cannot be marked `final`, `strictfp`, or `native`. (More on these modifiers later.)
- An interface can *extend* one or more other interfaces.
- An interface cannot extend anything but another interface.
- An interface cannot implement another interface or class.
- An interface must be declared with the keyword `interface`.
- Interface types can be used polymorphically (see Chapter 2 for more details).

The following is a legal interface declaration:

```
public abstract interface Rollable { }
```

Typing in the `abstract` modifier is considered redundant; interfaces are implicitly abstract whether you type `abstract` or not. You just need to know that both of these declarations are legal, and functionally identical:

```
public abstract interface Rollable { }
public interface Rollable { }
```

The `public` modifier is required if you want the interface to have public rather than default access.

We've looked at the interface declaration but now we'll look closely at the methods within an interface:

```
public interface Bounceable {
    public abstract void bounce();
    public abstract void setBounceFactor(int bf);
}
```

Typing in the `public` and `abstract` modifiers on the methods is redundant, though, since all interface methods are implicitly `public` and `abstract`. Given that rule, you can see that the following code is exactly equivalent to the preceding interface:

```
public interface Bounceable {
    void bounce();                // No modifiers
    void setBounceFactor(int bf); // No modifiers
}
```

You must remember that all interface methods are public and abstract regardless of what you see in the interface definition.

Look for interface methods declared with any combination of `public`, `abstract`, or no modifiers. For example, the following five method declarations, if declared within their own interfaces, are legal and identical!

```
void bounce();  
public void bounce();  
abstract void bounce();  
public abstract void bounce();  
abstract public void bounce();
```

The following interface method declarations won't compile:

```
final void bounce();    // final and abstract can never be used  
                        // together, and abstract is implied  
static void bounce();   // interfaces define instance methods  
private void bounce();  // interface methods are always public  
protected void bounce(); // (same as above)
```

Declaring Interface Constants

You're allowed to put constants in an interface. By doing so, you guarantee that any class implementing the interface will have access to the same constant.

By placing the constants right in the interface, any class that implements the interface has direct access to the constants, just as if the class had inherited them.

You need to remember one key rule for interface constants. They must always be

```
public static final
```

So that sounds simple, right? After all, interface constants are no different from any other publicly accessible constants, so they obviously must be declared `public`, `static`, and `final`. But before you breeze past the rest of this discussion, think about the implications: **Because interface constants are defined in an interface, they don't have to be *declared* as `public`, `static`, or `final`. They must be `public`, `static`, and `final`, but you don't have to actually declare them that way.** Just as interface methods are always public and abstract whether you say so in the code or not, any variable defined in an interface must be—and implicitly is—a public

constant. See if you can spot the problem with the following code (assume two separate files):

```
interface Foo {
    int BAR = 42;
    void go();
}

class Zap implements Foo {
    public void go() {
        BAR = 27;
    }
}
```

You can't change the value of a constant! Once the value has been assigned, the value can never be modified. The assignment happens in the interface itself (where the constant is declared), so the implementing class can access it and use it, but as a read-only value. So the `BAR = 27` assignment will not compile.

exam

Watch

Look for interface definitions that define constants, but without explicitly using the required modifiers. For example, the following are all identical:

```
public int x = 1;           // Looks non-static and non-final,
                           // but isn't!
int x = 1;                 // Looks default, non-final,
                           // non-static, but isn't!
static int x = 1;          // Doesn't show final or public
final int x = 1;           // Doesn't show static or public
public static int x = 1;    // Doesn't show final
public final int x = 1;     // Doesn't show static
static final int x = 1      // Doesn't show public
public static final int x = 1; // what you get implicitly
```

Any combination of the required (but implicit) modifiers is legal, as is using no modifiers at all! On the exam, you can expect to see questions you won't be able to answer correctly unless you know, for example, that an interface variable is `final` and can never be given a value by the implementing (or any other) class.

CERTIFICATION OBJECTIVE**Declare Class Members (Objectives 1.3 and 1.4)**

1.3 Develop code that declares, initializes, and uses primitives, arrays, enums, and objects as static, instance, and local variables. Also, use legal identifiers for variable names.

1.4 Develop code that declares both static and non-static methods, and—if appropriate—use method names that adhere to the JavaBeans naming standards. Also develop code that declares and uses a variable-length argument list.

We've looked at what it means to use a modifier in a class declaration, and now we'll look at what it means to modify a method or variable declaration.

Methods and instance (nonlocal) variables are collectively known as members. You can modify a member with both access and nonaccess modifiers, and you have more modifiers to choose from (and combine) than when you're declaring a class.

Access Modifiers

Because method and variable members are usually given access control in exactly the same way, we'll cover both in this section.

Whereas a *class* can use just two of the four access control levels (default or `public`), *members* can use all four:

- `public`
- `protected`
- `default`
- `private`

Default protection is what you get when you don't type an access modifier in the member declaration. The default and `protected` access control types have almost identical behavior, except for one difference that will be mentioned later.

It's crucial that you know access control inside and out for the exam. There will be quite a few questions with access control playing a role. Some questions test

several concepts of access control at the same time, so not knowing one small part of access control could blow an entire question.

What does it mean for code in one class to have access to a member of another class? For now, ignore any differences between methods and variables. If class A has access to a member of class B, it means that class B's member is visible to class A. When a class does not have access to another member, the compiler will slap you for trying to access something that you're not even supposed to know exists!

You need to understand two different access issues:

- Whether method code in one class can *access* a member of another class
- Whether a subclass can *inherit* a member of its superclass

The first type of access is when a method in one class tries to access a method or a variable of another class, using the dot operator (.) to invoke a method or retrieve a variable. For example:

```
class Zoo {
    public String coolMethod() {
        return "Wow  baby";
    }
}
class Moo {
    public void useAZoo() {
        Zoo z = new Zoo();
        // If the preceding line compiles Moo has access
        // to the Zoo class
        // But... does it have access to the coolMethod()?
        System.out.println("A Zoo says, " + z.coolMethod());
        // The preceding line works because Moo can access the
        // public method
    }
}
```

The second type of access revolves around which, if any, members of a superclass a subclass can access through inheritance. We're not looking at whether the subclass can, say, invoke a method on an instance of the superclass (which would just be an example of the first type of access). Instead, we're looking at whether the subclass *inherits* a member of its superclass. Remember, if a subclass *inherits* a member, it's exactly as if the subclass actually declared the member itself. In other words, if a subclass *inherits* a member, the subclass *has* the member.

```
class Zoo {
    public String coolMethod() {
        return "Wow  baby";
    }
}
class Moo extends Zoo {
    public void useMyCoolMethod() {
        // Does an instance of Moo inherit the coolMethod()?
        System.out.println("Moo says, " + this.coolMethod());
        // The preceding line works because Moo can inherit the
        // public method
        // Can an instance of Moo invoke coolMethod() on an
        // instance of Zoo?
        Zoo z = new Zoo();
        System.out.println("Zoo says, " + z.coolMethod());
        // coolMethod() is public, so Moo can invoke it on a Zoo
        //reference
    }
}
```

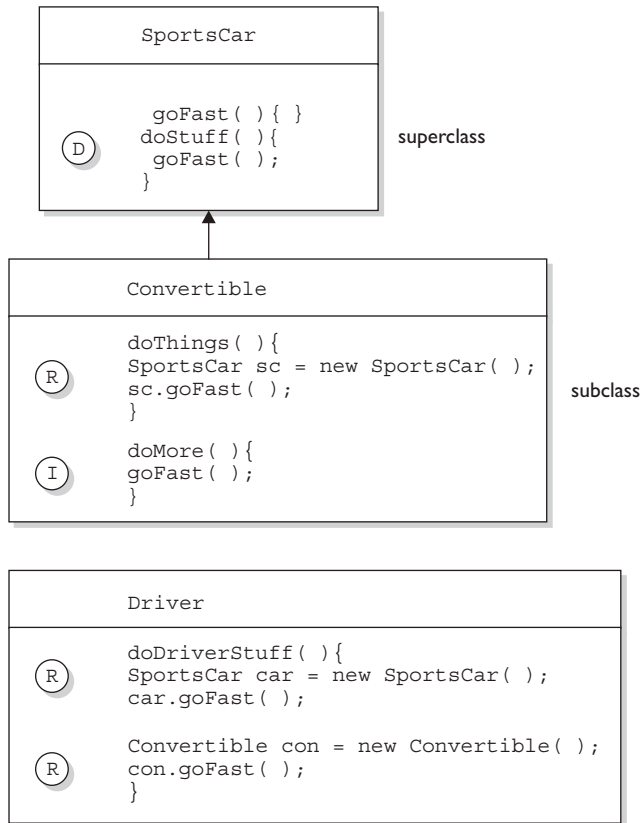
Figure 1-2 compares a class inheriting a member of another class, and accessing a member of another class using a reference of an instance of that class.

Much of access control (both types) centers on whether the two classes involved are in the same or different packages. Don't forget, though, if class A *itself* can't be accessed by class B, then no members within class A can be accessed by class B.

You need to know the effect of different combinations of class and member access (such as a default class with a `public` variable). To figure this out, first look at the access level of the class. If the class itself will not be visible to another class, then none of the members will be either, even if the member is declared `public`. Once you've confirmed that the class is visible, then it makes sense to look at access levels on individual members.

Public Members

When a method or variable member is declared `public`, it means all other classes, regardless of the package they belong to, can access the member (assuming the class itself is visible).

FIGURE I-2 Comparison of inheritance vs. dot operator for member access.

Three ways to access a method:

- (D) Invoking a method declared in the same class
- (R) Invoking a method using a reference of the class
- (I) Invoking an inherited method

Look at the following source file:

```
package book;
import cert.*; // Import all classes in the cert package
class Goo {
    public static void main(String[] args) {
        Sludge o = new Sludge();
        o.testIt();
    }
}
```

Now look at the second file:

```
package cert;
public class Sludge {
    public void testIt() { System.out.println("sludge"); }
}
```

As you can see, Goo and Sludge are in different packages. However, Goo can invoke the method in Sludge without problems because both the Sludge class and its testIt() method are marked public.

For a subclass, if a member of its superclass is declared public, the subclass inherits that member regardless of whether both classes are in the same package:

```
package cert;
public class Roo {
    public String doRooThings() {
        // imagine the fun code that goes here
        return "fun";
    }
}
```

The Roo class declares the doRooThings() member as public. So if we make a subclass of Roo, any code in that Roo subclass can call its own inherited doRooThings() method.

```
package notcert; //Not the package Roo is in
import cert.Roo;
class Cloo extends Roo {
    public void testCloo() {
        System.out.println(doRooThings());
    }
}
```

Notice in the preceding code that the `doRooThings()` method is invoked without having to preface it with a reference. Remember, if you see a method invoked (or a variable accessed) without the dot operator (`.`), it means the method or variable belongs to the class where you see that code. It also means that the method or variable is implicitly being accessed using the `this` reference. So in the preceding code, the call to `doRooThings()` in the `Cloo` class could also have been written as `this.doRooThings()`. The reference `this` always refers to the currently executing object—in other words, the object running the code where you see the `this` reference. Because the `this` reference is implicit, you don't need to preface your member access code with it, but it won't hurt. Some programmers include it to make the code easier to read for new (or non) Java programmers.

Besides being able to invoke the `doRooThings()` method on itself, code from some other class can call `doRooThings()` on a `Cloo` instance, as in the following:

```
class Toon {
    public static void main(String[] args) {
        Cloo c = new Cloo();
        System.out.println(c.doRooThings()); //No problem; method
                                              // is public
    }
}
```

Private Members

Members marked `private` can't be accessed by code in any class other than the class in which the `private` member was declared. Let's make a small change to the `Roo` class from an earlier example.

```
package cert;
public class Roo {
    private String doRooThings() {
        // imagine the fun code that goes here, but only the Roo
        // class knows
        return "fun";
    }
}
```

The `doRooThings()` method is now `private`, so no other class can use it. If we try to invoke the method from any other class, we'll run into trouble:

```

package notcert;
import cert.Roo;
class UseARoo {
    public void testIt() {
        Roo r = new Roo(); //So far so good; class Roo is public
        System.out.println(r.doRooThings()); //Compiler error!
    }
}

```

If we try to compile UseARoo, we get a compiler error something like this:

```

cannot find symbol
symbol   : method doRooThings()

```

It's as if the method `doRooThings()` doesn't exist, and as far as any code outside of the Roo class is concerned, it's true. A private member is invisible to any code outside the member's own class.

What about a subclass that tries to inherit a private member of its superclass? When a member is declared private, a subclass can't inherit it. For the exam, you need to recognize that a subclass can't see, use, or even think about the private members of its superclass. You can, however, declare a matching method in the subclass. But regardless of how it looks, **it is not an overriding method!** It is simply a method that happens to have the same name as a private method (which you're not supposed to know about) in the superclass. The rules of overriding do not apply, so you can make this newly-declared-but-just-happens-to-match method declare new exceptions, or change the return type, or anything else you want to do with it.

```

package cert;
public class Roo {
    private String doRooThings() {
        // imagine the fun code that goes here, but no other class
        // will know
        return "fun";
    }
}

```

The `doRooThings()` method is now off limits to all subclasses, even those in the same package as the superclass:

```
package cert;                //Cloo and Roo are in the same package
class Cloo extends Roo {    //Still OK, superclass Roo is public
    public void testCloo() {
        System.out.println(doRooThings()); //Compiler error!
    }
}
```

If we try to compile the subclass Cloo, the compiler is delighted to spit out an error something like this:

```
%javac Cloo.java
Cloo.java:4: Undefined method: doRooThings()
    System.out.println(doRooThings());
1 error
```



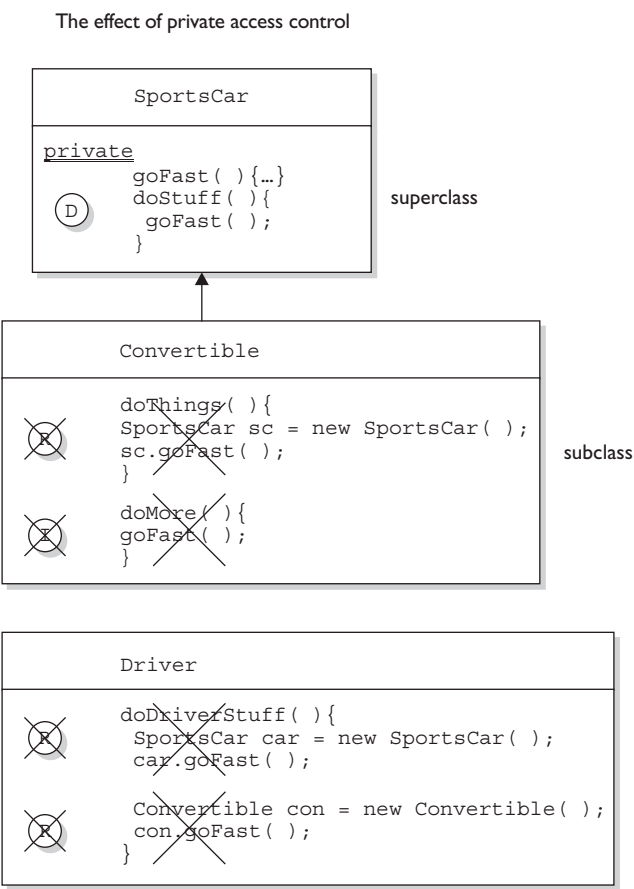
Although you're allowed to mark instance variables as `public`, in practice it's nearly always best to keep all variables `private` or `protected`. If variables need to be changed, set, or read, programmers should use `public` accessor methods, so that code in any other class has to ask to get or set a variable (by going through a method), rather than access it directly. `JavaBean`-compliant accessor methods take the form `get<propertyName>` or, for booleans, `is<propertyName>` and `set<propertyName>`, and provide a place to check and/or validate before returning or modifying a value.

Without this protection, the weight variable of a `Cat` object, for example, could be set to a negative number if the offending code goes straight to the public variable as in `someCat.weight = -20`. But an accessor method, `setWeight(int wt)`, could check for an inappropriate number. (OK, wild speculation, but we're guessing a negative weight might be inappropriate for a cat. Or not.) Chapter 2 will discuss this data protection (encapsulation) in more detail.

Can a `private` method be overridden by a subclass? That's an interesting question, but the answer is technically no. Since the subclass, as we've seen, cannot inherit a `private` method, it therefore cannot override the method—overriding depends on inheritance. We'll cover the implications of this in more detail a little later in this section as well as in Chapter 2, but for now just remember that a method marked `private` cannot be overridden. Figure 1-3 illustrates the effects of the `public` and `private` modifiers on classes from the same or different packages.

FIGURE I-3

Effects of public and private access



Three ways to access a method:

- (D) Invoking a method declared in the same class
- (R) Invoking a method using a reference of the class
- (I) Invoking an inherited method

Protected and Default Members

The protected and default access control levels are almost identical, but with one critical difference. A *default* member may be accessed only if the class accessing the member belongs to the same package, whereas a *protected* member can be accessed (through inheritance) by a subclass **even if the subclass is in a different package**.

Take a look at the following two classes:

```
package certification;
public class OtherClass {
    void testIt() {    // No modifier means method has default
                      // access
        System.out.println("OtherClass");
    }
}
```

In another source code file you have the following:

```
package somethingElse;
import certification.OtherClass;
class AccessClass {
    static public void main(String[] args) {
        OtherClass o = new OtherClass();
        o.testIt();
    }
}
```

As you can see, the `testIt()` method in the first file has *default* (think: *package-level*) access. Notice also that class `OtherClass` is in a different package from the `AccessClass`. Will `AccessClass` be able to use the method `testIt()`? Will it cause a compiler error? Will Daniel ever marry Francesca? Stay tuned.

```
No method matching testIt() found in class
certification.OtherClass.    o.testIt();
```

From the preceding results, you can see that `AccessClass` can't use the `OtherClass` method `testIt()` because `testIt()` has default access, and `AccessClass` is not in the same package as `OtherClass`. So `AccessClass` can't see it, the compiler complains, and we have no idea who Daniel and Francesca are.

Default and *protected* behavior differ only when we talk about subclasses. If the `protected` keyword is used to define a member, any subclass of the class declaring the member can access it *through inheritance*. It doesn't matter if the superclass and subclass are in different packages, the `protected` superclass member is still visible to the subclass (although visible only in a very specific way as we'll see a little later). This is in contrast to the default behavior, which doesn't allow a subclass to access a superclass member unless the subclass is in the same package as the superclass.

Whereas default access doesn't extend any special consideration to subclasses (you're either in the package or you're not), the protected modifier respects the parent-child relationship, even when the child class moves away (and joins a new package). So, when you think of *default* access, think *package* restriction. No exceptions. But when you think *protected*, think *package + kids*. A class with a protected member is marking that member as having package-level access for all classes, but with a special exception for subclasses outside the package.

But what does it mean for a subclass-outside-the-package to have access to a superclass (parent) member? It means the subclass inherits the member. It does not, however, mean the subclass-outside-the-package can access the member using a reference to an instance of the superclass. In other words, protected = inheritance. Protected does not mean that the subclass can treat the protected superclass member as though it were public. So if the subclass-outside-the-package gets a reference to the superclass (by, for example, creating an instance of the superclass somewhere in the subclass' code), the subclass cannot use the dot operator on the superclass reference to access the protected member. To a subclass-outside-the-package, a protected member might as well be default (or even private), when the subclass is using a reference to the superclass. **The subclass can see the protected member only through inheritance.**

Are you confused? So are we. Hang in there and it will all become clear with the next batch of code examples. (And don't worry; we're not actually confused. We're just trying to make you feel better if you are. You know, like it's OK for you to feel as though nothing makes sense, and that it isn't your fault. Or is it? <insert evil laugh>)

Protected Details

Let's take a look at a protected instance variable (remember, an instance variable is a member) of a superclass.

```
package certification;
public class Parent {
    protected int x = 9; // protected access
}
```

The preceding code declares the variable `x` as protected. This makes the variable *accessible* to all other classes *inside* the certification package, as well as *inheritable* by any subclasses *outside* the package. Now let's create a subclass in a different package, and attempt to use the variable `x` (that the subclass inherits):

```

package other; // Different package
import certification.Parent;
class Child extends Parent {
    public void testIt() {
        System.out.println("x is " + x); // No problem; Child
                                         // inherits x
    }
}

```

The preceding code compiles fine. Notice, though, that the Child class is accessing the protected variable through inheritance. Remember, any time we talk about a subclass having access to a superclass member, we could be talking about the subclass inheriting the member, not simply accessing the member through a reference to an instance of the superclass (the way any other nonsubclass would access it). Watch what happens if the subclass Child (outside the superclass' package) tries to access a protected variable using a Parent class reference.

```

package other;
import certification.Parent;
class Child extends Parent {
    public void testIt() {
        System.out.println("x is " + x); // No problem; Child
                                         // inherits x

        Parent p = new Parent(); // Can we access x using the
                                  // p reference?
        System.out.println("X in parent is " + p.x); // Compiler
                                                         // error!
    }
}

```

The compiler is more than happy to show us the problem:

```

%javac -d . other/Child.java
other/Child.java:9: x has protected access in certification.Par-
ent
System.out.println("X in parent is " + p.x);
                        ^
1 error

```

So far we've established that a protected member has essentially package-level or default access to all classes except for subclasses. We've seen that subclasses outside the package can inherit a protected member. Finally, we've seen that subclasses

outside the package can't use a superclass reference to access a protected member. ***For a subclass outside the package, the protected member can be accessed only through inheritance.***

But there's still one more issue we haven't looked at...what does a protected member look like to other classes trying to use the subclass-outside-the-package to get to the subclass' inherited protected superclass member? For example, using our previous Parent/Child classes, what happens if some other class—Neighbor, say—in the same package as the Child (subclass), has a reference to a Child instance and wants to access the member variable `x`? In other words, how does that protected member behave once the subclass has inherited it? Does it maintain its protected status, such that classes in the Child's package can see it?

No! Once the subclass-outside-the-package inherits the protected member, that member (as inherited by the subclass) becomes private to any code outside the subclass, with the exception of subclasses of the subclass. So if class Neighbor instantiates a Child object, then even if class Neighbor is in the same package as class Child, class Neighbor won't have access to the Child's inherited (but protected) variable `x`. Figure 1-4 illustrates the effect of protected access on classes and subclasses in the same or different packages.

Whew! That wraps up protected, the most misunderstood modifier in Java. Again, it's used only in very special cases, but you can count on it showing up on the exam. Now that we've covered the protected modifier, we'll switch to default member access, a piece of cake compared to protected.

Default Details

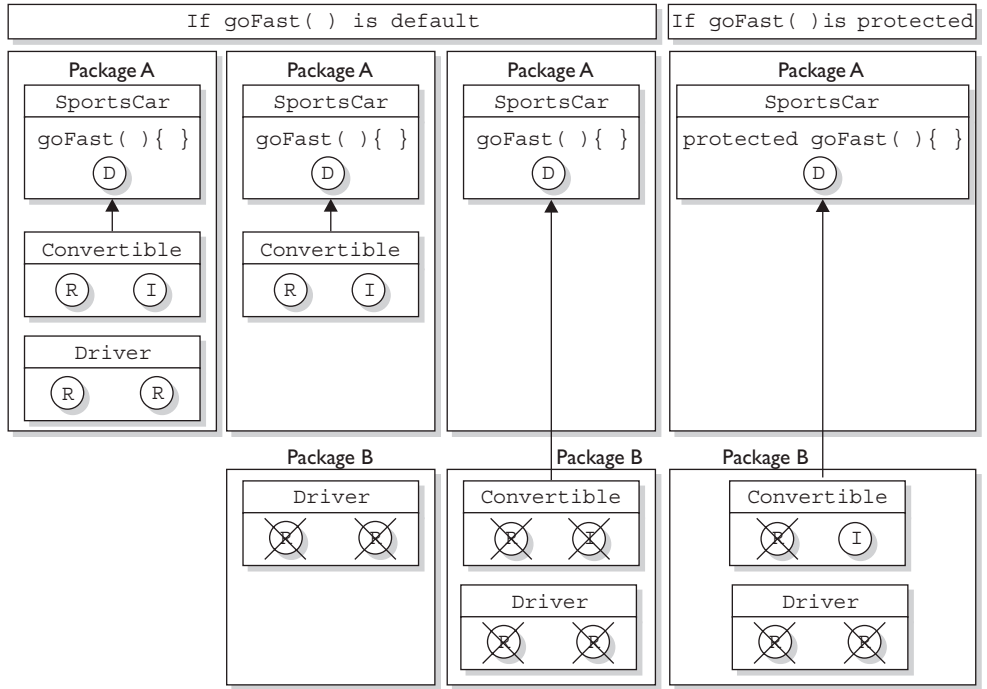
Let's start with the default behavior of a member in a superclass. We'll modify the Parent's member `x` to make it default.

```
package certification;
public class Parent {
    int x = 9; // No access modifier, means default
              // (package) access
}
```

Notice we didn't place an access modifier in front of the variable `x`. Remember that if you don't type an access modifier before a class or member declaration, the access control is default, which means package level. We'll now attempt to access the default member from the Child class that we saw earlier.

FIGURE I-4

Effects of
protected
access



When we compile the child file, we get an error something like this:

```
Child.java:4: Undefined variable: x
    System.out.println("Variable x is " + x);
    1 error
```

The compiler gives the same error as when a member is declared as `private`. The subclass `Child` (in a different package from the superclass `Parent`) can't see or use the default superclass member `x` ! Now, what about default access for two classes in the same package?

```
package certification;
public class Parent{
    int x = 9; // default access
}
```

And in the second class you have the following:

```
package certification;
class Child extends Parent{
    static public void main(String[] args) {
        Child sc = new Child();
        sc.testIt();
    }
    public void testIt() {
        System.out.println("Variable x is " + x); // No problem;
    }
}
```

The preceding source file compiles fine, and the class `Child` runs and displays the value of `x`. Just remember that default members are visible to subclasses only if those subclasses are in the same package as the superclass.

Local Variables and Access Modifiers

Can access modifiers be applied to local variables? NO!

There is never a case where an access modifier can be applied to a local variable, so watch out for code like the following:

```
class Foo {
    void doStuff() {
        private int x = 7;
        this.doMore(x);
    }
}
```

You can be certain that any local variable declared with an access modifier will not compile. In fact, there is only one modifier that can ever be applied to local variables—`final`.

That about does it for our discussion on member access modifiers. Table 1-2 shows all the combinations of access and visibility; you really should spend some time with it. Next, we're going to dig into the other (nonaccess) modifiers that you can apply to member declarations.

TABLE 1-2 Determining Access to Class Members

Visibility	Public	Protected	Default	Private
From the same class	Yes	Yes	Yes	Yes
From any class in the same package	Yes	Yes	Yes	No
From a subclass in the same package	Yes	Yes	Yes	No
From a subclass outside the same package	Yes	Yes, <i>through inheritance</i>	No	No
From any non-subclass class outside the package	Yes	No	No	No

Nonaccess Member Modifiers

We've discussed member access, which refers to whether code from one class can invoke a method (or access an instance variable) from another class. That still leaves a boatload of other modifiers you can use on member declarations. Two you're already familiar with—`final` and `abstract`—because we applied them to class declarations earlier in this chapter. But we still have to take a quick look at `transient`, `synchronized`, `native`, `strictfp`, and then a long look at the Big One—`static`.

We'll look first at modifiers applied to methods, followed by a look at modifiers applied to instance variables. We'll wrap up this section with a look at how `static` works when applied to variables and methods.

Final Methods

The `final` keyword prevents a method from being overridden in a subclass, and is often used to enforce the API functionality of a method. For example, the `Thread` class has a method called `isAlive()` that checks whether a thread is still active. If you extend the `Thread` class, though, there is really no way that you can correctly implement this method yourself (it uses native code, for one thing), so the designers have made it `final`. Just as you can't subclass the `String` class (because we need to be able to trust in the behavior of a `String` object), you can't override many of the methods in the core class libraries. This can't-be-overridden restriction provides for safety and security, but you should use it with great caution. Preventing a subclass from overriding a method stifles many of the benefits of OO including extensibility through polymorphism. A typical `final` method declaration looks like this:

```
class SuperClass{
    public final void showSample() {
        System.out.println("One thing.");
    }
}
```

It's legal to extend `SuperClass`, since the *class* isn't marked `final`, but we can't override the *final method* `showSample()`, as the following code attempts to do:

```
class SubClass extends SuperClass{
    public void showSample() { // Try to override the final
                               // superclass method
        System.out.println("Another thing.");
    }
}
```

Attempting to compile the preceding code gives us something like this:

```
%javac FinalTest.java
FinalTest.java:5: The method void showSample() declared in class
SubClass cannot override the final method of the same signature
declared in class SuperClass.
Final methods cannot be overridden.
    public void showSample() { }
1 error
```


Final Arguments

Method arguments are the variable declarations that appear in between the parentheses in a method declaration. A typical method declaration with multiple arguments looks like this:

```
public Record getRecord(int fileNumber, int recNumber) {}
```

Method arguments are essentially the same as local variables. In the preceding example, the variables `fileNumber` and `recNumber` will both follow all the rules applied to local variables. This means they can also have the modifier `final`:

```
public Record getRecord(int fileNumber, final int recordNumber) {}
```

In this example, the variable `recNumber` is declared as `final`, which of course means it can't be modified within the method. In this case, "modified" means reassigning a new value to the variable. In other words, a `final` argument must keep the same value that the parameter had when it was passed into the method.

Abstract Methods

An abstract method is a method that's been *declared* (as `abstract`) but not *implemented*. In other words, the method contains no functional code. And if you recall from the earlier section "Abstract Classes," an abstract method declaration doesn't even have curly braces for where the implementation code goes, but instead closes with a semicolon. In other words, *it has no method body*. You mark a method `abstract` when you want to force subclasses to provide the implementation. For example, if you write an abstract class `Car` with a method `goUpHill()`, you might want to force each subtype of `Car` to define its own `goUpHill()` behavior, specific to that particular type of car.

```
public abstract void showSample();
```

Notice that the abstract method ends with a semicolon instead of curly braces. **It is illegal to have even a single abstract method in a class that is not explicitly declared `abstract`!** Look at the following illegal class:

```
public class IllegalClass{
    public abstract void doIt();
}
```

The preceding class will produce the following error if you try to compile it:

```
IllegalClass.java:1: class IllegalClass must be declared
abstract.
It does not define void doIt() from class IllegalClass.
public class IllegalClass{
1 error
```

You can, however, have an abstract class with no abstract methods. The following example will compile fine:

```
public abstract class LegalClass{
    void goodMethod() {
        // lots of real implementation code here
    }
}
```

In the preceding example, `goodMethod()` is not abstract. Three different clues tell you it's not an abstract method:

- The method is not marked `abstract`.
- The method declaration includes curly braces, as opposed to ending in a semicolon. In other words, the method has a method body.
- The method provides actual implementation code.

Any class that extends an abstract class must implement all abstract methods of the superclass, unless the subclass is *also* abstract. The rule is this:

The first concrete subclass of an abstract class must implement *all* abstract methods of the superclass.

Concrete just means nonabstract, so if you have an abstract class extending another abstract class, the abstract subclass doesn't need to provide implementations for the inherited abstract methods. Sooner or later, though, somebody's going to make a nonabstract subclass (in other words, a class that can be instantiated), and that subclass will have to implement all the abstract methods from up the inheritance tree. The following example demonstrates an inheritance tree with two abstract classes and one concrete class:

```

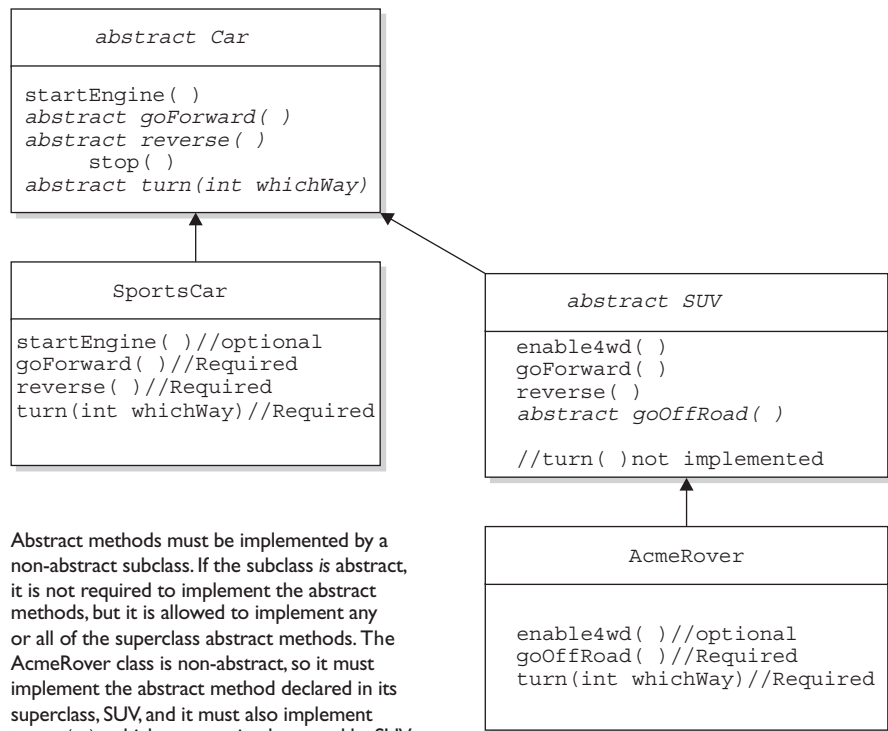
public abstract class Vehicle {
    private String type;
    public abstract void goUpHill(); // Abstract method
    public String getType() {        // Non-abstract method
        return type;
    }
}

public abstract class Car extends Vehicle {
    public abstract void goUpHill(); // Still abstract
    public void doCarThings() {
        // special car code goes here
    }
}

public class Mini extends Car {
    public void goUpHill() {
        // Mini-specific going uphill code
    }
}

```

So how many methods does class `Mini` have? Three. It inherits both the `getType()` and `doCarThings()` methods, because they're public and concrete (nonabstract). But because `goUpHill()` is abstract in the superclass `Vehicle`, and is never implemented in the `Car` class (so it remains abstract), it means class `Mini`—as the first concrete class below `Vehicle`—must implement the `goUpHill()` method. In other words, class `Mini` can't pass the buck (of abstract method implementation) to the next class down the inheritance tree, but class `Car` can, since `Car`, like `Vehicle`, is abstract. Figure 1-5 illustrates the effects of the abstract modifier on concrete and abstract subclasses.

FIGURE 1-5 The effects of the `abstract` modifier on concrete and abstract subclasses

Look for concrete classes that don't provide method implementations for abstract methods of the superclass. The following code won't compile:

```
public abstract class A {
    abstract void foo();
}
class B extends A {
    void foo(int I) { }
}
```

Class `B` won't compile because it doesn't implement the inherited abstract method `foo( )`. Although the `foo(int I)` method in class `B` might appear to be

an implementation of the superclass' abstract method, it is simply an overloaded method (a method using the same identifier, but different arguments), so it doesn't fulfill the requirements for implementing the superclass' abstract method. We'll look at the differences between overloading and overriding in detail in Chapter 2.

A method can never, ever, ever be marked as both `abstract` and `final`, or both `abstract` and `private`. Think about it—abstract methods must be implemented (which essentially means overridden by a subclass) whereas `final` and `private` methods cannot ever be overridden by a subclass. Or to phrase it another way, an `abstract` designation means the superclass doesn't know anything about how the subclasses should behave in that method, whereas a `final` designation means the superclass knows everything about how all subclasses (however far down the inheritance tree they may be) should behave in that method. The `abstract` and `final` modifiers are virtually opposites. Because `private` methods cannot even be seen by a subclass (let alone inherited), they too cannot be overridden, so they too cannot be marked `abstract`.

Finally, you need to know that the `abstract` modifier can never be combined with the `static` modifier. We'll cover `static` methods later in this objective, but for now just remember that the following would be illegal:

```
abstract static void doStuff();
```

And it would give you an error that should be familiar by now:

```
MyClass.java:2: illegal combination of modifiers: abstract and
static
    abstract static void doStuff();
```

Synchronized Methods

The `synchronized` keyword indicates that a method can be accessed by only one thread at a time. We'll discuss this nearly to death in Chapter 9, but for now all we're concerned with is knowing that the `synchronized` modifier can be applied only to methods—not variables, not classes, just methods. A typical `synchronized` declaration looks like this:

```
public synchronized Record retrieveUserInfo(int id) { }
```

You should also know that the `synchronized` modifier can be matched with any of the four access control levels (which means it can be paired with any of the three access modifier keywords).

Native Methods

The `native` modifier indicates that a method is implemented in platform-dependent code, often in C. You don't need to know how to use native methods for the exam, other than knowing that `native` is a modifier (thus a reserved keyword) and that `native` can be applied only to *methods*—not classes, not variables, just methods. Note that a native method's body must be a semicolon (;) (like abstract methods), indicating that the implementation is omitted.

Strictfp Methods

We looked earlier at using `strictfp` as a class modifier, but even if you don't declare a class as `strictfp`, you can still declare an individual method as `strictfp`. Remember, `strictfp` forces floating points (and any floating-point operations) to adhere to the IEEE 754 standard. With `strictfp`, you can predict how your floating points will behave regardless of the underlying platform the JVM is running on. The downside is that if the underlying platform is capable of supporting greater precision, a `strictfp` method won't be able to take advantage of it.

You'll want to study the IEEE 754 if you need something to help you fall asleep. For the exam, however, you don't need to know anything about `strictfp` other than what it's used for, that it can modify a class or method declaration, and that a variable can never be declared `strictfp`.

Methods with Variable Argument Lists (var-args)

As of 5.0, Java allows you to create methods that can take a variable number of arguments. Depending on where you look, you might hear this capability referred to as "variable-length argument lists," "variable arguments," "var-args," "varargs," or our personal favorite (from the department of obfuscation), "variable arity parameter." They're all the same thing, and we'll use the term "var-args" from here on out.

As a bit of background, we'd like to clarify how we're going to use the terms "argument" and "parameter" throughout this book.

- **arguments** The things you specify between the parentheses when you're *invoking* a method:

```
doStuff("a", 2); // invoking doStuff, so a & 2 are arguments
```

- **parameters** The things in the *method's signature* that indicate what the method must receive when it's invoked:

```
void doStuff(String s, int a) { } // we're expecting two
                                // parameters: String and int
```

We'll cover using var-arg methods more in the next few chapters, for now let's review the declaration rules for var-args:

- **Var-arg type** When you declare a var-arg parameter, you must specify the type of the argument(s) this parameter of your method can receive. (This can be a primitive type or an object type.)
- **Basic syntax** To declare a method using a var-arg parameter, you follow the type with an ellipsis (...), a space, and then the name of the array that will hold the parameters received.
- **Other parameters** It's legal to have other parameters in a method that uses a var-arg.
- **Var-args limits** The var-arg must be the last parameter in the method's signature, and you can have only one var-arg in a method.

Let's look at some legal and illegal var-arg declarations:

Legal:

```
void doStuff(int... x) { } // expects from 0 to many ints
                        // as parameters
void doStuff2(char c, int... x) { } // expects first a char,
                                    // then 0 to many ints
void doStuff3(Animal... animal) { } // 0 to many Animals
```

Illegal:

```
void doStuff4(int x...) { } // bad syntax
void doStuff5(int... x, char... y) { } // too many var-args
void doStuff6(String... s, byte b) { } // var-arg must be last
```

Constructor Declarations

In Java, objects are constructed. Every time you make a new object, at least one constructor is invoked. Every class has a constructor, although if you don't create one explicitly, the compiler will build one for you. There are tons of rules concerning

constructors, and we're saving our detailed discussion for Chapter 2. For now, let's focus on the basic declaration rules. Here's a simple example:

```
class Foo {
    protected Foo() { }           // this is Foo's constructor

    protected void Foo() { }      // this is a badly named,
                                   // but legal, method
}
```

The first thing to notice is that constructors look an awful lot like methods. A key difference is that a constructor can't ever, ever, ever, have a return type...ever! Constructor declarations can however have all of the normal access modifiers, and they can take arguments (including var-args), just like methods. The other BIG RULE, to understand about constructors is that they must have the same name as the class in which they are declared. Constructors can't be marked `static` (they are after all associated with object instantiation), they can't be marked `final` or `abstract` (because they can't be overridden). Here are some legal and illegal constructor declarations:

```
class Foo2 {

    // legal constructors

    Foo2() { }
    private Foo2(byte b) { }
    Foo2(int x) { }
    Foo2(int x, int... y) { }

    // illegal constructors

    void Foo2() { }           // it's a method, not a constructor
    Foo() { }                 // not a method or a constructor
    Foo2(short s);           // looks like an abstract method
    static Foo2(float f) { } // can't be static
    final Foo2(long x) { }   // can't be final
    abstract Foo2(char c) { } // can't be abstract
    Foo2(int... x, int t) { } // bad var-arg syntax
}
```


Variable Declarations

There are two types of variables in Java:

- **Primitives** A primitive can be one of eight types: `char`, `boolean`, `byte`, `short`, `int`, `long`, `double`, or `float`. Once a primitive has been declared, its primitive type can never change, although in most cases its value can change.
- **Reference variables** A reference variable is used to refer to (or access) an object. A reference variable is declared to be of a specific type and that type can never be changed. A reference variable can be used to refer to any object of the declared type, or of a *subtype* of the declared type (a compatible type). We'll talk a lot more about using a reference variable to refer to a subtype in Chapter 2, when we discuss polymorphism.

Declaring Primitives and Primitive Ranges

Primitive variables can be declared as class variables (statics), instance variables, method parameters, or local variables. You can declare one or more primitives, of the same primitive type, in a single line. In Chapter 3 we will discuss the various ways in which they can be initialized, but for now we'll leave you with a few examples of primitive variable declarations:

```
byte b;  
boolean myBooleanPrimitive;  
int x, y, z;                // declare three int primitives
```

On previous versions of the exam you needed to know how to calculate ranges for all the Java primitives. For the current exam, you can skip some of that detail, but it's still important to understand that for the integer types the sequence from small to big is `byte`, `short`, `int`, `long`, and that `doubles` are bigger than `floats`.

You will also need to know that the number types (both integer and floating-point types) are all signed, and how that affects their ranges. First, let's review the concepts.

All six number types in Java are made up of a certain number of 8-bit bytes, and are *signed*, meaning they can be negative or positive. The leftmost bit (the most significant digit) is used to represent the sign, where a 1 means negative and 0 means positive, as shown in Figure 1-6. The rest of the bits represent the value, using two's complement notation.

FIGURE 1-6 The Sign bit for a byte

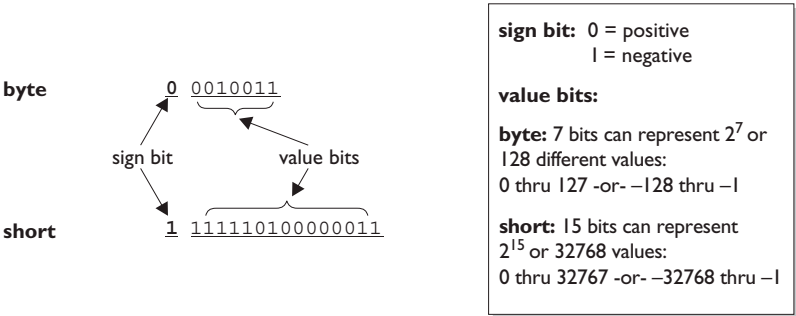


Table 1-3 shows the primitive types with their sizes and ranges. Figure 1-6 shows that with a byte, for example, there are 256 possible numbers (or 2^8). Half of these are negative, and half - 1 are positive. The positive range is one less than the negative range because the number zero is stored as a positive binary number. We use the formula $-2^{(\text{bits}-1)}$ to calculate the negative range, and we use $2^{(\text{bits}-1)}-1$ for the positive range. Again, if you know the first two columns of this table, you'll be in good shape for the exam.

TABLE 1-3 Ranges of Numeric Primitives

Type	Bits	Bytes	Minimum Range	Maximum Range
byte	8	1	-2^7	2^7-1
short	16	2	-2^{15}	$2^{15}-1$
int	32	4	-2^{31}	$2^{31}-1$
long	64	8	-2^{63}	$2^{63}-1$
float	32	4	n/a	n/a
double	64	8	n/a	n/a

The range for floating-point numbers is complicated to determine, but luckily you don't need to know these for the exam (although you are expected to know that a double holds 64 bits and a float 32).

For boolean types there is not a range; a boolean can be only `true` or `false`. If someone asks you for the bit depth of a boolean, look them straight in the eye and say, "That's virtual-machine dependent." They'll be impressed.

The `char` type (a character) contains a single, 16-bit Unicode character. Although the extended ASCII set known as ISO Latin-1 needs only 8 bits (256 different characters), a larger range is needed to represent characters found in languages other than English. Unicode characters are actually represented by unsigned 16-bit integers, which means 2^{16} possible values, ranging from 0 to 65535 (2^{16})-1. You'll learn in Chapter 3 that because a `char` is really an integer type, it can be assigned to any number type large enough to hold 65535 (which means anything larger than a `short`. Although both `chars` and `shorts` are 16-bit types, remember that a `short` uses 1 bit to represent the sign, so fewer positive numbers are acceptable in a `short`).

Declaring Reference Variables

Reference variables can be declared as static variables, instance variables, method parameters, or local variables. You can declare one or more reference variables, of the same type, in a single line. In Chapter 3 we will discuss the various ways in which they can be initialized, but for now we'll leave you with a few examples of reference variable declarations:

```
Object o;
Dog myNewDogReferenceVariable;
String s1, s2, s3;           // declare three String vars.
```

Instance Variables

Instance variables are defined inside the class, but outside of any method, and are only initialized when the class is instantiated. Instance variables are the fields that belong to each unique object. For example, the following code defines fields (instance variables) for the name, title, and manager for employee objects:

```
class Employee {
    // define fields (instance variables) for employee instances
    private String name;
    private String title,
```

```
private String manager;  
// other code goes here including access methods for private  
// fields  
}
```

The preceding `Employee` class says that each `employee` instance will know its own name, title, and manager. In other words, each instance can have its own unique values for those three fields. If you see the term "field," "instance variable," "property," or "attribute," they mean virtually the same thing. (There actually are subtle but occasionally important distinctions between the terms, but those distinctions aren't used on the exam.)

For the exam, you need to know that instance variables

- Can use any of the four access *levels* (which means they can be marked with any of the three access *modifiers*)
- Can be marked `final`
- Can be marked `transient`
- Cannot be marked `abstract`
- Cannot be marked `synchronized`
- Cannot be marked `strictfp`
- Cannot be marked `native`
- Cannot be marked `static`, because then they'd become class variables.

We've already covered the effects of applying access control to instance variables (it works the same way as it does for member methods). A little later in this chapter we'll look at what it means to apply the `final` or `transient` modifier to an instance variable. First, though, we'll take a quick look at the difference between instance and local variables. Figure 1-7 compares the way in which modifiers can be applied to methods vs. variables.

FIGURE I-7 Comparison of modifiers on variables vs. methods

Local Variables	Variables (non-local)	Methods
<code>final</code>	<code>final</code> <code>public</code> <code>protected</code> <code>private</code> <code>static</code> <code>transient</code> <code>volatile</code>	<code>final</code> <code>public</code> <code>protected</code> <code>private</code> <code>static</code> <code>abstract</code> <code>synchronized</code> <code>strictfp</code> <code>native</code>

Local (Automatic/Stack/Method) Variables

Local variables are variables declared within a method. That means the variable is not just initialized within the method, but also declared within the method. Just as the local variable starts its life inside the method, it's also destroyed when the method has completed. Local variables are always on the stack, not the heap. (We'll talk more about the stack and the heap in Chapter 3). Although the value of the variable might be passed into, say, another method that then stores the value in an instance variable, the variable itself lives only within the scope of the method.

Just don't forget that while the local variable is on the stack, if the variable is an object reference, the object itself will still be created on the heap. There is no such thing as a stack object, only a stack variable. You'll often hear programmers use the phrase, "local object," but what they really mean is, "locally declared reference variable." So if you hear a programmer use that expression, you'll know that he's just too lazy to phrase it in a technically precise way. You can tell him we said that—unless he knows where we live.

Local variable declarations can't use most of the modifiers that can be applied to instance variables, such as `public` (or the other access modifiers), `transient`, `volatile`, `abstract`, or `static`, but as we saw earlier, local variables can be marked `final`. And as you'll learn in Chapter 3 (but here's a preview), before a local variable can be *used*, it must be *initialized* with a value. For instance:

```

class TestServer {
    public void logIn() {
        int count = 10;
    }
}

```

Typically, you'll initialize a local variable in the same line in which you declare it, although you might still need to reinitialize it later in the method. The key is to remember that a local variable must be initialized before you try to use it. The compiler will reject any code that tries to use a local variable that hasn't been assigned a value, because—unlike instance variables—local variables don't get default values.

A local variable can't be referenced in any code outside the method in which it's declared. In the preceding code example, it would be impossible to refer to the variable `count` anywhere else in the class except within the scope of the method `logIn()`. Again, that's not to say that the value of `count` can't be passed out of the method to take on a new life. But the variable holding that value, `count`, can't be accessed once the method is complete, as the following illegal code demonstrates:

```

class TestServer {
    public void logIn() {
        int count = 10;
    }
    public void doSomething(int i) {
        count = i; // Won't compile! Can't access count outside
                  // method logIn()
    }
}

```

It is possible to declare a local variable with the same name as an instance variable. It's known as *shadowing*, as the following code demonstrates:

```

class TestServer {
    int count = 9; // Declare an instance variable named count
    public void logIn() {
        int count = 10; // Declare a local variable named count
        System.out.println("local variable count is " + count);
    }
    public void count() {
        System.out.println("instance variable count is " + count);
    }
    public static void main(String[] args) {

```

```

        new TestServer().login();
        new TestServer().count();
    }
}

```

The preceding code produces the following output:

```

local variable count is 10
instance variable count is 9

```

Why on earth (or the planet of your choice) would you want to do that? Normally, you won't. But one of the more common reasons is to name a parameter with the same name as the instance variable to which the parameter will be assigned.

The following (wrong) code is trying to set an instance variable's value using a parameter:

```

class Foo {
    int size = 27;
    public void setSize(int size) {
        size = size;  // ??? which size equals which size???
    }
}

```

So you've decided that—for overall readability—you want to give the parameter the same name as the instance variable its value is destined for, but how do you resolve the naming collision? Use the keyword `this`. The keyword `this` always, always, always refers to the object currently running. The following code shows this in action:

```

class Foo {
    int size = 27;
    public void setSize(int size) {
        this.size = size;  // this.size means the current object's
                           // instance variable, size. The size
                           // on the right is the parameter
    }
}

```

Array Declarations

In Java, arrays are objects that store multiple variables of the same type, or variables that are all subclasses of the same type. Arrays can hold either primitives or object

references, but the array itself will always be an object on the heap, even if the array is declared to hold primitive elements. In other words, there is no such thing as a primitive array, but you can make an array of primitives.

For the exam, you need to know three things:

- How to make an array reference variable (declare)
- How to make an array object (construct)
- How to populate the array with elements (initialize)

For this objective, you only need to know how to declare an array, we'll cover constructing and initializing arrays in Chapter 3.



Arrays are efficient, but many times you'll want to use one of the Collection types from java.util (including HashMap, ArrayList, and TreeSet). Collection classes offer more flexible ways to access an object (for insertion, deletion, reading, and so on) and unlike arrays, can expand or contract dynamically as you add or remove elements. There's a Collection type for a wide range of needs. Do you need a fast sort? A group of objects with no duplicates? A way to access a name-value pair? Chapter 7 covers them in more detail.

Arrays are declared by stating the type of elements the array will hold (an object or a primitive), followed by square brackets to either side of the identifier.

Declaring an Array of Primitives

```
int[] key; // Square brackets before name (recommended)
int key []; // Square brackets after name (legal but less
            // readable)
```

Declaring an Array of Object References

```
Thread[] threads; // Recommended
Thread threads []; // Legal but less readable
```



When declaring an array reference, you should always put the array brackets immediately after the declared type, rather than after the identifier (variable name). That way, anyone reading the code can easily tell that, for example, key is a reference to an int array object, and not an int primitive.

We can also declare multidimensional arrays, which are in fact arrays of arrays. This can be done in the following manner:

```
String[] [] [] occupantName;
String[] managerName [];
```

The first example is a three-dimensional array (an array of arrays of arrays) and the second is a two-dimensional array. Notice in the second example we have one square bracket before the variable name and one after. This is perfectly legal to the compiler, proving once again that just because it's legal doesn't mean it's right.

exam

Watch

It is never legal to include the size of the array in your declaration. Yes, we know you can do that in some other languages, which is why you might see a question or two that include code similar to the following:

```
int[5] scores;
```

The preceding code won't compile. Remember, the JVM doesn't allocate space until you actually instantiate the array object. That's when size matters.

In Chapter 3, we'll spend a lot of time discussing arrays, how to initialize and use them, and how to deal with multi-dimensional arrays...stay tuned!

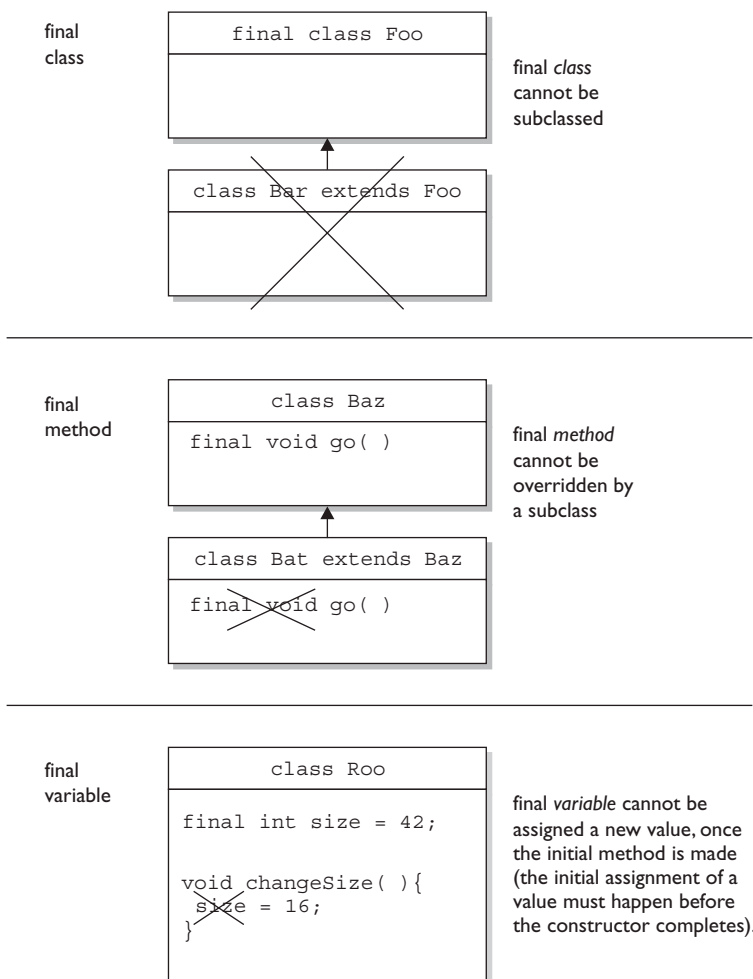
Final Variables

Declaring a variable with the `final` keyword makes it impossible to reinitialize that variable once it has been initialized with an explicit value (notice we said explicit rather than default). For primitives, this means that once the variable is assigned a value, the value can't be altered. For example, if you assign 10 to the `int` variable `x`, then `x` is going to stay 10, forever. So that's straightforward for primitives, but what does it mean to have a `final` object reference variable? A reference variable marked `final` can't ever be reassigned to refer to a different object. The data within the object can be modified, but the reference variable cannot be changed. In other words, a `final` reference still allows you to modify the state of the object it refers

to, but you can't modify the reference variable to make it refer to a different object. Burn this in: there are no `final` objects, only `final` references. We'll explain this in more detail in Chapter 3.

We've now covered how the `final` modifier can be applied to classes, methods, and variables. Figure 1-8 highlights the key points and differences of the various applications of `final`.

FIGURE 1-8 Effect of `final` on variables, methods, and classes



Transient Variables

If you mark an instance variable as `transient`, you're telling the JVM to skip (ignore) this variable when you attempt to serialize the object containing it. Serialization is one of the coolest features of Java; it lets you save (sometimes called "flatten") an object by writing its state (in other words, the value of its instance variables) to a special type of I/O stream. With serialization you can save an object to a file, or even ship it over a wire for reinflating (deserializing) at the other end, in another JVM. Serialization has been added to the exam as of Java 5, and we'll cover it in great detail in Chapter 6.

Volatile Variables

The `volatile` modifier tells the JVM that a thread accessing the variable must always reconcile its own private copy of the variable with the master copy in memory. Say what? Don't worry about it. For the exam, all you need to know about `volatile` is that, as with `transient`, it can be applied only to instance variables. Make no mistake, the idea of multiple threads accessing an instance variable is scary stuff, and very important for any Java programmer to understand. But as you'll see in Chapter 9, you'll probably use synchronization, rather than the `volatile` modifier, to make your data thread-safe.

The `volatile` modifier may also be applied to project managers :)



Static Variables and Methods

The `static` modifier is used to create variables and methods that will exist independently of any instances created for the class. All `static` members exist before you ever make a new instance of a class, and there will be only one copy of a `static` member regardless of the number of instances of that class. In other words, all instances of a given class share the same value for any given `static` variable. We'll cover `static` members in great detail in the next chapter.

Things you can mark as `static`:

- Methods
- Variables
- A class nested within another class, but not within a method (more on this in Chapter 8).
- Initialization blocks

Things you can't mark as `static`:

- Constructors (makes no sense; a constructor is used only to create instances)
- Classes (unless they are nested)
- Interfaces
- Method local inner classes (we'll explore this in Chapter 8)
- Inner class methods and instance variables
- Local variables

Declaring Enums

As of 5.0, Java lets you restrict a variable to having one of only a few pre-defined values—in other words, one value from an enumerated list. (The items in the enumerated list are called, surprisingly, `enums`.)

Using `enums` can help reduce the bugs in your code. For instance, in your coffee shop application you might want to restrict your size selections to `BIG`, `HUGE`, and `OVERWHELMING`. If you let an order for a `LARGE` or a `GRANDE` slip in, it might cause an error. `Enums` to the rescue. With the following simple declaration, you can guarantee that the compiler will stop you from assigning anything to a `CoffeeSize` except `BIG`, `HUGE`, or `OVERWHELMING`:

```
enum CoffeeSize { BIG, HUGE, OVERWHELMING };
```

From then on, the only way to get a `CoffeeSize` will be with a statement something like this:

```
CoffeeSize cs = CoffeeSize.BIG;
```

It's not required that `enum` constants be in all caps, but borrowing from the Sun code convention that constants are named in caps, it's a good idea.

The basic components of an `enum` are its constants (i.e., `BIG`, `HUGE`, and `OVERWHELMING`), although in a minute you'll see that there can be a lot more to an `enum`. `Enums` can be declared as their own separate class, or as a class member, however they must not be declared within a method!

Declaring an enum *outside* a class:

```
enum CoffeeSize { BIG, HUGE, OVERWHELMING } // this cannot be
                                              // private or protected

class Coffee {
    CoffeeSize size;
}

public class CoffeeTest1 {
    public static void main(String[] args) {
        Coffee drink = new Coffee();
        drink.size = CoffeeSize.BIG;           // enum outside class
    }
}
```

The preceding code can be part of a single file. (Remember, the file must be named `CoffeeTest1.java` because that's the name of the `public` class in the file.) The key point to remember is that an enum that isn't enclosed in a class can be declared with only the `public` or default modifier, just like a non-inner class. Here's an example of declaring an enum *inside* a class:

```
class Coffee2 {
    enum CoffeeSize {BIG, HUGE, OVERWHELMING }

    CoffeeSize size;
}

public class CoffeeTest2 {
    public static void main(String[] args) {
        Coffee2 drink = new Coffee2();
        drink.size = Coffee2.CoffeeSize.BIG; // enclosing class
                                              // name required
    }
}
```

The key points to take away from these examples are that enums can be declared as their own class, or enclosed in another class, and that the syntax for accessing an enum's members depends on where the enum was declared.

The following is NOT legal:

```
public class CoffeeTest1 {
    public static void main(String[] args) {
        enum CoffeeSize { BIG, HUGE, OVERWHELMING } // WRONG! Cannot
                                                    // declare enums
                                                    // in methods

        Coffee drink = new Coffee();
        drink.size = CoffeeSize.BIG;
    }
}
```

To make it more confusing for you, the Java language designers made it optional to put a semicolon at the end of the enum declaration (when no other declarations for this enum follow):

```
public class CoffeeTest1 {

    enum CoffeeSize { BIG, HUGE, OVERWHELMING }; // <--semicolon
                                                    // is optional here

    public static void main(String[] args) {
        Coffee drink = new Coffee();
        drink.size = CoffeeSize.BIG;
    }
}
```

So what gets created when you make an enum? The most important thing to remember is that enums are not Strings or ints! Each of the enumerated `CoffeeSize` types are actually instances of `CoffeeSize`. In other words, `BIG` is of type `CoffeeSize`. Think of an enum as a kind of class, that looks something (but not exactly) like this:

```
// conceptual example of how you can think
// about enums

class CoffeeSize {
    public static final CoffeeSize BIG =
        new CoffeeSize("BIG", 0);
    public static final CoffeeSize HUGE =
        new CoffeeSize("HUGE", 1);
    public static final CoffeeSize OVERWHELMING =
        new CoffeeSize("OVERWHELMING", 2);
}
```

```

public CoffeeSize(String enumName, int index) {
    // stuff here
}
public static void main(String[] args) {
    System.out.println(CoffeeSize.BIG);
}
}

```

Notice how each of the enumerated values, `BIG`, `HUGE`, and `OVERWHELMING`, are instances of type `CoffeeSize`. They're represented as `static` and `final`, which in the Java world, is thought of as a constant. Also notice that each enum value knows its index or position...in other words, the order in which enum values are declared matters. You can think of the `CoffeeSize` enums as existing in an array of type `CoffeeSize`, and as you'll see in a later chapter, you can iterate through the values of an enum by invoking the `values()` method on any enum type. (Don't worry about that in this chapter.)

Declaring Constructors, Methods, and Variables in an enum

Because an enum really is a special kind of class, you can do more than just list the enumerated constant values. You can add constructors, instance variables, methods, and something really strange known as a *constant specific class body*. To understand why you might need more in your enum, think about this scenario: imagine you want to know the actual size, in ounces, that map to each of the three `CoffeeSize` constants. For example, you want to know that `BIG` is 8 ounces, `HUGE` is 10 ounces, and `OVERWHELMING` is a whopping 16 ounces.

You could make some kind of a lookup table, using some other data structure, but that would be a poor design and hard to maintain. The simplest way is to treat your enum values (`BIG`, `HUGE`, and `OVERWHELMING`), as objects that can each have their own instance variables. Then you can assign those values at the time the enums are initialized, by passing a value to the enum constructor. This takes a little explaining, but first look at the following code:

```

enum CoffeeSize {
    // 8, 10 & 16 are passed to the constructor
    BIG(8), HUGE(10), OVERWHELMING(16);
    CoffeeSize(int ounces) { // constructor
        this.ounces = ounces;
    }

    private int ounces; // an instance variable
    public int getOunces() {

```

```

        return ounces;
    }
}

class Coffee {
    CoffeeSize size;    // each instance of Coffee has an enum

    public static void main(String[] args) {
        Coffee drink1 = new Coffee();
        drink1.size = CoffeeSize.BIG;

        Coffee drink2 = new Coffee();
        drink2.size = CoffeeSize.OVERWHELMING;

        System.out.println(drink1.size.getOunces()); // prints 8
        for(CoffeeSize cs: CoffeeSize.values())
            System.out.println(cs + " " + cs.getOunces());
    }
}

```

Which produces:

```

8
BIG 8
HUGE 10
OVERWHELMING 16

```

Note: Every enum has a static method, `values()`, that returns an array of the enum's values in the order they're declared.

The key points to remember about enum constructors are

- You can NEVER invoke an enum constructor directly. The enum constructor is invoked automatically, with the arguments you define after the constant value. For example, `BIG(8)` invokes the `CoffeeSize` constructor that takes an `int`, passing the `int` literal 8 to the constructor. (Behind the scenes, of course, you can imagine that `BIG` is also passed to the constructor, but we don't have to know—or care—about the details.)
- You can define more than one argument to the constructor, and you can overload the enum constructors, just as you can overload a normal class constructor. We discuss constructors in much more detail in Chapter 2. To initialize a `CoffeeType` with both the number of ounces and, say, a lid type, you'd pass two arguments to the constructor as `BIG(8, "A")`, which means you have a constructor in `CoffeeSize` that takes both an `int` and a `String`.

CERTIFICATION SUMMARY

After absorbing the material in this chapter, you should be familiar with some of the nuances of the Java language. You may also be experiencing confusion around why you ever wanted to take this exam in the first place. That's normal at this point. If you hear yourself saying, "What was I thinking?" just lie down until it passes. We would like to tell you that it gets easier...that this was the toughest chapter and it's all downhill from here...

Let's briefly review what you'll need to know for the exam.

There will be many questions dealing with keywords indirectly, so be sure you can identify which are keywords and which aren't.

Although naming conventions like the use of camelCase won't be on the exam directly, you will need to understand the basics of JavaBeans naming, which uses camelCase.

You need to understand the rules associated with creating legal identifiers, and the rules associated with source code declarations, including the use of package and import statements.

You now have a good understanding of access control as it relates to classes, methods, and variables. You've looked at how access modifiers (`public`, `protected`, and `private`) define the access control of a class or member.

You learned that `abstract` classes can contain both `abstract` and `nonabstract` methods, but that if even a single method is marked `abstract`, the class must be marked `abstract`. Don't forget that a concrete (`nonabstract`) subclass of an `abstract` class must provide implementations for all the `abstract` methods of the superclass, but that an `abstract` class does not have to implement the `abstract` methods from its superclass. An `abstract` subclass can "pass the buck" to the first concrete subclass.

We covered interface implementation. Remember that interfaces can extend another interface (even multiple interfaces), and that any class that implements an interface must implement all methods from all the interfaces in the inheritance tree of the interface the class is implementing.

You've also looked at the other modifiers including `static`, `final`, `abstract`, `synchronized`, and so on. You've learned how some modifiers can never be combined in a declaration, such as mixing `abstract` with either `final` or `private`.

Keep in mind that there are no `final` objects in Java. A reference variable marked `final` can never be changed, but the object it refers to can be modified.

You've seen that `final` applied to methods means a subclass can't override them, and when applied to a class, the final class can't be subclassed.

Remember that as of Java 5, methods can be declared with a var-arg parameter (which can take from zero to many arguments of the declared type), but that you can have only one var-arg per method, and it must be the method's last parameter.

Make sure you're familiar with the relative sizes of the numeric primitives. Remember that while the values of non-final variables can change, a reference variable's type can never change.

You also learned that arrays are objects that contain many variables of the same type. Arrays can also contain other arrays.

Remember what you've learned about `static` variables and methods, especially that `static` members are per-class as opposed to per-instance. Don't forget that a `static` method can't directly access an instance variable from the class it's in, because it doesn't have an explicit reference to any particular instance of the class.

Finally, we covered a feature new to Java 5, enums. An enum is a much safer and more flexible way to implement constants than was possible in earlier versions of Java. Because they are a special kind of class, enums can be declared very simply, or they can be quite complex—including such attributes as methods, variables, constructors, and a special type of inner class called a constant specific class body.

Before you hurl yourself at the practice test, spend some time with the following optimistically named "Two-Minute Drill." Come back to this particular drill often, as you work through this book and especially when you're doing that last-minute cramming. Because—and here's the advice you wished your mother had given you before you left for college—it's not what you know, it's when you know it.

For the exam, knowing what you can't do with the Java language is just as important as knowing what you can do. Give the sample questions a try! They're very similar to the difficulty and structure of the real exam questions, and should be an eye opener for how difficult the exam can be. Don't worry if you get a lot of them wrong. If you find a topic that you are weak in, spend more time reviewing and studying. Many programmers need two or three serious passes through a chapter (or an individual objective) before they can answer the questions confidently.



TWO-MINUTE DRILL

Remember that in this chapter, when we talk about classes, we're referring to non-inner classes, or *top-level* classes. We'll devote all of Chapter 8 to inner classes.

Identifiers (Objective 1.3)

- ☐ Identifiers can begin with a letter, an underscore, or a currency character.
- ☐ After the first character, identifiers can also include digits.
- ☐ Identifiers can be of any length.
- ☐ JavaBeans methods must be named using camelCase, and depending on the method's purpose, must start with `set`, `get`, `is`, `add`, or `remove`.

Declaration Rules (Objective 1.1)

- ☐ A source code file can have only one `public` class.
- ☐ If the source file contains a `public` class, the filename must match the `public` class name.
- ☐ A file can have only one `package` statement, but multiple `imports`.
- ☐ The `package` statement (if any) must be the first (non-comment) line in a source file.
- ☐ The `import` statements (if any) must come after the `package` and before the class declaration.
- ☐ If there is no `package` statement, `import` statements must be the first (non-comment) statements in the source file.
- ☐ `package` and `import` statements apply to all classes in the file.
- ☐ A file can have more than one nonpublic class.
- ☐ Files with no `public` classes have no naming restrictions.

Class Access Modifiers (Objective 1.1)

- ☐ There are three access modifiers: `public`, `protected`, and `private`.
- ☐ There are four access levels: `public`, `protected`, default, and `private`.
- ☐ Classes can have only `public` or default access.
- ☐ A class with default access can be seen only by classes within the same package.
- ☐ A class with `public` access can be seen by all classes from all packages.

- ❑ Class visibility revolves around whether code in one class can
 - ❑ Create an instance of another class
 - ❑ Extend (or subclass), another class
 - ❑ Access methods and variables of another class

Class Modifiers (Nonaccess) (Objective 1.2)

- ❑ Classes can also be modified with `final`, `abstract`, or `strictfp`.
- ❑ A class cannot be both `final` and `abstract`.
- ❑ A `final` class cannot be subclassed.
- ❑ An `abstract` class cannot be instantiated.
- ❑ A single `abstract` method in a class means the whole class must be `abstract`.
- ❑ An `abstract` class can have both `abstract` and `nonabstract` methods.
- ❑ The first concrete class to extend an `abstract` class must implement all of its `abstract` methods.

Interface Implementation (Objective 1.2)

- ❑ Interfaces are contracts for what a class can do, but they say nothing about the way in which the class must do it.
- ❑ Interfaces can be implemented by any class, from any inheritance tree.
- ❑ An interface is like a 100-percent `abstract` class, and is implicitly `abstract` whether you type the `abstract` modifier in the declaration or not.
- ❑ An interface can have only `abstract` methods, no concrete methods allowed.
- ❑ Interface methods are by default `public` and `abstract`—explicit declaration of these modifiers is optional.
- ❑ Interfaces can have constants, which are always implicitly `public`, `static`, and `final`.
- ❑ Interface constant declarations of `public`, `static`, and `final` are optional in any combination.
- ❑ A legal `nonabstract` implementing class has the following properties:
 - ❑ It provides concrete implementations for the interface's methods.
 - ❑ It must follow all legal override rules for the methods it implements.
 - ❑ It must not declare any new checked exceptions for an implementation method.

- ☐ It must not declare any checked exceptions that are broader than the exceptions declared in the interface method.
- ☐ It may declare runtime exceptions on any interface method implementation regardless of the interface declaration.
- ☐ It must maintain the exact signature (allowing for covariant returns) and return type of the methods it implements (but does not have to declare the exceptions of the interface).
- ☐ A class implementing an interface can itself be `abstract`.
- ☐ An `abstract` implementing class does not have to implement the interface methods (but the first concrete subclass must).
- ☐ A class can extend only one class (no multiple inheritance), but it can implement many interfaces.
- ☐ Interfaces can extend one or more other interfaces.
- ☐ Interfaces cannot extend a class, or implement a class or interface.
- ☐ When taking the exam, verify that interface and class declarations are legal before verifying other code logic.

Member Access Modifiers (Objectives 1.3 and 1.4)

- ☐ Methods and instance (nonlocal) variables are known as "members."
- ☐ Members can use all four access levels: `public`, `protected`, `default`, `private`.
- ☐ Member access comes in two forms:
 - ☐ Code in one class can access a member of another class.
 - ☐ A subclass can inherit a member of its superclass.
- ☐ If a class cannot be accessed, its members cannot be accessed.
- ☐ Determine class visibility before determining member visibility.
- ☐ `public` members can be accessed by all other classes, even in other packages.
- ☐ If a superclass member is `public`, the subclass inherits it—regardless of package.
- ☐ Members accessed without the dot operator (`.`) must belong to the same class.
- ☐ `this.` always refers to the currently executing object.
- ☐ `this.aMethod()` is the same as just invoking `aMethod()`.
- ☐ `private` members can be accessed only by code in the same class.
- ☐ `private` members are not visible to subclasses, so `private` members cannot be inherited.

- ❑ Default and `protected` members differ only when subclasses are involved:
 - ❑ Default members can be accessed only by classes in the same package.
 - ❑ `protected` members can be accessed by other classes in the same package, plus subclasses regardless of package.
 - ❑ `protected` = package plus kids (kids meaning subclasses).
 - ❑ For subclasses outside the package, the `protected` member can be accessed only through inheritance; a subclass outside the package cannot access a `protected` member by using a reference to a superclass instance (in other words, inheritance is the only mechanism for a subclass outside the package to access a `protected` member of its superclass).
- ❑ A `protected` member inherited by a subclass from another package is not accessible to any other class in the subclass package, except for the subclass' own subclasses.

Local Variables (Objective 1.3)

- ❑ Local (method, automatic, or stack) variable declarations cannot have access modifiers.
- ❑ `final` is the only modifier available to local variables.
- ❑ Local variables don't get default values, so they must be initialized before use.

Other Modifiers—Members (Objective 1.3)

- ❑ `final` methods cannot be overridden in a subclass.
- ❑ `abstract` methods are declared, with a signature, a return type, and an optional throws clause, but are not implemented.
- ❑ `abstract` methods end in a semicolon—no curly braces.
- ❑ Three ways to spot a non-`abstract` method:
 - ❑ The method is not marked `abstract`.
 - ❑ The method has curly braces.
 - ❑ The method has code between the curly braces.
- ❑ The first non-`abstract` (concrete) class to extend an `abstract` class must implement all of the `abstract` class' `abstract` methods.
- ❑ The `synchronized` modifier applies only to methods and code blocks.
- ❑ `synchronized` methods can have any access control and can also be marked `final`.

- ☐ abstract methods must be implemented by a subclass, so they must be inheritable. For that reason:
 - ☐ abstract methods cannot be `private`.
 - ☐ abstract methods cannot be `final`.
- ☐ The `native` modifier applies only to methods.
- ☐ The `strictfp` modifier applies only to classes and methods.

Methods with var-args (Objective 1.4)

- ☐ As of Java 5, methods can declare a parameter that accepts from zero to many arguments, a so-called var-arg method.
- ☐ A var-arg parameter is declared with the syntax `type... name`; for instance:
`doStuff(int... x) { }`
- ☐ A var-arg method can have only one var-arg parameter.
- ☐ In methods with normal parameters and a var-arg, the var-arg must come last.

Variable Declarations (Objective 1.3)

- ☐ Instance variables can
 - ☐ Have any access control
 - ☐ Be marked `final` or `transient`
- ☐ Instance variables can't be `abstract`, `synchronized`, `native`, or `strictfp`.
- ☐ It is legal to declare a local variable with the same name as an instance variable; this is called "shadowing."
- ☐ `final` variables have the following properties:
 - ☐ `final` variables cannot be reinitialized once assigned a value.
 - ☐ `final` reference variables cannot refer to a different object once the object has been assigned to the `final` variable.
 - ☐ `final` reference variables must be initialized before the constructor completes.
- ☐ There is no such thing as a `final` object. An object reference marked `final` does not mean the object itself is immutable.
- ☐ The `transient` modifier applies only to instance variables.
- ☐ The `volatile` modifier applies only to instance variables.

Array Declarations (Objective 1.3)

- ☐ Arrays can hold primitives or objects, but the array itself is always an object.
- ☐ When you declare an array, the brackets can be to the left or right of the variable name.
- ☐ It is never legal to include the size of an array in the declaration.
- ☐ An array of objects can hold any object that passes the IS-A (or instanceof) test for the declared type of the array. For example, if Horse extends Animal, then a Horse object can go into an Animal array.

Static Variables and Methods (Objective 1.4)

- ☐ They are not tied to any particular instance of a class.
- ☐ No classes instances are needed in order to use `static` members of the class.
- ☐ There is only one copy of a `static` variable / class and all instances share it.
- ☐ `static` methods do not have direct access to non-static members.

Enums (Objective 1.3)

- ☐ An enum specifies a list of constant values assigned to a type.
- ☐ An enum is NOT a String or an int; an enum constant's type is the enum type. For example, SUMMER and FALL are of the enum type Season.
- ☐ An enum can be declared outside or inside a class, but NOT in a method.
- ☐ An enum declared outside a class must NOT be marked `static`, `final`, `abstract`, `protected`, or `private`.
- ☐ Enums can contain constructors, methods, variables, and constant class bodies.
- ☐ enum constants can send arguments to the enum constructor, using the syntax `BIG(8)`, where the int literal 8 is passed to the enum constructor.
- ☐ enum constructors can have arguments, and can be overloaded.
- ☐ enum constructors can NEVER be invoked directly in code. They are always called automatically when an enum is initialized.
- ☐ The semicolon at the end of an enum declaration is optional. These are legal:

```
enum Foo { ONE, TWO, THREE}  
enum Foo { ONE, TWO, THREE};
```
- ☐ `MyEnum.values()` returns an array of `MyEnum`'s values.

SELF TEST

The following questions will help you measure your understanding of the material presented in this chapter. Read all of the choices carefully, as there may be more than one correct answer. Choose all correct answers for each question. Stay focused.

If you have a rough time with these at first, don't beat yourself up. Be positive. Repeat nice affirmations to yourself like, "I am smart enough to understand enums" and "OK, so that other guy knows enums better than I do, but I bet he can't <insert something you *are* good at> like me."

1. Which is true? (Choose all that apply.)
 - A. "X extends Y" is correct if and only if X is a class and Y is an interface
 - B. "X extends Y" is correct if and only if X is an interface and Y is a class
 - C. "X extends Y" is correct if X and Y are either both classes or both interfaces
 - D. "X extends Y" is correct for all combinations of X and Y being classes and/or interfaces

2. Which method names follow the JavaBeans standard? (Choose all that apply.)
 - A. addSize
 - B. getCust
 - C. deleteRep
 - D. isColorado
 - E. putDimensions

3. Given:


```

1. class Voop {
2.     public static void main(String [] args) {
3.         doStuff(1);
4.         doStuff(1,2);
5.     }
6.     // insert code here
7. }
```

Which, inserted independently at line 6, will compile? (Choose all that apply.)

- A. `static void doStuff(int... doArgs) { }`
- B. `static void doStuff(int[] doArgs) { }`
- C. `static void doStuff(int doArgs...) { }`
- D. `static void doStuff(int... doArgs, int y) { }`
- E. `static void doStuff(int x, int... doArgs) { }`

4. Given:

```
1. enum Animals {
2.     DOG("woof"), CAT("meow"), FISH("burble");
3.     String sound;
4.     Animals(String s) { sound = s; }
5. }
6. class TestEnum {
7.     static Animals a;
8.     public static void main(String[] args) {
9.         System.out.println(a.DOG.sound + " " + a.FISH.sound);
10.    }
11. }
```

What is the result?

- A. woof burble
- B. Multiple compilation errors
- C. Compilation fails due to an error on line 2
- D. Compilation fails due to an error on line 3
- E. Compilation fails due to an error on line 4
- F. Compilation fails due to an error on line 9

5. Given two files:

```
1. package pkgA;
2. public class Foo {
3.     int a = 5;
4.     protected int b = 6;
5.     public int c = 7;
6. }

3. package pkgB;
4. import pkgA.*;
5. public class Baz {
6.     public static void main(String[] args) {
7.         Foo f = new Foo();
8.         System.out.print(" " + f.a);
9.         System.out.print(" " + f.b);
10.        System.out.println(" " + f.c);
11.    }
12. }
```

What is the result? (Choose all that apply.)

- A. 5 6 7
- B. 5 followed by an exception
- C. Compilation fails with an error on line 7
- D. Compilation fails with an error on line 8
- E. Compilation fails with an error on line 9
- F. Compilation fails with an error on line 10

6. Given:

```
1. public class Electronic implements Device
    { public void doIt() { } }
2.
3. abstract class Phone1 extends Electronic { }
4.
5. abstract class Phone2 extends Electronic
    { public void doIt(int x) { } }
6.
7. class Phone3 extends Electronic implements Device
    { public void doStuff() { } }
8.
9. interface Device { public void doIt(); }
```

What is the result? (Choose all that apply.)

- A. Compilation succeeds
- B. Compilation fails with an error on line 1
- C. Compilation fails with an error on line 3
- D. Compilation fails with an error on line 5
- E. Compilation fails with an error on line 7
- F. Compilation fails with an error on line 9

7. Given:

```
4. class Announce {
5.     public static void main(String[] args) {
6.         for(int __x = 0; __x < 3; __x++) ;
7.         int #1b = 7;
```

```
8.      long [] x [5];
9.      Boolean [][]ba[];
10.     enum Traffic { RED, YELLOW, GREEN };
11.     }
12. }
```

What is the result? (Choose all that apply.)

- A. Compilation succeeds
- B. Compilation fails with an error on line 6
- C. Compilation fails with an error on line 7
- D. Compilation fails with an error on line 8
- E. Compilation fails with an error on line 9
- F. Compilation fails with an error on line 10

8. Given:

```
3. public class TestDays {
4.     public enum Days { MON, TUE, WED };
5.     public static void main(String[] args) {
6.         for(Days d : Days.values() )
7.             ;
8.         Days [] d2 = Days.values();
9.         System.out.println(d2[2]);
10.    }
11. }
```

What is the result? (Choose all that apply.)

- A. TUE
- B. WED
- C. The output is unpredictable
- D. Compilation fails due to an error on line 4
- E. Compilation fails due to an error on line 6
- F. Compilation fails due to an error on line 8
- G. Compilation fails due to an error on line 9

9. Given:

```
4. public class Frodo extends Hobbit {  
5.     public static void main(String[] args) {  
6.         Short myGold = 7;  
7.         System.out.println(countGold(myGold, 6));  
8.     }  
9. }  
10. class Hobbit {  
11.     int countGold(int x, int y) { return x + y; }  
12. }
```

What is the result?

- A. 13
- B. Compilation fails due to multiple errors
- C. Compilation fails due to an error on line 6
- D. Compilation fails due to an error on line 7
- E. Compilation fails due to an error on line 11

SELF TEST ANSWERS

1. Which is true? (Choose all that apply.)
- A. "X extends Y" is correct if and only if X is a class and Y is an interface
 - B. "X extends Y" is correct if and only if X is an interface and Y is a class
 - C. "X extends Y" is correct if X and Y are either both classes or both interfaces
 - D. "X extends Y" is correct for all combinations of X and Y being classes and/or interfaces

Answer:

- ☒ **C** is correct.
- ☒ **A** is incorrect because classes implement interfaces, they don't extend them. **B** is incorrect because interfaces only "inherit from" other interfaces. **D** is incorrect based on the preceding rules. (Objective 1.2)

2. Which method names follow the JavaBeans standard? (Choose all that apply.)

- A. addSize
- B. getCust
- C. deleteRep
- D. isColorado
- E. putDimensions

Answer:

- ☒ **B** and **D** use the valid prefixes 'get' and 'is'.
- ☒ **A** is incorrect because 'add' can be used only with Listener methods. **C** and **E** are incorrect because 'delete' and 'put' are not standard JavaBeans name prefixes. (Objective 1.4)

3. Given:

```
1. class Voop {  
2.     public static void main(String[] args) {  
3.         doStuff(1);  
4.         doStuff(1,2);  
5.     }  
6.     // insert code here  
7. }
```

Which, inserted independently at line 6, will compile? (Choose all that apply.)

- A. `static void doStuff(int... doArgs) { }`
- B. `static void doStuff(int[] doArgs) { }`
- C. `static void doStuff(int doArgs...) { }`
- D. `static void doStuff(int... doArgs, int y) { }`
- E. `static void doStuff(int x, int... doArgs) { }`

Answer:

- ☒ A and E use valid var-args syntax.
- ☒ B and C are invalid var-arg syntax, and D is invalid because the var-arg must be the last of a method's arguments. (Objective 1.4)

4. Given:

```

1. enum Animals {
2.     DOG("woof"), CAT("meow"), FISH("burble");
3.     String sound;
4.     Animals(String s) { sound = s; }
5. }
6. class TestEnum {
7.     static Animals a;
8.     public static void main(String [] args) {
9.         System.out.println(a.DOG.sound + " " + a.FISH.sound);
10.    }
11. }
```

What is the result?

- A. `woof burble`
- B. Multiple compilation errors
- C. Compilation fails due to an error on line 2
- D. Compilation fails due to an error on line 3
- E. Compilation fails due to an error on line 4
- F. Compilation fails due to an error on line 9

Answer:

- ☒ A is correct; enums can have constructors and variables.
- ☒ B, C, D, E, and F are incorrect; these lines all use correct syntax. (Objective 1.3)

5. Given two files:

```

1. package pkgA;
2. public class Foo {
3.     int a = 5;
4.     protected int b = 6;
5.     public int c = 7;
6. }

3. package pkgB;
4. import pkgA.*;
5. public class Baz {
6.     public static void main(String[] args) {
7.         Foo f = new Foo();
8.         System.out.print(" " + f.a);
9.         System.out.print(" " + f.b);
10.        System.out.print(" " + f.c);
11.    }
12. }
```

What is the result? (Choose all that apply.)

- A. 5 6 7
- B. 5 followed by an exception
- C. Compilation fails with an error on line 7
- D. Compilation fails with an error on line 8
- E. Compilation fails with an error on line 9
- F. Compilation fails with an error on line 10

Answer:

- ☒ D and E are correct. Variable a has default access, so it cannot be accessed from outside the package. Variable b has protected access in pkgA.
- ☒ A, B, C, and F are incorrect based on the above information. (Objective 1.1)

6. Given:

```

1. public class Electronic implements Device
    { public void doIt() { } }
2.
3. abstract class Phone1 extends Electronic { }
4.
5. abstract class Phone2 extends Electronic
    { public void doIt(int x) { } }
6.
```

```
7. class Phone3 extends Electronic implements Device
    { public void doStuff() { } }
8.
9. interface Device { public void doIt(); }
```

What is the result? (Choose all that apply.)

- A. Compilation succeeds
- B. Compilation fails with an error on line 1
- C. Compilation fails with an error on line 3
- D. Compilation fails with an error on line 5
- E. Compilation fails with an error on line 7
- F. Compilation fails with an error on line 9

Answer:

- ☒ A is correct; all of these are legal declarations.
- ☒ B, C, D, E, and F are incorrect based on the above information. (Objective 1.2)

7. Given:

```
4. class Announce {
5.     public static void main(String[] args) {
6.         for(int __x = 0; __x < 3; __x++) ;
7.         int #1b = 7;
8.         long [] x [5];
9.         Boolean []ba[];
10.        enum Traffic { RED, YELLOW, GREEN };
11.    }
12. }
```

What is the result? (Choose all that apply.)

- A. Compilation succeeds
- B. Compilation fails with an error on line 6
- C. Compilation fails with an error on line 7
- D. Compilation fails with an error on line 8
- E. Compilation fails with an error on line 9
- F. Compilation fails with an error on line 10

Answer:

- ☒ **C, D, and F** are correct. Variable names cannot begin with a #, an array declaration can't include a size without an instantiation, and enums can't be declared within a method.
- ☒ **A, B, and E** are incorrect based on the above information. (Objective 1.3)

8. Given:

```

3. public class TestDays {
4.     public enum Days { MON, TUE, WED };
5.     public static void main(String[] args) {
6.         for(Days d : Days.values() )
7.             ;
8.         Days [] d2 = Days.values();
9.         System.out.println(d2[2]);
10.    }
11. }
```

What is the result? (Choose all that apply.)

- A.** TUE
- B.** WED
- C.** The output is unpredictable
- D.** Compilation fails due to an error on line 4
- E.** Compilation fails due to an error on line 6
- F.** Compilation fails due to an error on line 8
- G.** Compilation fails due to an error on line 9

Answer:

- ☒ **B** is correct. Every enum comes with a `static values()` method that returns an array of the enum's values, in the order in which they are declared in the enum.
- ☒ **A, C, D, E, F, and G** are incorrect based on the above information. (Objective 1.3)

9. Given:

```

4. public class Frodo extends Hobbit {
5.     public static void main(String[] args) {
6.         Short myGold = 7;
7.         System.out.println(countGold(myGold, 6));
8.     }
9. }
10. class Hobbit {
11.     int countGold(int x, int y) { return x + y; }
12. }
```

What is the result?

- A. 13
- B. Compilation fails due to multiple errors
- C. Compilation fails due to an error on line 6
- D. Compilation fails due to an error on line 7
- E. Compilation fails due to an error on line 11

Answer:

- ☒ D is correct. The `Short myGold` is autoboxed correctly, but the `countGold()` method cannot be invoked from a static context.
- ☒ A, B, C, and E are incorrect based on the above information. (Objective 1.4)



2

Object Orientation

CERTIFICATION OBJECTIVES

- Declare Interfaces
- Declare, Initialize, and Use Class Members
- Use Overloading and Overriding
- Develop Constructors
- Describe Encapsulation, Coupling, and Cohesion
- Use Polymorphism
- Relate Modifiers and Inheritance
- Use Superclass Constructors and Overloaded Constructors
- Use IS-A and HAS-A Relationships
- ✓ Two-Minute Drill
- Q&A Self Test

Being an SCJP 6 means you must be at one with the object-oriented aspects of Java. You must dream of inheritance hierarchies, the power of polymorphism must flow through you, cohesion and loose coupling must become second nature to you, and composition must be your bread and butter. This chapter will prepare you for all of the object-oriented objectives and questions you'll encounter on the exam. We have heard of many experienced Java programmers who haven't really become fluent with the object-oriented tools that Java provides, so we'll start at the beginning.

CERTIFICATION OBJECTIVE

Encapsulation (Exam Objective 5.1)

5.1 Develop code that implements tight encapsulation, loose coupling, and high cohesion in classes, and describe the benefits.

Imagine you wrote the code for a class, and another dozen programmers from your company all wrote programs that used your class. Now imagine that later on, you didn't like the way the class behaved, because some of its instance variables were being set (by the other programmers from within their code) to values you hadn't anticipated. *Their* code brought out errors in *your* code. (Relax, this is just hypothetical.) Well, it is a Java program, so you should be able just to ship out a newer version of the class, which they could replace in their programs without changing any of their own code.

This scenario highlights two of the promises/benefits of Object Orientation (OO): flexibility and maintainability. But those benefits don't come automatically. You have to do something. You have to write your classes and code in a way that supports flexibility and maintainability. So what if Java supports OO? It can't design your code for you. For example, imagine if you made your class with `public` instance variables, and those other programmers were setting the instance variables directly, as the following code demonstrates:

```
public class BadOO {  
    public int size;
```

```

    public int weight;
    ...
}
public class ExploitBadOO {
    public static void main (String [] args) {
        BadOO b = new BadOO();
        b.size = -5; // Legal but bad!!
    }
}

```

And now you're in trouble. How are you going to change the class in a way that lets you handle the issues that come up when somebody changes the `size` variable to a value that causes problems? Your only choice is to go back in and write method code for adjusting `size` (a `setSize(int a)` method, for example), and then protect the `size` variable with, say, a private access modifier. But as soon as you make that change to your code, you break everyone else's!

The ability to make changes in your implementation code without breaking the code of others who use your code is a key benefit of encapsulation. You want to hide implementation details behind a public programming interface. By interface, we mean the set of accessible methods your code makes available for other code to call—in other words, your code's API. By hiding implementation details, you can rework your method code (perhaps also altering the way variables are used by your class) without forcing a change in the code that calls your changed method.

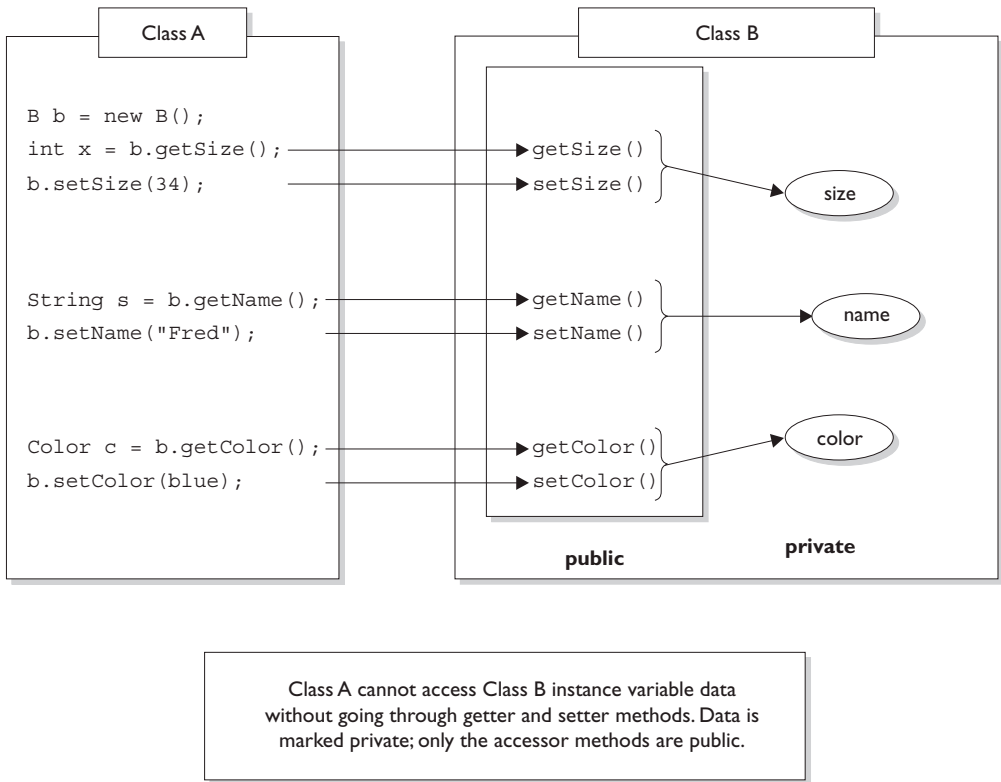
If you want maintainability, flexibility, and extensibility (and of course, you do), your design must include encapsulation. How do you do that?

- Keep instance variables protected (with an access modifier, often `private`).
- Make `public` accessor methods, and force calling code to use those methods rather than directly accessing the instance variable.
- For the methods, use the JavaBeans naming convention of `set<someProperty>` and `get<someProperty>`.

Figure 2-1 illustrates the idea that encapsulation forces callers of our code to go through methods rather than accessing variables directly.

FIGURE 2-1

The nature of encapsulation



We call the access methods getters and setters although some prefer the fancier terms accessors and mutators. (Personally, we don't like the word "mutate".) Regardless of what you call them, they're methods that other programmers must go through in order to access your instance variables. They look simple, and you've probably been using them forever:

```
public class Box {
    // protect the instance variable; only an instance
    // of Box can access it
    private int size;
    // Provide public getters and setters
    public int getSize() {
        return size;
    }
}
```



```

    }
    public void setSize(int newSize) {
        size = newSize;
    }
}

```

Wait a minute...how useful is the previous code? It doesn't even do any validation or processing. What benefit can there be from having getters and setters that add no additional functionality? The point is, you can change your mind later, and add more code to your methods without breaking your API. Even if today you don't think you really need validation or processing of the data, good OO design dictates that you plan for the future. To be safe, force calling code to go through your methods rather than going directly to instance variables. *Always*. Then you're free to rework your method implementations later, without risking the wrath of those dozen programmers who know where you live.

exam

Watch

Look out for code that appears to be asking about the behavior of a method, when the problem is actually a lack of encapsulation. Look at the following example, and see if you can figure out what's going on:

```

class Foo {
    public int left = 9;
    public int right = 3;
    public void setLeft(int leftNum) {
        left = leftNum;
        right = leftNum/3;
    }
    // lots of complex test code here
}

```

Now consider this question: Is the value of right always going to be one-third the value of left? It looks like it will, until you realize that users of the Foo class don't need to use the setLeft() method! They can simply go straight to the instance variables and change them to any arbitrary int value.

CERTIFICATION OBJECTIVE

Inheritance, Is-A, Has-A (Exam Objective 5.5)

5.5 Develop code that implements "is-a" and/or "has-a" relationships.

Inheritance is everywhere in Java. It's safe to say that it's almost (almost?) impossible to write even the tiniest Java program without using inheritance. In order to explore this topic we're going to use the `instanceof` operator, which we'll discuss in more detail in Chapter 4. For now, just remember that `instanceof` returns `true` if the reference variable being tested is of the type being compared to. This code:

```
class Test {
    public static void main(String [] args) {
        Test t1 = new Test();
        Test t2 = new Test();
        if (!t1.equals(t2))
            System.out.println("they're not equal");
        if (t1 instanceof Object)
            System.out.println("t1's an Object");
    }
}
```

Produces the output:

```
they're not equal
t1's an Object
```

Where did that `equals` method come from? The reference variable `t1` is of type `Test`, and there's no `equals` method in the `Test` class. Or is there? The second `if` test asks whether `t1` is an instance of class `Object`, and because it is (more on that soon), the `if` test succeeds.

Hold on...how can `t1` be an instance of type `Object`, we just said it was of type `Test`? I'm sure you're way ahead of us here, but it turns out that every class in Java is a subclass of class `Object`, (except of course class `Object` itself). In other words, every class you'll ever use or ever write will inherit from class `Object`. You'll always have an `equals` method, a `clone` method, `notify`, `wait`, and others, available to use. Whenever you create a class, you automatically inherit all of class `Object`'s methods.

Why? Let's look at that `equals` method for instance. Java's creators correctly assumed that it would be very common for Java programmers to want to compare instances of their classes to check for equality. If class `Object` didn't have an `equals` method, you'd have to write one yourself; you and every other Java programmer. That one `equals` method has been inherited billions of times. (To be fair, `equals` has also been *overridden* billions of times, but we're getting ahead of ourselves.)

For the exam you'll need to know that you can create inheritance relationships in Java by *extending* a class. It's also important to understand that the two most common reasons to use inheritance are

- To promote code reuse
- To use polymorphism

Let's start with reuse. A common design approach is to create a fairly generic version of a class with the intention of creating more specialized subclasses that inherit from it. For example:

```
class GameShape {
    public void displayShape() {
        System.out.println("displaying shape");
    }
    // more code
}

class PlayerPiece extends GameShape {
    public void movePiece() {
        System.out.println("moving game piece");
    }
    // more code
}

public class TestShapes {
    public static void main (String[] args) {
        PlayerPiece shape = new PlayerPiece();
        shape.displayShape();
        shape.movePiece();
    }
}
```

Outputs:

```
displaying shape
moving game piece
```

Notice that the `PlayingPiece` class inherits the generic `display()` method from the less-specialized class `GameShape`, and also adds its own method, `movePiece()`. Code reuse through inheritance means that methods with generic functionality (like `display()`)—that could apply to a wide range of different kinds of shapes in a game—don't have to be reimplemented. That means all specialized subclasses of `GameShape` are guaranteed to have the capabilities of the more generic superclass. You don't want to have to rewrite the `display()` code in each of your specialized components of an online game.

But you knew that. You've experienced the pain of duplicate code when you make a change in one place and have to track down all the other places where that same (or very similar) code exists.

The second (and related) use of inheritance is to allow your classes to be accessed polymorphically—a capability provided by interfaces as well, but we'll get to that in a minute. Let's say that you have a `GameLauncher` class that wants to loop through a list of different kinds of `GameShape` objects, and invoke `display()` on each of them. At the time you write this class, you don't know every possible kind of `GameShape` subclass that anyone else will ever write. And you sure don't want to have to redo *your* code just because somebody decided to build a `Dice` shape six months later.

The beautiful thing about polymorphism ("many forms") is that you can treat any *subclass* of `GameShape` as a `GameShape`. In other words, you can write code in your `GameLauncher` class that says, "I don't care what kind of object you are as long as you inherit from (extend) `GameShape`. And as far as I'm concerned, if you extend `GameShape` then you've definitely got a `display()` method, so I know I can call it."

Imagine we now have two specialized subclasses that extend the more generic `GameShape` class, `PlayerPiece` and `TilePiece`:

```
class GameShape {
    public void displayShape() {
        System.out.println("displaying shape");
    }
    // more code
}
```

```

class PlayerPiece extends GameShape {
    public void movePiece() {
        System.out.println("moving game piece");
    }
    // more code
}

class TilePiece extends GameShape {
    public void getAdjacent() {
        System.out.println("getting adjacent tiles");
    }
    // more code
}

```

Now imagine a test class has a method with a declared argument type of `GameShape`, that means it can take any kind of `GameShape`. In other words, any subclass of `GameShape` can be passed to a method with an argument of type `GameShape`. This code

```

public class TestShapes {
    public static void main (String[] args) {
        PlayerPiece player = new PlayerPiece();
        TilePiece tile = new TilePiece();
        doShapes(player);
        doShapes(tile);
    }

    public static void doShapes(GameShape shape) {
        shape.displayShape();
    }
}

```

Outputs:

```

displaying shape
displaying shape

```

The key point is that the `doShapes()` method is declared with a `GameShape` argument but can be passed any subtype (in this example, a subclass) of `GameShape`. The method can then invoke any method of `GameShape`, without any concern for the actual runtime class type of the object passed to the method. There are

implications, though. The `doShapes()` method knows only that the objects are a type of `GameShape`, since that's how the parameter is declared. And using a reference variable declared as type `GameShape`—regardless of whether the variable is a method parameter, local variable, or instance variable—means that *only* the methods of `GameShape` can be invoked on it. The methods you can call on a reference are totally dependent on the *declared* type of the variable, no matter what the actual object is, that the reference is referring to. That means you can't use a `GameShape` variable to call, say, the `getAdjacent()` method even if the object passed in is of type `TilePiece`. (We'll see this again when we look at interfaces.)

IS-A and HAS-A Relationships

For the exam you need to be able to look at code and determine whether the code demonstrates an IS-A or HAS-A relationship. The rules are simple, so this should be one of the few areas where answering the questions correctly is almost a no-brainer.

IS-A

In OO, the concept of IS-A is based on class inheritance or interface implementation. IS-A is a way of saying, "this thing is a type of that thing." For example, a Mustang is a type of horse, so in OO terms we can say, "Mustang IS-A Horse." Subaru IS-A Car. Broccoli IS-A Vegetable (not a very fun one, but it still counts). You express the IS-A relationship in Java through the keywords `extends` (for class inheritance) and `implements` (for interface implementation).

```
public class Car {
    // Cool Car code goes here
}

public class Subaru extends Car {
    // Important Subaru-specific stuff goes here
    // Don't forget Subaru inherits accessible Car members which
    // can include both methods and variables.
}
```

A Car is a type of Vehicle, so the inheritance tree might start from the Vehicle class as follows:

```
public class Vehicle { ... }
public class Car extends Vehicle { ... }
public class Subaru extends Car { ... }
```

In OO terms, you can say the following:

Vehicle is the superclass of Car.
Car is the subclass of Vehicle.
Car is the superclass of Subaru.
Subaru is the subclass of Vehicle.
Car inherits from Vehicle.
Subaru inherits from both Vehicle and Car.
Subaru is derived from Car.
Car is derived from Vehicle.
Subaru is derived from Vehicle.
Subaru is a subtype of both Vehicle and Car.

Returning to our IS-A relationship, the following statements are true:

"Car extends Vehicle" means "Car IS-A Vehicle."

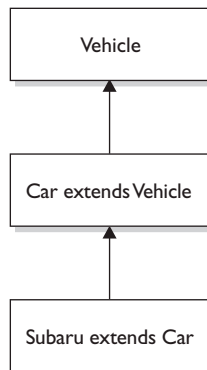
"Subaru extends Car" means "Subaru IS-A Car."

And we can also say:

"Subaru IS-A Vehicle" because a class is said to be "a type of" anything further up in its inheritance tree. If the expression (`Foo instanceof Bar`) is true, then class `Foo` IS-A `Bar`, even if `Foo` doesn't directly extend `Bar`, but instead extends some other class that is a subclass of `Bar`. Figure 2-2 illustrates the inheritance tree for `Vehicle`, `Car`, and `Subaru`. The arrows move from the subclass to the superclass. In other words, a class' arrow points toward the class from which it extends.

FIGURE 2-2

Inheritance tree
for `Vehicle`, `Car`,
`Subaru`



HAS-A

HAS-A relationships are based on usage, rather than inheritance. In other words, class A HAS-A B if code in class A has a reference to an instance of class B. For example, you can say the following,

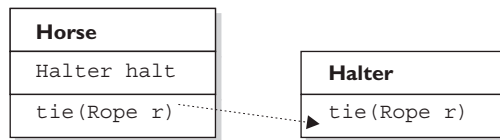
A Horse IS-A Animal. A Horse HAS-A Halter.
The code might look like this:

```
public class Animal { }
public class Horse extends Animal {
    private Halter myHalter;
}
```

In the preceding code, the Horse class has an instance variable of type Halter, so you can say that "Horse HAS-A Halter." In other words, Horse has a reference to a Halter. Horse code can use that Halter reference to invoke methods on the Halter, and get Halter behavior without having Halter-related code (methods) in the Horse class itself. Figure 2-3 illustrates the HAS-A relationship between Horse and Halter.

FIGURE 2-3

HAS-A
relationship
between Horse
and Halter



Horse class has a Halter, because Horse declares an instance variable of type Halter. When code invokes `tie()` on a Horse instance, the Horse invokes `tie()` on the Horse object's Halter instance variable.

HAS-A relationships allow you to design classes that follow good OO practices by not having monolithic classes that do a gazillion different things. Classes (and their resulting objects) should be specialists. As our friend Andrew says, "specialized classes can actually help reduce bugs." The more specialized the class, the more likely it is that you can reuse the class in other applications. If you put all the Halter-related code directly into the Horse class, you'll end up duplicating code in the Cow class, UnpaidIntern class, and any other class that might need Halter behavior. By keeping the Halter code in a separate, specialized Halter class, you have the chance to reuse the Halter class in multiple applications.

FROM THE CLASSROOM

Object-Oriented Design

IS-A and HAS-A relationships and encapsulation are just the tip of the iceberg when it comes to object-oriented design. Many books and graduate theses have been dedicated to this topic. The reason for the emphasis on proper design is simple: money. The cost to deliver a software application has been estimated to be as much as ten times more expensive for poorly designed programs. Having seen the ramifications of poor designs, I can assure you that this estimate is not far-fetched.

Even the best object-oriented designers make mistakes. It is difficult to visualize the relationships between hundreds, or even thousands, of classes. When mistakes are discovered during the implementation (code writing) phase of a project, the amount of code that has to be rewritten can sometimes cause programming teams to start over from scratch.

The software industry has evolved to aid the designer. Visual object modeling languages, like the Unified Modeling Language (UML), allow designers to design and easily modify classes without having to write code first,

because object-oriented components are represented graphically. This allows the designer to create a map of the class relationships and helps them recognize errors before coding begins. Another innovation in object-oriented design is design patterns. Designers noticed that many object-oriented designs apply consistently from project to project, and that it was useful to apply the same designs because it reduced the potential to introduce new design errors. Object-oriented designers then started to share these designs with each other. Now, there are many catalogs of these design patterns both on the Internet and in book form.

Although passing the Java certification exam does not require you to understand object-oriented design thoroughly, hopefully this background information will help you better appreciate why the test writers chose to include encapsulation, and IS-A, and HAS-A relationships on the exam.

—Jonathan Meeks, Sun Certified Java Programmer

Users of the `Horse` class (that is, code that calls methods on a `Horse` instance), think that the `Horse` class has `Halter` behavior. The `Horse` class might have a `tie(LeadRope rope)` method, for example. Users of the `Horse` class should never have to know that when they invoke the `tie()` method, the `Horse` object turns around and delegates the call to its `Halter` class by invoking `myHalter.tie(rope)`. The scenario just described might look like this:

```
public class Horse extends Animal {
    private Halter myHalter = new Halter();
    public void tie(LeadRope rope) {
        myHalter.tie(rope); // Delegate tie behavior to the
                           // Halter object
    }
}

public class Halter {
    public void tie(LeadRope aRope) {
        // Do the actual tie work here
    }
}
```

In OO, we don't want callers to worry about which class or which object is actually doing the real work. To make that happen, the `Horse` class hides implementation details from `Horse` users. `Horse` users ask the `Horse` object to do things (in this case, tie itself up), and the `Horse` will either do it or, as in this example, ask something else to do it. To the caller, though, it always appears that the `Horse` object takes care of itself. Users of a `Horse` should not even need to know that there is such a thing as a `Halter` class.

CERTIFICATION OBJECTIVE

Polymorphism (Exam Objective 5.2)

5.2 Given a scenario, develop code that demonstrates the use of polymorphism. Further, determine when casting will be necessary and recognize compiler vs. runtime errors related to object reference casting.

Remember, any Java object that can pass more than one IS-A test can be considered polymorphic. Other than objects of type `Object`, *all* Java objects are polymorphic in that they pass the IS-A test for their own type and for class `Object`.

Remember that the only way to access an object is through a reference variable, and there are a few key things to remember about references:

- A reference variable can be of only one type, and once declared, that type can never be changed (although the object it references can change).
- A reference is a variable, so it can be reassigned to other objects, (unless the reference is declared `final`).
- A reference variable's type determines the methods that can be invoked on the object the variable is referencing.
- A reference variable can refer to any object of the same type as the declared reference, or—this is the big one—it **can refer to any subtype of the declared type!**
- A reference variable can be declared as a class type or an interface type. If the variable is declared as an interface type, it can reference any object of any class that *implements* the interface.

Earlier we created a `GameShape` class that was extended by two other classes, `PlayerPiece` and `TilePiece`. Now imagine you want to animate some of the shapes on the game board. But not *all* shapes can be animatable, so what do you do with class inheritance?

Could we create a class with an `animate()` method, and have only *some* of the `GameShape` subclasses inherit from that class? If we can, then we could have `PlayerPiece`, for example, extend *both* the `GameShape` class and `Animatable` class, while the `TilePiece` would extend only `GameShape`. But no, this won't work! Java supports only single inheritance! That means a class can have only one immediate superclass. In other words, if `PlayerPiece` is a class, there is no way to say something like this:

```
class PlayerPiece extends GameShape, Animatable { // NO!
    // more code
}
```

A class cannot *extend* more than one class. That means one parent per class. A class *can* have multiple ancestors, however, since class B could extend class A, and class C could extend class B, and so on. So any given class might have multiple classes up its inheritance tree, but that's not the same as saying a class directly extends two classes.



Some languages (like C++) allow a class to extend more than one other class. This capability is known as "multiple inheritance." The reason that Java's creators chose not to allow multiple inheritance is that it can become quite messy. In a nutshell, the problem is that if a class extended two other classes, and both superclasses had, say, a `doStuff()` method, which version of `doStuff()` would the subclass inherit? This issue can lead to a scenario known as the "Deadly Diamond of Death," because of the shape of the class diagram that can be created in a multiple inheritance design. The diamond is formed when classes B and C both extend A, and both B and C inherit a method from A. If class D extends both B and C, and both B and C have overridden the method in A, class D has, in theory, inherited two different implementations of the same method. Drawn as a class diagram, the shape of the four classes looks like a diamond.

So if that doesn't work, what else could you do? You could simply put the `animate()` code in `GameShape`, and then disable the method in classes that can't be animated. But that's a bad design choice for many reasons, including it's more error-prone, it makes the `GameShape` class less cohesive (more on cohesion in a minute), and it means the `GameShape` API "advertises" that all shapes can be animated, when in fact that's not true since only some of the `GameShape` subclasses will be able to successfully run the `animate()` method.

So what *else* could you do? You already know the answer—create an `Animatable` interface, and have only the `GameShape` subclasses that can be animated implement that interface. Here's the interface:

```
public interface Animatable {
    public void animate();
}
```

And here's the modified `PlayerPiece` class that implements the interface:

```

class PlayerPiece extends GameShape implements Animatable {
    public void movePiece() {
        System.out.println("moving game piece");
    }
    public void animate() {
        System.out.println("animating...");
    }
    // more code
}

```

So now we have a `PlayerPiece` that passes the IS-A test for both the `GameShape` class and the `Animatable` interface. That means a `PlayerPiece` can be treated polymorphically as one of four things at any given time, depending on the declared type of the reference variable:

- An `Object` (since any object inherits from `Object`)
- A `GameShape` (since `PlayerPiece` extends `GameShape`)
- A `PlayerPiece` (since that's what it really is)
- An `Animatable` (since `PlayerPiece` implements `Animatable`)

The following are all legal declarations. Look closely:

```

PlayerPiece player = new PlayerPiece();
Object o = player;
GameShape shape = player;
Animatable mover = player;

```

There's only one object here—an instance of type `PlayerPiece`—but there are four different types of reference variables, all referring to that one object on the heap. Pop quiz: which of the preceding reference variables can invoke the `displayShape()` method? Hint: only two of the four declarations can be used to invoke the `displayShape()` method.

Remember that method invocations allowed by the compiler are based solely on the declared type of the reference, regardless of the object type. So looking at the four reference types again—`Object`, `GameShape`, `PlayerPiece`, and `Animatable`—which of these four types know about the `displayShape()` method?

You guessed it—both the `GameShape` class and the `PlayerPiece` class are known (by the compiler) to have a `displayShape()` method, so either of those reference types

can be used to invoke `displayShape()`. Remember that to the compiler, a `PlayerPiece` IS-A `GameShape`, so the compiler says, "I see that the declared type is `PlayerPiece`, and since `PlayerPiece` extends `GameShape`, that means `PlayerPiece` inherited the `displayShape()` method. Therefore, `PlayerPiece` can be used to invoke the `displayShape()` method."

Which methods can be invoked when the `PlayerPiece` object is being referred to using a reference declared as type `Animatable`? Only the `animate()` method. Of course the cool thing here is that any class from any inheritance tree can also implement `Animatable`, so that means if you have a method with an argument declared as type `Animatable`, you can pass in `PlayerPiece` objects, `SpinningLogo` objects, and anything else that's an instance of a class that implements `Animatable`. And you can use that parameter (of type `Animatable`) to invoke the `animate()` method, but not the `displayShape()` method (which it might not even have), or anything other than what is known to the compiler based on the reference type. The compiler always knows, though, that you can invoke the methods of class `Object` on any object, so those are safe to call regardless of the reference—class or interface—used to refer to the object.

We've left out one big part of all this, which is that even though the compiler only knows about the declared reference type, the Java Virtual Machine (JVM) at runtime knows what the object really is. And that means that even if the `PlayerPiece` object's `displayShape()` method is called using a `GameShape` reference variable, if the `PlayerPiece` overrides the `displayShape()` method, the JVM will invoke the `PlayerPiece` version! The JVM looks at the real object at the other end of the reference, "sees" that it has overridden the method of the declared reference variable type, and invokes the method of the object's actual class. But one other thing to keep in mind:

Polymorphic method invocations apply only to *instance methods*. You can always refer to an object with a more general reference variable type (a superclass or interface), but at runtime, the **ONLY** things that are dynamically selected based on the actual *object* (rather than the *reference type*) are instance methods. Not *static* methods. Not *variables*. Only overridden instance methods are dynamically invoked based on the real object's type.

Since this definition depends on a clear understanding of overriding, and the distinction between static methods and instance methods, we'll cover those next.

CERTIFICATION OBJECTIVE**Overriding / Overloading
(Exam Objectives 1.5 and 5.4)**

1.5 Given a code example, determine if a method is correctly overriding or overloading another method, and identify legal return values (including covariant returns), for the method.

5.4 Given a scenario, develop code that declares and/or invokes overridden or overloaded methods and code that declares and/or invokes superclass, overridden, or overloaded constructors.

Overridden Methods

Any time you have a class that inherits a method from a superclass, you have the opportunity to override the method (unless, as you learned earlier, the method is marked `final`). The key benefit of overriding is the ability to define behavior that's specific to a particular subclass type. The following example demonstrates a `Horse` subclass of `Animal` overriding the `Animal` version of the `eat()` method:

```
public class Animal {
    public void eat() {
        System.out.println("Generic Animal Eating Generically");
    }
}
class Horse extends Animal {
    public void eat() {
        System.out.println("Horse eating hay, oats, "
                           + "and horse treats");
    }
}
```

For abstract methods you inherit from a superclass, you have no choice. You *must* implement the method in the subclass **unless the subclass is also abstract**. Abstract methods must be *implemented* by the concrete subclass, but this is a lot like saying that the concrete subclass *overrides* the abstract methods of the superclass. So you could think of abstract methods as methods you're forced to override.

The `Animal` class creator might have decided that for the purposes of polymorphism, all `Animal` subtypes should have an `eat()` method defined in a unique, specific way. Polymorphically, when someone has an `Animal` reference that refers not to an `Animal` instance, but to an `Animal` subclass instance, the caller should be able to invoke `eat()` on the `Animal` reference, but the actual runtime object (say, a `Horse` instance) will run its own specific `eat()` method. Marking the `eat()` method abstract is the `Animal` programmer's way of saying to all subclass developers, "It doesn't make any sense for your new subtype to use a generic `eat()` method, so you have to come up with your *own* `eat()` method implementation!" A (non-abstract), example of using polymorphism looks like this:

```
public class TestAnimals {
    public static void main (String [] args) {
        Animal a = new Animal();
        Animal b = new Horse(); //Animal ref, but a Horse object
        a.eat(); // Runs the Animal version of eat()
        b.eat(); // Runs the Horse version of eat()
    }
}

class Animal {
    public void eat() {
        System.out.println("Generic Animal Eating Generically");
    }
}

class Horse extends Animal {
    public void eat() {
        System.out.println("Horse eating hay, oats, "
                           + "and horse treats");
    }
    public void buck() { }
}
```

In the preceding code, the test class uses an `Animal` reference to invoke a method on a `Horse` object. Remember, the compiler will allow only methods in class `Animal` to be invoked when using a reference to an `Animal`. The following would not be legal given the preceding code:

```
Animal c = new Horse();
c.buck(); // Can't invoke buck();
          // Animal class doesn't have that method
```


To reiterate, the compiler looks only at the reference type, not the instance type. Polymorphism lets you use a more abstract supertype (including an interface) reference to refer to one of its subtypes (including interface implementers).

The overriding method cannot have a more restrictive access modifier than the method being overridden (for example, you can't override a method marked `public` and make it `protected`). Think about it: if the `Animal` class advertises a `public eat()` method and someone has an `Animal` reference (in other words, a reference declared as type `Animal`), that someone will assume it's safe to call `eat()` on the `Animal` reference regardless of the actual instance that the `Animal` reference is referring to. If a subclass were allowed to sneak in and change the access modifier on the overriding method, then suddenly at runtime—when the JVM invokes the true object's (`Horse`) version of the method rather than the reference type's (`Animal`) version—the program would die a horrible death. (Not to mention the emotional distress for the one who was betrayed by the rogue subclass.) Let's modify the polymorphic example we saw earlier in this section:

```
public class TestAnimals {
    public static void main (String [] args) {
        Animal a = new Animal();
        Animal b = new Horse(); //Animal ref, but a Horse object
        a.eat(); // Runs the Animal version of eat()
        b.eat(); // Runs the Horse version of eat()
    }
}

class Animal {
    public void eat() {
        System.out.println("Generic Animal Eating Generically");
    }
}

class Horse extends Animal {
    private void eat() { // whoa! - it's private!
        System.out.println("Horse eating hay, oats, "
            + "and horse treats");
    }
}
```

If this code compiled (which it doesn't), the following would fail at runtime:

```
Animal b = new Horse(); // Animal ref, but a Horse
                        // object , so far so good
b.eat();                // Meltdown at runtime!
```

The variable `b` is of type `Animal`, which has a `public eat()` method. But remember that at runtime, Java uses virtual method invocation to dynamically select the actual version of the method that will run, based on the actual instance. An `Animal` reference can always refer to a `Horse` instance, because `Horse IS-A(n) Animal`. What makes that superclass reference to a subclass instance possible is that the subclass is guaranteed to be able to do everything the superclass can do. Whether the `Horse` instance overrides the inherited methods of `Animal` or simply inherits them, anyone with an `Animal` reference to a `Horse` instance is free to call all accessible `Animal` methods. For that reason, an overriding method must fulfill the contract of the superclass.

The rules for overriding a method are as follows:

- The argument list must exactly match that of the overridden method. If they don't match, you can end up with an overloaded method you didn't intend.
- The return type must be the same as, or a subtype of, the return type declared in the original overridden method in the superclass. (More on this in a few pages when we discuss covariant returns.)
- The access level can't be more restrictive than the overridden method's.
- The access level CAN be less restrictive than that of the overridden method.
- Instance methods can be overridden only if they are inherited by the subclass. A subclass within the same package as the instance's superclass can override any superclass method that is not marked `private` or `final`. A subclass in a different package can override only those non-`final` methods marked `public` or `protected` (since `protected` methods are inherited by the subclass).
- The overriding method CAN throw any unchecked (runtime) exception, regardless of whether the overridden method declares the exception. (More in Chapter 5.)
- The overriding method must NOT throw checked exceptions that are new or broader than those declared by the overridden method. For example, a method that declares a `FileNotFoundException` cannot be overridden by a method that declares a `SQLException`, `Exception`, or any other non-runtime exception unless it's a subclass of `FileNotFoundException`.
- The overriding method can throw narrower or fewer exceptions. Just because an overridden method "takes risks" doesn't mean that the overriding subclass' exception takes the same risks. Bottom line: an overriding method doesn't

have to declare any exceptions that it will never throw, regardless of what the overridden method declares.

- You cannot override a method marked `final`.
- You cannot override a method marked `static`. We'll look at an example in a few pages when we discuss `static` methods in more detail.
- If a method can't be inherited, you cannot override it. Remember that overriding implies that you're reimplementing a method you inherited! For example, the following code is not legal, and even if you added an `eat()` method to `Horse`, it wouldn't be an override of `Animal`'s `eat()` method.

```
public class TestAnimals {
    public static void main (String [] args) {
        Horse h = new Horse();
        h.eat(); // Not legal because Horse didn't inherit eat()
    }
}
class Animal {
    private void eat() {
        System.out.println("Generic Animal Eating Generically");
    }
}
class Horse extends Animal { }
```

Invoking a Superclass Version of an Overridden Method

Often, you'll want to take advantage of some of the code in the superclass version of a method, yet still override it to provide some additional specific behavior. It's like saying, "Run the superclass version of the method, then come back down here and finish with my subclass additional method code." (Note that there's no requirement that the superclass version run before the subclass code.) It's easy to do in code using the keyword `super` as follows:

```
public class Animal {
    public void eat() { }
    public void printYourself() {
        // Useful printing code goes here
    }
}
class Horse extends Animal {
    public void printYourself() {
        // Take advantage of Animal code, then add some more
    }
}
```

```

        super.printYourself(); // Invoke the superclass
                               // (Animal) code
                               // Then do Horse-specific
                               // print work here
    }
}

```

Note: Using `super` to invoke an overridden method only applies to instance methods. (Remember, static methods can't be overridden.)

exam

Watch

If a method is overridden but you use a polymorphic (supertype) reference to refer to the subtype object with the overriding method, the compiler assumes you're calling the supertype version of the method. If the supertype version declares a checked exception, but the overriding subtype method does not, the compiler still thinks you are calling a method that declares an exception (more in Chapter 5). Let's take a look at an example:

```

class Animal {
    public void eat() throws Exception {
        // throws an Exception
    }
}

class Dog2 extends Animal {
    public void eat() { /* no Exceptions */}
    public static void main(String [] args) {
        Animal a = new Dog2();
        Dog2 d = new Dog2();
        d.eat();           // ok
        a.eat();           // compiler error -
                           // unreported exception
    }
}

```

This code will not compile because of the Exception declared on the `Animal eat()` method. This happens even though, at runtime, the `eat()` method used would be the `Dog` version, which does not declare the exception.

Examples of Legal and Illegal Method Overrides

Let's take a look at overriding the `eat()` method of `Animal`:

```
public class Animal {
    public void eat() { }
}
```

Table 2-1 lists examples of illegal overrides of the `Animal eat()` method, given the preceding version of the `Animal` class.

TABLE 2-1 Examples of Illegal Overrides

Illegal Override Code	Problem with the Code
<code>private void eat() { }</code>	Access modifier is more restrictive
<code>public void eat() throws IOException { }</code>	Declares a checked exception not defined by superclass version
<code>public void eat(String food) { }</code>	A legal overload, not an override, because the argument list changed
<code>public String eat() { }</code>	Not an override because of the return type, not an overload either because there's no change in the argument list

Overloaded Methods

You're wondering what overloaded methods are doing in an OO chapter, but we've included them here since one of the things newer Java developers are most confused about are all of the subtle differences between *overloaded* and *overridden* methods.

Overloaded methods let you reuse the same method name in a class, but with different arguments (and optionally, a different return type). Overloading a method often means you're being a little nicer to those who call your methods, because your code takes on the burden of coping with different argument types rather than forcing the caller to do conversions prior to invoking your method. The rules are simple:

- Overloaded methods **MUST** change the argument list.
- Overloaded methods **CAN** change the return type.
- Overloaded methods **CAN** change the access modifier.
- Overloaded methods **CAN** declare new or broader checked exceptions.

- A method can be overloaded in the *same* class or in a *subclass*. In other words, if class A defines a `doStuff(int i)` method, the subclass B could define a `doStuff(String s)` method without overriding the superclass version that takes an `int`. So two methods with the same name but in different classes can still be considered overloaded, if the subclass inherits one version of the method and then declares another overloaded version in its class definition.

exam

Watch

Be careful to recognize when a method is overloaded rather than overridden. You might see a method that appears to be violating a rule for overriding, but that is actually a legal overload, as follows:

```
public class Foo {
    public void doStuff(int y, String s) { }
    public void moreThings(int x) { }
}
class Bar extends Foo {
    public void doStuff(int y, long s) throws IOException { }
}
```

It's tempting to see the `IOException` as the problem, because the overridden `doStuff()` method doesn't declare an exception, and `IOException` is checked by the compiler. But the `doStuff()` method is not overridden! Subclass `Bar` overloads the `doStuff()` method, by varying the argument list, so the `IOException` is fine.

Legal Overloads

Let's look at a method we want to overload:

```
public void changeSize(int size, String name, float pattern) { }
```

The following methods are legal overloads of the `changeSize()` method:

```
public void changeSize(int size, String name) { }
public int changeSize(int size, float pattern) { }
public void changeSize(float pattern, String name)
    throws IOException { }
```

Invoking Overloaded Methods

Note that there's a lot more to this discussion on how the compiler knows which method to invoke, but the rest is covered in Chapter 3 when we look at boxing and var-args—both of which have a huge impact on overloading. (You still have to pay attention to the part covered here, though.)

When a method is invoked, more than one method of the same name might exist for the object type you're invoking a method on. For example, the `Horse` class might have three methods with the same name but with different argument lists, which means the method is overloaded.

Deciding which of the matching methods to invoke is based on the arguments. If you invoke the method with a `String` argument, the overloaded version that takes a `String` is called. If you invoke a method of the same name but pass it a `float`, the overloaded version that takes a `float` will run. If you invoke the method of the same name but pass it a `Foo` object, and there isn't an overloaded version that takes a `Foo`, then the compiler will complain that it can't find a match. The following are examples of invoking overloaded methods:

```
class Adder {
    public int addThem(int x, int y) {
        return x + y;
    }

    // Overload the addThem method to add doubles instead of ints
    public double addThem(double x, double y) {
        return x + y;
    }
}

// From another class, invoke the addThem() method
public class TestAdder {
    public static void main (String [] args) {
        Adder a = new Adder();
        int b = 27;
        int c = 3;
        int result = a.addThem(b,c); // Which addThem is invoked?
        double doubleResult = a.addThem(22.5,9.3); // Which addThem?
    }
}
```

In the preceding `TestAdder` code, the first call to `a.addThem(b,c)` passes two `ints` to the method, so the first version of `addThem()`—the overloaded version

that takes two `int` arguments—is called. The second call to `a.addThem(22.5, 9.3)` passes two `doubles` to the method, so the second version of `addThem()`—the overloaded version that takes two `double` arguments—is called.

Invoking overloaded methods that take object references rather than primitives is a little more interesting. Say you have an overloaded method such that one version takes an `Animal` and one takes a `Horse` (subclass of `Animal`). If you pass a `Horse` object in the method invocation, you'll invoke the overloaded version that takes a `Horse`. Or so it looks at first glance:

```
class Animal { }
class Horse extends Animal { }
class UseAnimals {
    public void doStuff(Animal a) {
        System.out.println("In the Animal version");
    }
    public void doStuff(Horse h) {
        System.out.println("In the Horse version");
    }
    public static void main (String [] args) {
        UseAnimals ua = new UseAnimals();
        Animal animalObj = new Animal();
        Horse horseObj = new Horse();
        ua.doStuff(animalObj);
        ua.doStuff(horseObj);
    }
}
```

The output is what you expect:

```
in the Animal version
in the Horse version
```

But what if you use an `Animal` reference to a `Horse` object?

```
Animal animalRefToHorse = new Horse();
    ua.doStuff(animalRefToHorse);
```

Which of the overloaded versions is invoked? You might want to say, "The one that takes a `Horse`, since it's a `Horse` object at runtime that's being passed to the method." But that's not how it works. The preceding code would actually print:

```
in the Animal version
```


Even though the actual object at runtime is a Horse and not an Animal, the choice of which overloaded method to call (in other words, the signature of the method) is NOT dynamically decided at runtime. Just remember, the *reference* type (not the object type) determines which overloaded method is invoked! To summarize, which *overridden* version of the method to call (in other words, from which class in the inheritance tree) is decided at *runtime* based on *object* type, but which *overloaded* version of the method to call is based on the *reference* type of the argument passed at *compile* time. If you invoke a method passing it an Animal reference to a Horse object, the compiler knows only about the Animal, so it chooses the overloaded version of the method that takes an Animal. It does not matter that at runtime there's actually a Horse being passed.

Polymorphism in Overloaded and Overridden Methods

How does polymorphism work with overloaded methods? From what we just looked at, it doesn't appear that polymorphism matters when a method is overloaded. If you pass an Animal reference, the overloaded method that takes an Animal will be invoked, even if the actual object passed is a Horse. Once the Horse masquerading as Animal gets in to the method, however, the Horse object is still a Horse despite being passed into a method expecting an Animal. So it's true that polymorphism doesn't determine which overloaded version is called; polymorphism does come into play when the decision is about which overridden version of a method is called. But sometimes, a method is both overloaded and overridden. Imagine the Animal and Horse classes look like this:

```
public class Animal {
    public void eat() {
        System.out.println("Generic Animal Eating Generically");
    }
}
public class Horse extends Animal {
    public void eat() {
        System.out.println("Horse eating hay ");
    }
    public void eat(String s) {
        System.out.println("Horse eating " + s);
    }
}
```

Notice that the Horse class has both overloaded and overridden the eat() method. Table 2-2 shows which version of the three eat() methods will run depending on how they are invoked.

TABLE 2-2 Examples of Illegal Overrides

Method Invocation Code	Result
<code>Animal a = new Animal(); a.eat();</code>	Generic Animal Eating Generically
<code>Horse h = new Horse(); h.eat();</code>	Horse eating hay
<code>Animal ah = new Horse(); ah.eat();</code>	Horse eating hay Polymorphism works—the actual object type (Horse), not the reference type (Animal), is used to determine which eat() is called.
<code>Horse he = new Horse(); he.eat("Apples");</code>	Horse eating Apples The overloaded eat(String s) method is invoked.
<code>Animal a2 = new Animal(); a2.eat("treats");</code>	Compiler error! Compiler sees that Animal class doesn't have an eat() method that takes a String.
<code>Animal ah2 = new Horse(); ah2.eat("Carrots");</code>	Compiler error! Compiler <i>still</i> looks only at the reference, and sees that Animal doesn't have an eat() method that takes a String. Compiler doesn't care that the actual object might be a Horse at runtime.

exam

Watch

Don't be fooled by a method that's overloaded but not overridden by a subclass. It's perfectly legal to do the following:

```
public class Foo {
    void doStuff() { }
}
class Bar extends Foo {
    void doStuff(String s) { }
}
```

The Bar class has two doStuff() methods: the no-arg version it inherits from Foo (and does not override), and the overloaded doStuff(String s) defined in the Bar class. Code with a reference to a Foo can invoke only the no-arg version, but code with a reference to a Bar can invoke either of the overloaded versions.

Table 2-3 summarizes the difference between overloaded and overridden methods.

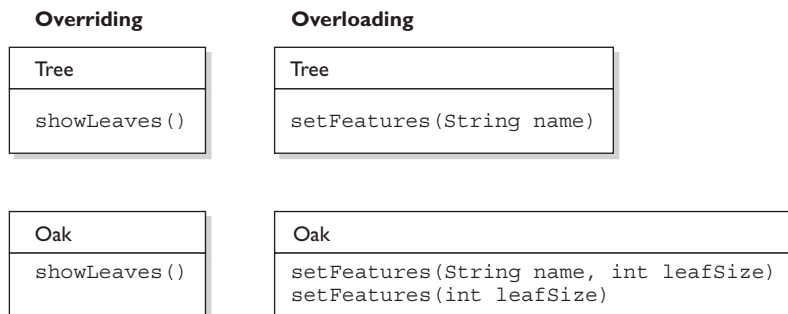
TABLE 2-3 Differences Between Overloaded and Overridden Methods

	Overloaded Method	Overridden Method
Argument(s)	Must change.	Must not change.
Return type	Can change.	Can't change except for covariant returns.
Exceptions	Can change.	Can reduce or eliminate. Must not throw new or broader checked exceptions.
Access	Can change.	Must not make more restrictive (can be less restrictive).
Invocation	<i>Reference</i> type determines which overloaded version (based on declared argument types) is selected. Happens at <i>compile</i> time. The actual <i>method</i> that's invoked is still a virtual method invocation that happens at runtime, but the compiler will already know the <i>signature</i> of the method to be invoked. So at runtime, the argument match will already have been nailed down, just not the <i>class</i> in which the method lives.	<i>Object</i> type (in other words, <i>the type of the actual instance on the heap</i>) determines which method is selected. Happens at <i>runtime</i> .

The current objective (5.4) covers both method and constructor overloading, but we'll cover constructor overloading in the next section, where we'll also cover the other constructor-related topics that are on the exam. Figure 2-4 illustrates the way overloaded and overridden methods appear in class relationships.

FIGURE 2-4

Overloaded and overridden methods in class relationships



CERTIFICATION OBJECTIVE

Reference Variable Casting (Objective 5.2)

5.2 Given a scenario, develop code that demonstrates the use of polymorphism. Further, determine when casting will be necessary and recognize compiler vs. runtime errors related to object reference casting.

We've seen how it's both possible and common to use generic reference variable types to refer to more specific object types. It's at the heart of polymorphism. For example, this line of code should be second nature by now:

```
Animal animal = new Dog();
```

But what happens when you want to use that `animal` reference variable to invoke a method that only class `Dog` has? You know it's referring to a `Dog`, and you want to do a `Dog`-specific thing? In the following code, we've got an array of `Animals`, and whenever we find a `Dog` in the array, we want to do a special `Dog` thing. Let's agree for now that all of this code is OK, except that we're not sure about the line of code that invokes the `playDead` method.

```
class Animal {
    void makeNoise() {System.out.println("generic noise"); }
}
class Dog extends Animal {
    void makeNoise() {System.out.println("bark"); }
    void playDead() { System.out.println("roll over"); }
}

class CastTest2 {
    public static void main(String [] args) {
        Animal [] a = {new Animal(), new Dog(), new Animal() };
        for(Animal animal : a) {
            animal.makeNoise();
            if(animal instanceof Dog) {
                animal.playDead();           // try to do a Dog behavior ?
            }
        }
    }
}
```

When we try to compile this code, the compiler says something like this:

```
cannot find symbol
```

The compiler is saying, "Hey, class `Animal` doesn't have a `playDead()` method". Let's modify the `if` code block:

```
if (animal instanceof Dog) {
    Dog d = (Dog) animal;    // casting the ref. var.
    d.playDead();
}
```

The new and improved code block contains a cast, which in this case is sometimes called a *downcast*, because we're casting down the inheritance tree to a more specific class. Now, the compiler is happy. Before we try to invoke `playDead`, we cast the `animal` variable to type `Dog`. What we're saying to the compiler is, "We know it's really referring to a `Dog` object, so it's okay to make a new `Dog` reference variable to refer to that object." In this case we're safe because before we ever try the cast, we do an `instanceof` test to make sure.

It's important to know that the compiler is forced to trust us when we do a downcast, even when we screw up:

```
class Animal { }
class Dog extends Animal { }
class DogTest {
    public static void main(String [] args) {
        Animal animal = new Animal();
        Dog d = (Dog) animal;    // compiles but fails later
    }
}
```

It can be maddening! This code compiles! When we try to run it, we'll get an exception something like this:

```
java.lang.ClassCastException
```

Why can't we trust the compiler to help us out here? Can't it see that `animal` is of type `Animal`? All the compiler can do is verify that the two types are in the same inheritance tree, so that depending on whatever code might have come before the downcast, it's possible that `animal` is of type `Dog`. The compiler must allow

things that might possibly work at runtime. However, if the compiler knows with certainty that the cast could not possibly work, compilation will fail. The following replacement code block will NOT compile:

```
Animal animal = new Animal();
Dog d = (Dog) animal;
String s = (String) animal; // animal can't EVER be a String
```

In this case, you'll get an error something like this:

```
inconvertible types
```

Unlike downcasting, upcasting (casting *up* the inheritance tree to a more general type) works implicitly (i.e., you don't have to type in the cast) because when you upcast you're implicitly restricting the number of methods you can invoke, as opposed to *downcasting*, which implies that later on, you might want to invoke a more *specific* method. For instance:

```
class Animal { }
class Dog extends Animal { }

class DogTest {
    public static void main(String [] args) {
        Dog d = new Dog();
        Animal a1 = d; // upcast ok with no explicit cast
        Animal a2 = (Animal) d; // upcast ok with an explicit cast
    }
}
```

Both of the previous upcasts will compile and run without exception, because a Dog IS-A Animal, which means that anything an Animal can do, a Dog can do. A Dog can do more, of course, but the point is—anyone with an Animal reference can safely call Animal methods on a Dog instance. The Animal methods may have been overridden in the Dog class, but all we care about now is that a Dog can always do at least everything an Animal can do. The compiler and JVM know it too, so the implicit upcast is always legal for assigning an object of a subtype to a reference of one of its supertype classes (or interfaces). If Dog implements Pet, and Pet defines `beFriendly()`, then a Dog can be implicitly cast to a Pet, but the only Dog method you can invoke then is `beFriendly()`, which Dog was forced to implement because Dog implements the Pet interface.

One more thing...if `Dog` implements `Pet`, then if `Beagle` extends `Dog`, but `Beagle` does not *declare* that it implements `Pet`, `Beagle` is still a `Pet`! `Beagle` is a `Pet` simply because it extends `Dog`, and `Dog`'s already taken care of the `Pet` parts of itself, and all its children. The `Beagle` class can always override any methods it inherits from `Dog`, including methods that `Dog` implemented to fulfill its interface contract.

And just one more thing...if `Beagle` does declare it implements `Pet`, just so that others looking at the `Beagle` class API can easily see that `Beagle` IS-A `Pet`, without having to look at `Beagle`'s superclasses, `Beagle` still doesn't need to implement the `beFriendly()` method if the `Dog` class (`Beagle`'s superclass) has already taken care of that. In other words, if `Beagle` IS-A `Dog`, and `Dog` IS-A `Pet`, then `Beagle` IS-A `Pet`, and has already met its `Pet` obligations for implementing the `beFriendly()` method since it inherits the `beFriendly()` method. The compiler is smart enough to say, "I know `Beagle` already IS a `Dog`, but it's OK to make it more obvious."

So don't be fooled by code that shows a concrete class that declares that it implements an interface, but doesn't implement the *methods* of the interface. Before you can tell whether the code is legal, you must know what the superclasses of this implementing class have declared. If any superclass in its inheritance tree has already provided concrete (i.e., non-abstract) method implementations, then, regardless of whether the superclass declares that it implements the interface, the subclass is under no obligation to re-implement (override) those methods.

exam

Watch

The exam creators will tell you that they're forced to jam tons of code into little spaces "because of the exam engine." While that's partially true, they ALSO like to obfuscate. The following code:

```
Animal a = new Dog();
Dog d = (Dog) a;
d.doDogStuff();
```

Can be replaced with this easy-to-read bit of fun:

```
Animal a = new Dog();
((Dog) a).doDogStuff();
```

In this case the compiler needs all of those parentheses, otherwise it thinks it's been handed an incomplete statement.

CERTIFICATION OBJECTIVE

Implementing an Interface (Exam Objective 1.2)

1.2 Develop code that declares an interface...

When you implement an interface, you're agreeing to adhere to the contract defined in the interface. That means you're agreeing to provide legal implementations for every method defined in the interface, and that anyone who knows what the interface methods look like (not how they're implemented, but how they can be called and what they return) can rest assured that they can invoke those methods on an instance of your implementing class.

For example, if you create a class that implements the `Runnable` interface (so that your code can be executed by a specific thread), you must provide the `public void run()` method. Otherwise, the poor thread could be told to go execute your `Runnable` object's code and—surprise surprise—the thread then discovers the object has no `run()` method! (At which point, the thread would blow up and the JVM would crash in a spectacular yet horrible explosion.) Thankfully, Java prevents this meltdown from occurring by running a compiler check on any class that claims to implement an interface. If the class says it's implementing an interface, it darn well better have an implementation for each method in the interface (with a few exceptions we'll look at in a moment).

Assuming an interface, `Bounceable`, with two methods: `bounce()`, and `setBounceFactor()`, the following class will compile:

```
public class Ball implements Bounceable { // Keyword
                                           // 'implements'
    public void bounce() { }
    public void setBounceFactor(int bf) { }
}
```

OK, we know what you're thinking: "This has got to be the worst implementation class in the history of implementation classes." It compiles, though. And runs. The interface contract guarantees that a class will have the method (in other words, others can call the method subject to access control), but it never guaranteed a good implementation—or even any actual implementation code in the body of the method. The compiler will never say to you, "Um, excuse me, but did you really

mean to put nothing between those curly braces? HELLO. This is a method after all, so shouldn't it do something?"

Implementation classes must adhere to the same rules for method implementation as a class extending an abstract class. In order to be a legal implementation class, a nonabstract implementation class must do the following:

- Provide concrete (nonabstract) implementations for all methods from the declared interface.
- Follow all the rules for legal overrides.
- Declare no checked exceptions on implementation methods other than those declared by the interface method, or subclasses of those declared by the interface method.
- Maintain the signature of the interface method, and maintain the same return type (or a subtype). (But it does not have to declare the exceptions declared in the interface method declaration.)

But wait, there's more! An implementation class can itself be abstract! For example, the following is legal for a class `Ball` implementing `Bounceable`:

```
abstract class Ball implements Bounceable { }
```

Notice anything missing? We never provided the implementation methods. And that's OK. If the implementation class is abstract, it can simply pass the buck to its first concrete subclass. For example, if class `BeachBall` extends `Ball`, and `BeachBall` is not abstract, then `BeachBall` will have to provide all the methods from `Bounceable`:

```
class BeachBall extends Ball {
    // Even though we don't say it in the class declaration above,
    // BeachBall implements Bounceable, since BeachBall's abstract
    // superclass (Ball) implements Bounceable

    public void bounce() {
        // interesting BeachBall-specific bounce code
    }
    public void setBounceFactor(int bf) {
        // clever BeachBall-specific code for setting
        // a bounce factor
    }
}
```

```

        // if class Ball defined any abstract methods,
        // they'll have to be
        // implemented here as well.
    }

```

Look for classes that claim to implement an interface but don't provide the correct method implementations. Unless the implementing class is abstract, the implementing class must provide implementations for all methods defined in the interface.

Two more rules you need to know and then we can put this topic to sleep (or put you to sleep; we always get those two confused):

1. A class can implement more than one interface. It's perfectly legal to say, for example, the following:

```

public class Ball implements Bounceable, Serializable, Runnable
{ ... }

```

You can extend only one class, but implement many interfaces. But remember that subclassing defines who and what you are, whereas implementing defines a role you can play or a hat you can wear, despite how different you might be from some other class implementing the same interface (but from a different inheritance tree). For example, a `Person` extends `HumanBeing` (although for some, that's debatable). But a `Person` may also implement `Programmer`, `Snowboarder`, `Employee`, `Parent`, or `PersonCrazyEnoughToTakeThisExam`.

2. An interface can itself extend another interface, but never implement anything. The following code is perfectly legal:

```

public interface Bounceable extends Moveable { }    // ok!

```

What does that mean? The first concrete (nonabstract) implementation class of `Bounceable` must implement all the methods of `Bounceable`, plus all the methods of `Moveable`! The subinterface, as we call it, simply adds more requirements to the contract of the superinterface. You'll see this concept applied in many areas of Java, especially J2EE where you'll often have to build your own interface that extends one of the J2EE interfaces.

Hold on though, because here's where it gets strange. An interface can extend more than one interface! Think about that for a moment. You know that when we're talking about classes, the following is illegal:

```
public class Programmer extends Employee, Geek { } // Illegal!
```

As we mentioned earlier, a class is not allowed to extend multiple classes in Java. An interface, however, is free to extend multiple interfaces.

```
interface Bounceable extends Moveable, Spherical {    // ok!
    void bounce();
    void setBounceFactor(int bf);
}
interface Moveable {
    void moveIt();
}
interface Spherical {
    void doSphericalThing();
}
```

In the next example, Ball is required to implement Bounceable, plus all methods from the interfaces that Bounceable extends (including any interfaces those interfaces extend, and so on until you reach the top of the stack—or is it the bottom of the stack?). So Ball would need to look like the following:

```
class Ball implements Bounceable {

    public void bounce() { }    // Implement Bounceable's methods
    public void setBounceFactor(int bf) { }

    public void moveIt() { }    // Implement Moveable's method

    public void doSphericalThing() { }    // Implement Spherical
}
```

If class Ball fails to implement any of the methods from Bounceable, Moveable, or Spherical, the compiler will jump up and down wildly, red in the face, until it does. Unless, that is, class Ball is marked `abstract`. In that case, Ball could choose to implement any, all, or none of the methods from any of the interfaces, thus leaving the rest of the implementations to a concrete subclass of Ball, as follows:

```

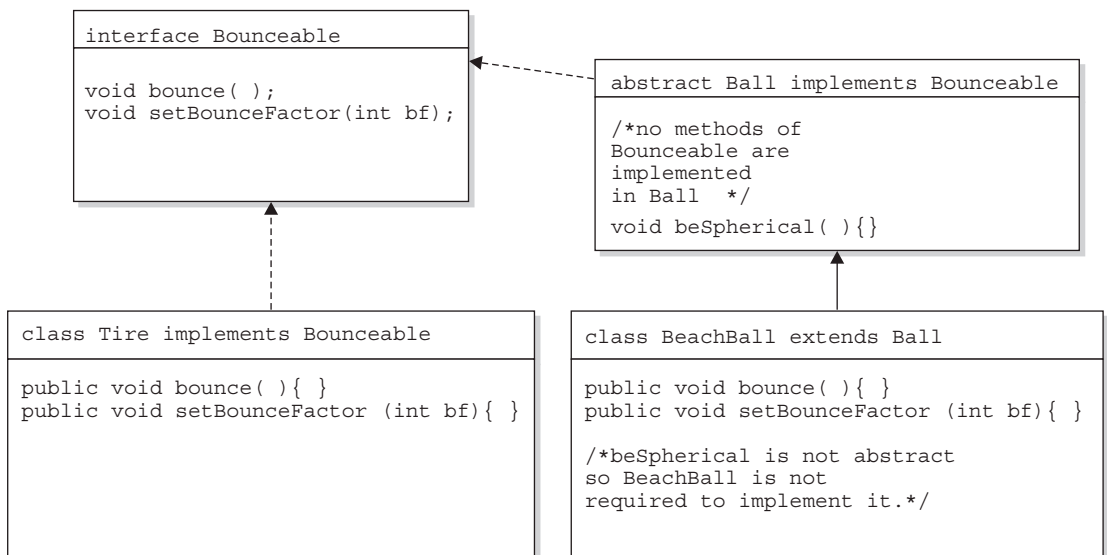
abstract class Ball implements Bounceable {
    public void bounce() { ... } // Define bounce behavior
    public void setBounceFactor(int bf) { ... }
    // Don't implement the rest; leave it for a subclass
}

class SoccerBall extends Ball { // class SoccerBall must
    // implement the interface methods that Ball didn't
    public void moveIt() { ... }
    public void doSphericalThing() { ... }
    // SoccerBall can choose to override the Bounceable methods
    // implemented by Ball
    public void bounce() { ... }
}

```

Figure 2-5 compares concrete and abstract examples of extends and implements, for both classes and interfaces.

FIGURE 2-5 Comparing concrete and abstract examples of extends and implements



Because `BeachBall` is the first concrete class to implement `Bounceable`, it must provide implementations for all methods of `Bounceable`, except those defined in the abstract class `Ball`. Because `Ball` did not provide implementations of `Bounceable` methods, `BeachBall` was required to implement all of them.

exam**Watch**

Look for illegal uses of extends and implements. The following shows examples of legal and illegal class and interface declarations:

```

class Foo { } // OK
class Bar implements Foo { } // No! Can't implement a class
interface Baz { } // OK
interface Fi { } // OK
interface Fee implements Baz { } // No! Interface can't
// implement an interface
interface Zee implements Foo { } // No! Interface can't
// implement a class
interface Zoo extends Foo { } // No! Interface can't
// extend a class
interface Boo extends Fi { } // OK. Interface can extend
// an interface
class Toon extends Foo, Button { } // No! Class can't extend
// multiple classes
class Zoom implements Fi, Baz { } // OK. class can implement
// multiple interfaces
interface Vroom extends Fi, Baz { } // OK. interface can extend
// multiple interfaces
class Yow extends Foo implements Fi { } // OK. Class can do both
// (extends must be 1st)

```

Burn these in, and watch for abuses in the questions you get on the exam. Regardless of what the question appears to be testing, the real problem might be the class or interface declaration. Before you get caught up in, say, tracing a complex threading flow, check to see if the code will even compile. (Just that tip alone may be worth your putting us in your will!) (You'll be impressed by the effort the exam developers put into distracting you from the real problem.) (How did people manage to write anything before parentheses were invented?)

CERTIFICATION OBJECTIVE**Legal Return Types (Exam Objective 1.5)**

1.5 Given a code example, determine if a method is correctly overriding or overloading another method, and identify legal return values (including covariant returns), for the method.

This objective covers two aspects of return types: what you can declare as a return type, and what you can actually return as a value. What you can and cannot declare is pretty straightforward, but it all depends on whether you're overriding an inherited method or simply declaring a new method (which includes overloaded methods). We'll take just a quick look at the difference between return type rules for overloaded and overriding methods, because we've already covered that in this chapter. We'll cover a small bit of new ground, though, when we look at polymorphic return types and the rules for what is and is not legal to actually return.

Return Type Declarations

This section looks at what you're allowed to declare as a return type, which depends primarily on whether you are overriding, overloading, or declaring a new method.

Return Types on Overloaded Methods

Remember that method overloading is not much more than name reuse. The overloaded method is a completely different method from any other method of the same name. So if you inherit a method but overload it in a subclass, you're not subject to the restrictions of overriding, which means you can declare any return type you like. What you can't do is change *only* the return type. To overload a method, remember, you must change the argument list. The following code shows an overloaded method:

```
public class Foo{
    void go() { }
}
public class Bar extends Foo {
    String go(int x) {
```

```

        return null;
    }
}

```

Notice that the Bar version of the method uses a different return type. That's perfectly fine. As long as you've changed the argument list, you're overloading the method, so the return type doesn't have to match that of the superclass version. What you're NOT allowed to do is this:

```

public class Foo{
    void go() { }
}
public class Bar extends Foo {
    String go() { // Not legal! Can't change only the return type
        return null;
    }
}

```

Overriding and Return Types, and Covariant Returns

When a subclass wants to change the method implementation of an inherited method (an override), the subclass must define a method that matches the inherited version exactly. Or, as of Java 5, you're allowed to change the return type in the overriding method as long as the new return type is a *subtype* of the declared return type of the overridden (superclass) method.

Let's look at a covariant return in action:

```

class Alpha {
    Alpha doStuff(char c) {
        return new Alpha();
    }
}

class Beta extends Alpha {
    Beta doStuff(char c) { // legal override in Java 1.5
        return new Beta();
    }
}

```

As of Java 5, this code will compile. If you were to attempt to compile this code with a 1.4 compiler or with the source flag as follows:

```
javac -source 1.4 Beta.java
```

you would get a compiler error something like this:

```
attempting to use incompatible return type
```

(We'll talk more about compiler flags in Chapter 10.)

Other rules apply to overriding, including those for access modifiers and declared exceptions, but those rules aren't relevant to the return type discussion.

For the exam, be sure you know that overloaded methods can change the return type, but overriding methods can do so only within the bounds of covariant returns. Just that knowledge alone will help you through a wide range of exam questions.

Returning a Value

You have to remember only six rules for returning a value:

1. You can return `null` in a method with an object reference return type.

```
public Button doStuff() {  
    return null;  
}
```

2. An array is a perfectly legal return type.

```
public String[] go() {  
    return new String[] { "Fred", "Barney", "Wilma" };  
}
```

3. In a method with a primitive return type, you can return any value or variable that can be implicitly converted to the declared return type.

```
public int foo() {  
    char c = 'c';  
    return c; // char is compatible with int  
}
```


4. In a method with a primitive return type, you can return any value or variable that can be explicitly cast to the declared return type.

```
public int foo () {
    float f = 32.5f;
    return (int) f;
}
```

5. You must *not* return anything from a method with a void return type.

```
public void bar() {
    return "this is it"; // Not legal!!
}
```

6. In a method with an object reference return type, you can return any object type that can be implicitly cast to the declared return type.

```
public Animal getAnimal() {
    return new Horse(); // Assume Horse extends Animal
}
```

```
public Object getObject() {
    int[] nums = {1,2,3};
    return nums; // Return an int array,
                // which is still an object
}
```

```
public interface Chewable { }
public class Gum implements Chewable { }
```

```
public class TestChewable {
    // Method with an interface return type
    public Chewable getChewable() {
        return new Gum(); // Return interface implementer
    }
}
```

exam

W a t c h

Watch for methods that declare an abstract class or interface return type, and know that any object that passes the IS-A test (in other words, would test true using the instanceof operator) can be returned from that method— for example:

```
public abstract class Animal { }
public class Bear extends Animal { }
public class Test {
    public Animal go() {
        return new Bear(); // OK, Bear "is-a" Animal
    }
}
```

This code will compile, the return value is a subtype.

CERTIFICATION OBJECTIVE

Constructors and Instantiation (Exam Objectives 1.6, 5.3, and 5.4)

1.6 Given a set of classes and superclasses, develop constructors for one or more of the classes. Given a class declaration, determine if a default constructor will be created, and if so, determine the behavior of that constructor. Given a nested or nonnested class listing, write code to instantiate the class.

5.3 Explain the effect of modifiers on inheritance with respect to constructors, instance or static variables, and instance or static methods.

5.4 Given a scenario, develop code that declares and/or invokes overridden or overloaded methods and code that declares and/or invokes superclass, overridden, or overloaded constructors.

Objects are constructed. You can't make a new object without invoking a constructor. In fact, you can't make a new object without invoking not just the constructor of the object's actual class type, but also the constructor of each of its superclasses! Constructors are the code that runs whenever you use the keyword `new`. OK, to be a bit more accurate, there can also be initialization blocks that run when you say `new`, but we're going to cover them (init blocks), and their static initialization counterparts, in the next chapter. We've got plenty to talk about here—we'll look at how constructors are coded, who codes them, and how they work at runtime. So grab your hardhat and a hammer, and let's do some object building.

Constructor Basics

Every class, *including abstract classes*, **MUST** have a constructor. Burn that into your brain. But just because a class must have one, doesn't mean the programmer has to type it. A constructor looks like this:

```
class Foo {  
    Foo() { } // The constructor for the Foo class  
}
```

Notice what's missing? There's no return type! Two key points to remember about constructors are that they have no return type and their names must exactly match the class name. Typically, constructors are used to initialize instance variable state, as follows:

```
class Foo {  
    int size;  
    String name;  
    Foo(String name, int size) {  
        this.name = name;  
        this.size = size;  
    }  
}
```

In the preceding code example, the `Foo` class does not have a no-arg constructor. That means the following will fail to compile:

```
Foo f = new Foo(); // Won't compile, no matching constructor
```

but the following will compile:

```
Foo f = new Foo("Fred", 43); // No problem. Arguments match
                               // the Foo constructor.
```

So it's very common (and desirable) for a class to have a no-arg constructor, regardless of how many other overloaded constructors are in the class (yes, constructors can be overloaded). You can't always make that work for your classes; occasionally you have a class where it makes no sense to create an instance without supplying information to the constructor. A `java.awt.Color` object, for example, can't be created by calling a no-arg constructor, because that would be like saying to the JVM, "Make me a new `Color` object, and I really don't care what color it is...you pick." Do you seriously want the JVM making your style decisions?

Constructor Chaining

We know that constructors are invoked at runtime when you say `new` on some class type as follows:

```
Horse h = new Horse();
```

But what *really* happens when you say `new Horse()` ?
(Assume `Horse` extends `Animal` and `Animal` extends `Object`.)

1. `Horse` constructor is invoked. Every constructor invokes the constructor of its superclass with an (implicit) call to `super()`, unless the constructor invokes an overloaded constructor of the same class (more on that in a minute).
2. `Animal` constructor is invoked (`Animal` is the superclass of `Horse`).
3. `Object` constructor is invoked (`Object` is the ultimate superclass of all classes, so class `Animal` extends `Object` even though you don't actually type "extends `Object`" into the `Animal` class declaration. It's implicit.) At this point we're on the top of the stack.
4. `Object` instance variables are given their explicit values. By *explicit* values, we mean values that are assigned at the time the variables are declared, like `int x = 27`, where "27" is the explicit value (as opposed to the default value) of the instance variable.
5. `Object` constructor completes.
6. `Animal` instance variables are given their explicit values (if any).
7. `Animal` constructor completes.

8. Horse instance variables are given their explicit values (if any).
9. Horse constructor completes.

Figure 2-6 shows how constructors work on the call stack.

FIGURE 2-6

Constructors on
the call stack

4. <code>Object()</code>
3. <code>Animal()</code> calls <code>super()</code>
2. <code>Horse()</code> calls <code>super()</code>
1. <code>main()</code> calls <code>new Horse()</code>

Rules for Constructors

The following list summarizes the rules you'll need to know for the exam (and to understand the rest of this section). You **MUST** remember these, so be sure to study them more than once.

- Constructors can use any access modifier, including `private`. (A `private` constructor means only code within the class itself can instantiate an object of that type, so if the `private` constructor class wants to allow an instance of the class to be used, the class must provide a static method or variable that allows access to an instance created from within the class.)
- The constructor name must match the name of the class.
- Constructors must not have a return type.
- It's legal (but stupid) to have a method with the same name as the class, but that doesn't make it a constructor. If you see a return type, it's a method rather than a constructor. In fact, you could have both a method and a constructor with the same name—the name of the class—in the same class, and that's not a problem for Java. Be careful not to mistake a method for a constructor—be sure to look for a return type.
- If you don't type a constructor into your class code, a default constructor will be automatically generated by the compiler.
- The default constructor is **ALWAYS** a no-arg constructor.
- If you want a no-arg constructor and you've typed any other constructor(s) into your class code, the compiler won't provide the no-arg constructor (or

any other constructor) for you. In other words, if you've typed in a constructor with arguments, you won't have a no-arg constructor unless you type it in yourself!

- Every constructor has, as its first statement, either a call to an overloaded constructor (`this()`) or a call to the superclass constructor (`super()`), although remember that this call can be inserted by the compiler.
- If you do type in a constructor (as opposed to relying on the compiler-generated default constructor), and you do not type in the call to `super()` or a call to `this()`, the compiler will insert a no-arg call to `super()` for you, as the very first statement in the constructor.
- A call to `super()` can be either a no-arg call or can include arguments passed to the super constructor.
- A no-arg constructor is not necessarily the default (i.e., compiler-supplied) constructor, although the default constructor is always a no-arg constructor. The default constructor is the one the compiler provides! While the default constructor is always a no-arg constructor, you're free to put in your own no-arg constructor.
- You cannot make a call to an instance method, or access an instance variable, until after the super constructor runs.
- Only static variables and methods can be accessed as part of the call to `super()` or `this()`. (Example: `super(Animal.NAME)` is OK, because `NAME` is declared as a static variable.)
- Abstract classes have constructors, and those constructors are always called when a concrete subclass is instantiated.
- Interfaces do not have constructors. Interfaces are not part of an object's inheritance tree.
- The only way a constructor can be invoked is from within another constructor. In other words, you can't write code that actually calls a constructor as follows:

```
class Horse {  
    Horse() { } // constructor  
    void doStuff() {  
        Horse(); // calling the constructor - illegal!  
    }  
}
```

Determine Whether a Default Constructor Will Be Created

The following example shows a Horse class with two constructors:

```
class Horse {  
    Horse() { }  
    Horse(String name) { }  
}
```

Will the compiler put in a default constructor for the class above? No!
How about for the following variation of the class?

```
class Horse {  
    Horse(String name) { }  
}
```

Now will the compiler insert a default constructor? No!
What about this class?

```
class Horse { }
```

Now we're talking. The compiler will generate a default constructor for the preceding class, because the class doesn't have any constructors defined.
OK, what about this class?

```
class Horse {  
    void Horse() { }  
}
```

It might look like the compiler won't create one, since there already is a constructor in the Horse class. Or is there? Take another look at the preceding Horse class.

What's wrong with the Horse() constructor? It isn't a constructor at all! It's simply a method that happens to have the same name as the class. Remember, the return type is a dead giveaway that we're looking at a method, and not a constructor.

How do you know for sure whether a default constructor will be created?

Because you didn't write any constructors in your class.

How do you know what the default constructor will look like?

Because...

- The default constructor has the same access modifier as the class.
- The default constructor has no arguments.
- The default constructor includes a no-arg call to the super constructor (`super()`).

Table 2-4 shows what the compiler will (or won't) generate for your class.

What happens if the super constructor has arguments?

Constructors can have arguments just as methods can, and if you try to invoke a method that takes, say, an `int`, but you don't pass anything to the method, the compiler will complain as follows:

```
class Bar {
    void takeInt(int x) { }
}

class UseBar {
    public static void main (String [] args) {
        Bar b = new Bar();
        b.takeInt(); // Try to invoke a no-arg takeInt() method
    }
}
```

The compiler will complain that you can't invoke `takeInt()` without passing an `int`. Of course, the compiler enjoys the occasional riddle, so the message it spits out on some versions of the JVM (your mileage may vary) is less than obvious:

```
UseBar.java:7: takeInt(int) in Bar cannot be applied to ()
    b.takeInt();
      ^
```

But you get the idea. The bottom line is that there must be a match for the method. And by match, we mean that the argument types must be able to accept the values or variables you're passing, and in the order you're passing them. Which brings us back to constructors (and here you were thinking we'd never get there), which work exactly the same way.

TABLE 2-4 Compiler-Generated Constructor Code

Class Code (What You Type)	Compiler Generated Constructor Code (in Bold)
<code>class Foo { }</code>	<code>class Foo { Foo() { super(); } }</code>
<code>class Foo { Foo() { } }</code>	<code>class Foo { Foo() { super(); } }</code>
<code>public class Foo { }</code>	<code>public class Foo { public Foo() { super(); } }</code>
<code>class Foo { Foo(String s) { } }</code>	<code>class Foo { Foo(String s) { super(); } }</code>
<code>class Foo { Foo(String s) { super(); } }</code>	<i>Nothing, compiler doesn't need to insert anything.</i>
<code>class Foo { void Foo() { } }</code>	<code>class Foo { void Foo() { } Foo() { super(); } }</code> (void Foo() is a method, not a constructor.)

So if your super constructor (that is, the constructor of your immediate superclass/parent) has arguments, you must type in the call to `super()`, supplying the appropriate arguments. Crucial point: if your superclass does not have a no-arg

constructor, you must type a constructor in your class (the subclass) because you need a place to put in the call to `super` with the appropriate arguments.

The following is an example of the problem:

```
class Animal {
    Animal(String name) { }
}

class Horse extends Animal {
    Horse() {
        super(); // Problem!
    }
}
```

And once again the compiler treats us with the stunningly lucid:

```
Horse.java:7: cannot resolve symbol
symbol   : constructor Animal ()
location: class Animal
    super(); // Problem!
    ^
```

If you're lucky (and it's a full moon), *your* compiler might be a little more explicit. But again, the problem is that there just isn't a match for what we're trying to invoke with `super()`—an `Animal` constructor with no arguments.

Another way to put this is that if your superclass does *not* have a no-arg constructor, then in your subclass you will not be able to use the default constructor supplied by the compiler. It's that simple. Because the compiler can *only* put in a call to a no-arg `super()`, you won't even be able to compile something like this:

```
class Clothing {
    Clothing(String s) { }
}

class TShirt extends Clothing { }
```

Trying to compile this code gives us exactly the same error we got when we put a constructor in the subclass with a call to the no-arg version of `super()`:

```
Clothing.java:4: cannot resolve symbol
symbol   : constructor Clothing ()
location: class Clothing
```

```
class TShirt extends Clothing { }
^
```

In fact, the preceding Clothing and TShirt code is implicitly the same as the following code, where we've supplied a constructor for TShirt that's identical to the default constructor supplied by the compiler:

```
class Clothing {
    Clothing(String s) { }
}
class TShirt extends Clothing {
    // Constructor identical to compiler-supplied
    // default constructor
    TShirt() {
        super(); // Won't work!
    }           // Invokes a no-arg Clothing() constructor,
                // but there isn't one!
```

One last point on the whole default constructor thing (and it's probably very obvious, but we have to say it or we'll feel guilty for years), **constructors are never inherited**. They aren't methods. They can't be overridden (because they aren't methods and only instance methods can be overridden). So the type of constructor(s) your superclass has in no way determines the type of default constructor you'll get. Some folks mistakenly believe that the default constructor somehow matches the super constructor, either by the arguments the default constructor will have (remember, the default constructor is always a no-arg), or by the arguments used in the compiler-supplied call to `super()`.

So, although constructors can't be overridden, you've already seen that they can be overloaded, and typically are.

Overloaded Constructors

Overloading a constructor means typing in multiple versions of the constructor, each having a different argument list, like the following examples:

```
class Foo {
    Foo() { }
    Foo(String s) { }
}
```

The preceding Foo class has two overloaded constructors, one that takes a string, and one with no arguments. Because there's no code in the no-arg version, it's actually identical to the default constructor the compiler supplies, but remember—since there's already a constructor in this class (the one that takes a string), the compiler won't supply a default constructor. If you want a no-arg constructor to overload the with-args version you already have, you're going to have to type it yourself, just as in the Foo example.

Overloading a constructor is typically used to provide alternate ways for clients to instantiate objects of your class. For example, if a client knows the animal name, they can pass that to an Animal constructor that takes a string. But if they don't know the name, the client can call the no-arg constructor and that constructor can supply a default name. Here's what it looks like:

```

1. public class Animal {
2.     String name;
3.     Animal(String name) {
4.         this.name = name;
5.     }
6.
7.     Animal() {
8.         this(makeRandomName());
9.     }
10.
11.     static String makeRandomName() {
12.         int x = (int) (Math.random() * 5);
13.         String name = new String[] {"Fluffy", "Fido",
                                         "Rover", "Spike",
                                         "Gigi"}[x];
14.         return name;
15.     }
16.
17.     public static void main (String [] args) {
18.         Animal a = new Animal();
19.         System.out.println(a.name);
20.         Animal b = new Animal("Zeus");
21.         System.out.println(b.name);
22.     }
23. }
```

Running the code four times produces this output:

```
% java Animal
Gigi
Zeus

% java Animal
Fluffy
Zeus

% java Animal
Rover
Zeus

% java Animal
Fluffy
Zeus
```

There's a lot going on in the preceding code. Figure 2-7 shows the call stack for constructor invocations when a constructor is overloaded. Take a look at the call stack, and then let's walk through the code straight from the top.

FIGURE 2-7

Overloaded
constructors on
the call stack

4. Object ()
3. Animal (String s) calls super ()
2. Animal () calls this (randomlyChosenNameString)
1. main () calls new Animal ()

- **Line 2** Declare a String instance variable name.
- **Lines 3–5** Constructor that takes a String, and assigns it to instance variable name.
- **Line 7** Here's where it gets fun. Assume every animal needs a name, but the client (calling code) might not always know what the name should be, so you'll assign a random name. The no-arg constructor generates a name by invoking the makeRandomName() method.
- **Line 8** The no-arg constructor invokes its own overloaded constructor that takes a String, in effect calling it the same way it would be called if

client code were doing a `new` to instantiate an object, passing it a `String` for the name. The overloaded invocation uses the keyword `this`, but uses it as though it were a method name, `this()`. So line 8 is simply calling the constructor on line 3, passing it a randomly selected `String` rather than a client-code chosen name.

- **Line 11** Notice that the `makeRandomName()` method is marked `static`! That's because you cannot invoke an instance (in other words, `nonstatic`) method (or access an instance variable) until after the super constructor has run. And since the super constructor will be invoked from the constructor on line 3, rather than from the one on line 7, line 8 can use only a static method to generate the name. If we wanted all animals not specifically named by the caller to have the same default name, say, "Fred," then line 8 could have read `this("Fred");` rather than calling a method that returns a string with the randomly chosen name.
- **Line 12** This doesn't have anything to do with constructors, but since we're all here to learn...it generates a random integer between 0 and 4.
- **Line 13** Weird syntax, we know. We're creating a new `String` object (just a single `String` instance), but we want the string to be selected randomly from a list. Except we don't have the list, so we need to make it. So in that one line of code we
 1. Declare a `String` variable, `name`.
 2. Create a `String` array (anonymously—we don't assign the array itself to anything).
 3. Retrieve the string at index `[x]` (`x` being the random number generated on line 12) of the newly created `String` array.
 4. Assign the string retrieved from the array to the declared instance variable `name`. We could have made it much easier to read if we'd just written

```
String[] nameList = {"Fluffy", "Fido", "Rover", "Spike",  
                    "Gigi"};  
String name = nameList[x];
```

But where's the fun in that? Throwing in unusual syntax (especially for code wholly unrelated to the real question) is in the spirit of the exam. Don't be

startled! (OK, be startled, but then just say to yourself, "Whoa" and get on with it.)

- **Line 18** We're invoking the no-arg version of the constructor (causing a random name from the list to be passed to the other constructor).
- **Line 20** We're invoking the overloaded constructor that takes a string representing the name.

The key point to get from this code example is in line 8. Rather than calling `super()`, we're calling `this()`, and `this()` always means a call to another constructor in the same class. OK, fine, but what happens after the call to `this()`? Sooner or later the `super()` constructor gets called, right? Yes indeed. A call to `this()` just means you're delaying the inevitable. Some constructor, somewhere, must make the call to `super()`.

Key Rule: The first line in a constructor must be a call to `super()` or a call to `this()`.

No exceptions. If you have neither of those calls in your constructor, the compiler will insert the no-arg call to `super()`. In other words, if constructor `A()` has a call to `this()`, the compiler knows that constructor `A()` will not be the one to invoke `super()`.

The preceding rule means a constructor can never have both a call to `super()` and a call to `this()`. Because each of those calls must be the first statement in a constructor, you can't legally use both in the same constructor. That also means the compiler will not put a call to `super()` in any constructor that has a call to `this()`.

Thought question: What do you think will happen if you try to compile the following code?

```
class A {
    A() {
        this("foo");
    }
    A(String s) {
        this();
    }
}
```

Your compiler may not actually catch the problem (it varies depending on your compiler, but most won't catch the problem). It assumes you know what you're

doing. Can you spot the flaw? Given that a super constructor must always be called, where would the call to `super()` go? Remember, the compiler won't put in a default constructor if you've already got one or more constructors in your class. And when the compiler doesn't put in a default constructor, it still inserts a call to `super()` in any constructor that doesn't explicitly have a call to the super constructor—unless, that is, the constructor already has a call to `this()`. So in the preceding code, where can `super()` go? The only two constructors in the class both have calls to `this()`, and in fact you'll get exactly what you'd get if you typed the following method code:

```
public void go() {  
    doStuff();  
}  
  
public void doStuff() {  
    go();  
}
```

Now can you see the problem? Of course you can. The stack explodes! It gets higher and higher and higher until it just bursts open and method code goes spilling out, oozing out of the JVM right onto the floor. Two overloaded constructors both calling `this()` are two constructors calling each other. Over and over and over, resulting in

```
% java A  
Exception in thread "main" java.lang.StackOverflowError
```

The benefit of having overloaded constructors is that you offer flexible ways to instantiate objects from your class. The benefit of having one constructor invoke another overloaded constructor is to avoid code duplication. In the `Animal` example, there wasn't any code other than setting the name, but imagine if after line 4 there was still more work to be done in the constructor. By putting all the other constructor work in just one constructor, and then having the other constructors invoke it, you don't have to write and maintain multiple versions of that other important constructor code. Basically, each of the other not-the-real-one overloaded constructors will call another overloaded constructor, passing it whatever data it needs (data the client code didn't supply).

Constructors and instantiation become even more exciting (just when you thought it was safe), when you get to inner classes, but we know you can stand to

have only so much fun in one chapter, so we're holding the rest of the discussion on instantiating inner classes until Chapter 8.

CERTIFICATION OBJECTIVE

Statics (Exam Objective 1.3)

1.3 Develop code that declares, initializes, and uses primitives, arrays, enums, and objects as static, instance, and local variables. Also, use legal identifiers for variable names.

Static Variables and Methods

The `static` modifier has such a profound impact on the behavior of a method or variable that we're treating it as a concept entirely separate from the other modifiers. To understand the way a `static` member works, we'll look first at a reason for using one. Imagine you've got a utility class with a method that always runs the same way; its sole function is to return, say, a random number. It wouldn't matter which instance of the class performed the method—it would always behave exactly the same way. In other words, the method's behavior has no dependency on the state (instance variable values) of an object. So why, then, do you need an object when the method will never be instance-specific? Why not just ask the class itself to run the method?

Let's imagine another scenario: Suppose you want to keep a running count of all instances instantiated from a particular class. Where do you actually keep that variable? It won't work to keep it as an instance variable within the class whose instances you're tracking, because the count will just be initialized back to a default value with each new instance. The answer to both the utility-method-always-runs-the-same scenario and the keep-a-running-total-of-instances scenario is to use the `static` modifier. Variables and methods marked `static` belong to the class, rather than to any particular instance. In fact, you can use a `static` method or variable without having any instances of that class at all. You need only have the class available to be able to invoke a `static` method or access a `static` variable. `static` variables, too, can be accessed without having an instance of a class. But if there are instances, a `static` variable of a class will be shared by all instances of that class; there is only one copy.

The following code declares and uses a `static` counter variable:

```

class Frog {
    static int frogCount = 0; // Declare and initialize
                               // static variable

    public Frog() {
        frogCount += 1; // Modify the value in the constructor
    }
    public static void main (String [] args) {
        new Frog();
        new Frog();
        new Frog();
        System.out.println("Frog count is now " + frogCount);
    }
}

```

In the preceding code, the static `frogCount` variable is set to zero when the `Frog` class is first loaded by the JVM, before any `Frog` instances are created! (By the way, you don't actually need to initialize a static variable to zero; static variables get the same default values instance variables get.) Whenever a `Frog` instance is created, the `Frog` constructor runs and increments the static `frogCount` variable. When this code executes, three `Frog` instances are created in `main()`, and the result is

```
Frog count is now 3
```

Now imagine what would happen if `frogCount` were an instance variable (in other words, nonstatic):

```

class Frog {
    int frogCount = 0; // Declare and initialize
                       // instance variable

    public Frog() {
        frogCount += 1; // Modify the value in the constructor
    }
    public static void main (String [] args) {
        new Frog();
        new Frog();
        new Frog();
        System.out.println("Frog count is now " + frogCount);
    }
}

```

When this code executes, it should still create three `Frog` instances in `main()`, but the result is...a compiler error! We can't get this code to compile, let alone run.

```
Frog.java:11: nonstatic variable frogCount cannot be referenced
from a static context
```

```
    System.out.println("Frog count is " + frogCount);
                                   ^
```

```
1 error
```

The JVM doesn't know which Frog object's `frogCount` you're trying to access. The problem is that `main()` is itself a `static` method, and thus isn't running against any particular instance of the class, rather just on the class itself. A `static` method can't access a `nonstatic` (instance) variable, because there is no instance! That's not to say there aren't instances of the class alive on the heap, but rather that even if there are, the `static` method doesn't know anything about them. The same applies to instance methods; a `static` method can't directly invoke a `nonstatic` method. Think `static` = class, `nonstatic` = instance. Making the method called by the JVM (`main()`) a `static` method means the JVM doesn't have to create an instance of your class just to start running code.

exam

Watch

One of the mistakes most often made by new Java programmers is attempting to access an instance variable (which means nonstatic variable) from the static `main()` method (which doesn't know anything about any instances, so it can't access the variable). The following code is an example of illegal access of a nonstatic variable from a static method:

```
class Foo {
    int x = 3;
    public static void main (String [] args) {
        System.out.println("x is " + x);
    }
}
```

Understand that this code will never compile, because you can't access a nonstatic (instance) variable from a static method. Just think of the compiler saying, "Hey, I have no idea which Foo object's `x` variable you're trying to print!" Remember, it's the class running the `main()` method, not an instance of the class.

exam**Watch**

(continued) **Of course, the tricky part for the exam is that the question won't look as obvious as the preceding code. The problem you're being tested for—accessing a nonstatic variable from a static method—will be buried in code that might appear to be testing something else. For example, the preceding code would be more likely to appear as**

```
class Foo {
    int x = 3;
    float y = 4.3f;
    public static void main (String [] args) {
        for (int z = x; z < ++x; z--, y = y + z)
            // complicated looping and branching code
        }
    }
}
```

So while you're trying to follow the logic, the real issue is that *x* and *y* can't be used within `main()`, because *x* and *y* are instance, not static, variables! The same applies for accessing nonstatic methods from a static method. The rule is, a static method of a class can't access a nonstatic (instance) method or variable of its own class.

Accessing Static Methods and Variables

Since you don't need to have an instance in order to invoke a static method or access a static variable, then how do you invoke or use a static member? What's the syntax? We know that with a regular old instance method, you use the dot operator on a reference to an instance:

```
class Frog {
    int frogSize = 0;
    public int getFrogSize() {
        return frogSize;
    }
    public Frog(int s) {
        frogSize = s;
    }
    public static void main (String [] args) {
```

```

        Frog f = new Frog(25);
        System.out.println(f.getFrogSize()); // Access instance
                                              // method using f
    }
}

```

In the preceding code, we instantiate a `Frog`, assign it to the reference variable `f`, and then use that `f` reference to invoke a method on the `Frog` instance we just created. In other words, the `getFrogSize()` method is being invoked on a specific `Frog` object on the heap.

But this approach (using a reference to an object) isn't appropriate for accessing a static method, because there might not be any instances of the class at all! So, the way we access a static method (or static variable) is to use the dot operator on the class name, as opposed to using it on a reference to an instance, as follows:

```

class Frog {
    static int frogCount = 0; // Declare and initialize
                              // static variable

    public Frog() {
        frogCount += 1; // Modify the value in the constructor
    }
}

class TestFrog {
    public static void main (String [] args) {
        new Frog();
        new Frog();
        new Frog();
        System.out.print("frogCount:"+Frog.frogCount); //Access
                                                         // static variable
    }
}

```

But just to make it really confusing, the Java language also allows you to use an object reference variable to access a static member:

```

Frog f = new Frog();
int frogs = f.frogCount; // Access static variable
                      // FrogCount using f

```

In the preceding code, we instantiate a Frog, assign the new Frog object to the reference variable `f`, and then use the `f` reference to invoke a `static` method! But even though we are using a specific Frog instance to access the `static` method, the rules haven't changed. This is merely a syntax trick to let you use an object reference variable (but not the object it refers to) to get to a `static` method or variable, but the `static` member is still unaware of the particular instance used to invoke the `static` member. In the Frog example, the compiler knows that the reference variable `f` is of type Frog, and so the Frog class `static` method is run with no awareness or concern for the Frog instance at the other end of the `f` reference. In other words, the compiler cares only that reference variable `f` is declared as type Frog. Figure 2-8 illustrates the effects of the `static` modifier on methods and variables.

FIGURE 2-8

The effects of
`static` on methods
and variables

```
class Foo

int size = 42;
static void doMore( ){
    int x = size;
}
```

static method cannot
access an instance
(non-static) variable

```
class Bar

void go ( );
static void doMore( ){
    go( );
}
```

static method cannot
access a non-static
method

```
class Baz

static int count;
static void woo( ){ }
static void doMore( ){
    woo( );
    int x = count;
}
```

static method
can access a static
method or variable

Finally, remember that *static methods can't be overridden*! This doesn't mean they can't be redefined in a subclass, but redefining and overriding aren't the same thing. Let's take a look at an example of a redefined (remember, not overridden), static method:

```
class Animal {
    static void doStuff() {
        System.out.print("a ");
    }
}
class Dog extends Animal {
    static void doStuff() {           // it's a redefinition,
                                    // not an override

        System.out.print("d ");
    }
    public static void main(String [] args) {
        Animal [] a = {new Animal(), new Dog(), new Animal()};
        for(int x = 0; x < a.length; x++)
            a[x].doStuff();           // invoke the static method
    }
}
```

Running this code produces the output:

```
a a a
```

Remember, the syntax `a[x].doStuff()` is just a shortcut (the syntax trick)...the compiler is going to substitute something like `Animal.doStuff()` instead. Notice that we didn't use the Java 1.5 *enhanced for loop* here (covered in Chapter 5), even though we could have. Expect to see a mix of both Java 1.4 and Java 5 coding styles and practices on the exam.

CERTIFICATION OBJECTIVE

Coupling and Cohesion (Exam Objective 5.1)

5.1 Develop code that implements tight encapsulation, loose coupling, and high cohesion in classes, and describe the benefits.

We're going to admit it up front. The Sun exam's definitions for cohesion and coupling are somewhat subjective, so what we discuss in this chapter is from the perspective of the exam, and by no means The One True Word on these two OO design principles. It may not be exactly the way that you've learned it, but it's what you need to understand to answer the questions. You'll have very few questions about coupling and cohesion on the real exam.

These two topics, coupling and cohesion, have to do with the quality of an OO design. In general, good OO design calls for *loose coupling* and shuns tight coupling, and good OO design calls for *high cohesion*, and shuns low cohesion. As with most OO design discussions, the goals for an application are

- Ease of creation
- Ease of maintenance
- Ease of enhancement

Coupling

Let's start by making an attempt at a definition of coupling. Coupling is the degree to which one class knows about another class. If the only knowledge that class A has about class B, is what class B has exposed through its interface, then class A and class B are said to be loosely coupled...that's a good thing. If, on the other hand, class A relies on parts of class B that are not part of class B's interface, then the coupling between the classes is tighter...*not* a good thing. In other words, if A knows more than it should about the way in which B was implemented, then A and B are tightly coupled.

Using this second scenario, imagine what happens when class B is enhanced. It's quite possible that the developer enhancing class B has no knowledge of class A, why would she? Class B's developer ought to feel that any enhancements that don't break the class's interface should be safe, so she might change some noninterface part of the class, which then causes class A to break.

At the far end of the coupling spectrum is the horrible situation in which class A knows non-API stuff about class B, and class B knows non-API stuff about class A... this is REALLY BAD. If either class is ever changed, there's a chance that the other class will break. Let's look at an obvious example of tight coupling, which has been enabled by poor encapsulation:

```
class DoTaxes {  
    float rate;
```



```

float doColorado() {
    SalesTaxRates str = new SalesTaxRates();
    rate = str.salesRate;           // ouch
                                   // this should be a method call:
                                   // rate = str.getSalesRate("CO");
    // do stuff with rate
}
}

class SalesTaxRates {
    public float salesRate;           // should be private
    public float adjustedSalesRate;  // should be private

    public float getSalesRate(String region) {
        salesRate = new DoTaxes().doColorado();    // ouch again!
        // do region-based calculations
        return adjustedSalesRate;
    }
}

```

All nontrivial OO applications are a mix of many classes and interfaces working together. Ideally, all interactions between objects in an OO system should use the APIs, in other words, the contracts, of the objects' respective classes. Theoretically, if all of the classes in an application have well-designed APIs, then it should be possible for all interclass interactions to use those APIs exclusively. As we discussed earlier in this chapter, an aspect of good class and API design is that classes should be well encapsulated.

The bottom line is that coupling is a somewhat subjective concept. Because of this, the exam will test you on really obvious examples of tight coupling; you won't be asked to make subtle judgment calls.

Cohesion

While coupling has to do with how classes interact with each other, cohesion is all about how a single class is designed. The term *cohesion* is used to indicate the degree to which a class has a single, well-focused purpose. Keep in mind that cohesion is a subjective concept. The more focused a class is, the higher its cohesiveness—a good thing. The key benefit of high cohesion is that such classes are typically much easier to maintain (and less frequently changed) than classes with low cohesion. Another benefit of high cohesion is that classes with a well-focused purpose tend to be more reusable than other classes. Let's take a look at a pseudo-code example:

```
class BudgetReport {  
    void connectToRDBMS() { }  
    void generateBudgetReport() { }  
    void saveToFile() { }  
    void print() { }  
}
```

Now imagine your manager comes along and says, "Hey you know that accounting application we're working on? The clients just decided that they're also going to want to generate a revenue projection report, oh and they want to do some inventory reporting also. They do like our reporting features however, so make sure that all of these reports will let them choose a database, choose a printer, and save generated reports to data files..." Ouch!

Rather than putting all the printing code into one report class, we probably would have been better off with the following design right from the start:

```
class BudgetReport {  
    Options getReportingOptions() { }  
    void generateBudgetReport(Options o) { }  
}  
  
class ConnectToRDBMS {  
    DBconnection getRDBMS() { }  
}  
  
class PrintStuff {  
    PrintOptions getPrintOptions() { }  
}  
  
class FileSaver {  
    SaveOptions getFileSaveOptions() { }  
}
```

This design is much more cohesive. Instead of one class that does everything, we've broken the system into four main classes, each with a very specific, or *cohesive*, role. Because we've built these specialized, reusable classes, it'll be much easier to write a new report, since we've already got the database connection class, the printing class, and the file saver class, and that means they can be reused by other classes that might want to print a report.

CERTIFICATION SUMMARY

We started the chapter by discussing the importance of encapsulation in good OO design, and then we talked about how good encapsulation is implemented: with private instance variables and public getters and setters.

Next, we covered the importance of inheritance; so that you can grasp overriding, overloading, polymorphism, reference casting, return types, and constructors.

We covered IS-A and HAS-A. IS-A is implemented using inheritance, and HAS-A is implemented by using instance variables that refer to other objects.

Polymorphism was next. Although a reference variable's type can't be changed, it can be used to refer to an object whose type is a subtype of its own. We learned how to determine what methods are invocable for a given reference variable.

We looked at the difference between overridden and overloaded methods, learning that an overridden method occurs when a subclass inherits a method from a superclass, and then reimplements the method to add more specialized behavior. We learned that, at runtime, the JVM will invoke the subclass version on an instance of a subclass, and the superclass version on an instance of the superclass. Abstract methods must be "overridden" (technically, abstract methods must be implemented, as opposed to overridden, since there really isn't anything to override).

We saw that overriding methods must declare the same argument list and return type (or, as of Java 5, they can return a subtype of the declared return type of the superclass overridden method), and that the access modifier can't be more restrictive. The overriding method also can't throw any new or broader checked exceptions that weren't declared in the overridden method. You also learned that the overridden method can be invoked using the syntax `super.doSomething()` ; .

Overloaded methods let you reuse the same method name in a class, but with different arguments (and, optionally, a different return type). Whereas overriding methods must not change the argument list, overloaded methods must. But unlike overriding methods, overloaded methods are free to vary the return type, access modifier, and declared exceptions any way they like.

We learned the mechanics of casting (mostly downcasting), reference variables, when it's necessary, and how to use the `instanceof` operator.

Implementing interfaces came next. An interface describes a *contract* that the implementing class must follow. The rules for implementing an interface are similar to those for extending an abstract class. Also remember that a class can implement more than one interface, and that interfaces can extend another interface.

We also looked at method return types, and saw that you can declare any return type you like (assuming you have access to a class for an object reference return

type), unless you're overriding a method. Barring a covariant return, an overriding method must have the same return type as the overridden method of the superclass. We saw that while overriding methods must not change the return type, overloaded methods can (as long as they also change the argument list).

Finally, you learned that it is legal to return any value or variable that can be implicitly converted to the declared return type. So, for example, a `short` can be returned when the return type is declared as an `int`. And (assuming `Horse` extends `Animal`), a `Horse` reference can be returned when the return type is declared as an `Animal`.

We covered constructors in detail, learning that if you don't provide a constructor for your class, the compiler will insert one. The compiler-generated constructor is called the default constructor, and it is always a no-arg constructor with a no-arg call to `super()`. The default constructor will never be generated if there is even a single constructor in your class (regardless of the arguments of that constructor), so if you need more than one constructor in your class and you want a no-arg constructor, you'll have to write it yourself. We also saw that constructors are not inherited, and that you can be confused by a method that has the same name as the class (which is legal). The return type is the giveaway that a method is not a constructor, since constructors do not have return types.

We saw how all of the constructors in an object's inheritance tree will always be invoked when the object is instantiated using `new`. We also saw that constructors can be overloaded, which means defining constructors with different argument lists. A constructor can invoke another constructor of the same class using the keyword `this()`, as though the constructor were a method named `this()`. We saw that every constructor must have either `this()` or `super()` as the first statement (although the compiler can insert it for you).

We looked at `static` methods and variables. Static members are tied to the class, not an instance, so there is only one copy of any `static` member. A common mistake is to attempt to reference an instance variable from a `static` method. Use the class name with the dot operator to access `static` members.

We discussed the OO concepts of coupling and cohesion. Loose coupling is the desirable state of two or more classes that interact with each other only through their respective APIs. Tight coupling is the undesirable state of two or more classes that know inside details about another class, details not revealed in the class's API. High cohesion is the desirable state of a single class whose purpose and responsibilities are limited and well-focused.

And once again, you learned that the exam includes tricky questions designed largely to test your ability to recognize just how tricky the questions can be.



TWO-MINUTE DRILL

Here are some of the key points from each certification objective in this chapter.

Encapsulation, IS-A, HAS-A (Objective 5.1)

- ☐ Encapsulation helps hide implementation behind an interface (or API).
- ☐ Encapsulated code has two features:
 - ☐ Instance variables are kept protected (usually with the `private` modifier).
 - ☐ Getter and setter methods provide access to instance variables.
- ☐ IS-A refers to inheritance or implementation.
- ☐ IS-A is expressed with the keyword `extends`.
- ☐ IS-A, "inherits from," and "is a subtype of" are all equivalent expressions.
- ☐ HAS-A means an instance of one class "has a" reference to an instance of another class or another instance of the same class.

Inheritance (Objective 5.5)

- ☐ Inheritance allows a class to be a subclass of a superclass, and thereby inherit `public` and `protected` variables and methods of the superclass.
- ☐ Inheritance is a key concept that underlies IS-A, polymorphism, overriding, overloading, and casting.
- ☐ All classes (except class `Object`), are subclasses of type `Object`, and therefore they inherit `Object`'s methods.

Polymorphism (Objective 5.2)

- ☐ Polymorphism means "many forms."
- ☐ A reference variable is always of a single, unchangeable type, but it can refer to a subtype object.
- ☐ A single object can be referred to by reference variables of many different types—as long as they are the same type or a supertype of the object.
- ☐ The reference variable's type (not the object's type), determines which methods can be called!
- ☐ Polymorphic method invocations apply only to overridden *instance* methods.

Overriding and Overloading (Objectives 1.5 and 5.4)

- ☐ Methods can be overridden or overloaded; constructors can be overloaded but not overridden.
- ☐ Abstract methods must be overridden by the first concrete (non-abstract) subclass.
- ☐ With respect to the method it overrides, the overriding method
 - ☐ Must have the same argument list.
 - ☐ Must have the same return type, except that as of Java 5, the return type can be a subclass—this is known as a covariant return.
 - ☐ Must not have a more restrictive access modifier.
 - ☐ May have a less restrictive access modifier.
 - ☐ Must not throw new or broader checked exceptions.
 - ☐ May throw fewer or narrower checked exceptions, or any unchecked exception.
- ☐ `final` methods cannot be overridden.
- ☐ Only inherited methods may be overridden, and remember that private methods are not inherited.
- ☐ A subclass uses `super.overriddenMethodName()` to call the superclass version of an overridden method.
- ☐ Overloading means reusing a method name, but with different arguments.
- ☐ Overloaded methods
 - ☐ Must have different argument lists
 - ☐ May have different return types, if argument lists are also different
 - ☐ May have different access modifiers
 - ☐ May throw different exceptions
- ☐ Methods from a superclass can be overloaded in a subclass.
- ☐ Polymorphism applies to overriding, not to overloading.
- ☐ Object type (not the reference variable's type), determines which overridden method is used at runtime.
- ☐ Reference type determines which overloaded method will be used at compile time.

Reference Variable Casting (Objective 5.2)

- ☐ There are two types of reference variable casting: downcasting and upcasting.
- ☐ Downcasting: If you have a reference variable that refers to a subtype object, you can assign it to a reference variable of the subtype. You must make an explicit cast to do this, and the result is that you can access the subtype's members with this new reference variable.
- ☐ Upcasting: You can assign a reference variable to a supertype reference variable explicitly or implicitly. This is an inherently safe operation because the assignment restricts the access capabilities of the new variable.

Implementing an Interface (Objective 1.2)

- ☐ When you implement an interface, you are fulfilling its contract.
- ☐ You implement an interface by properly and concretely overriding all of the methods defined by the interface.
- ☐ A single class can implement many interfaces.

Return Types (Objective 1.5)

- ☐ Overloaded methods can change return types; overridden methods cannot, except in the case of covariant returns.
- ☐ Object reference return types can accept `null` as a return value.
- ☐ An array is a legal return type, both to declare and return as a value.
- ☐ For methods with primitive return types, any value that can be implicitly converted to the return type can be returned.
- ☐ Nothing can be returned from a `void`, but you can return nothing. You're allowed to simply say `return`, in any method with a `void` return type, to bust out of a method early. But you can't return nothing from a method with a non-`void` return type.
- ☐ Methods with an object reference return type, can return a subtype.
- ☐ Methods with an interface return type, can return any implementer.

Constructors and Instantiation (Objectives 1.6 and 5.4)

- ☐ A constructor is always invoked when a new object is created.

- ☐ Each superclass in an object's inheritance tree will have a constructor called.
- ☐ Every class, even an abstract class, has at least one constructor.
- ☐ Constructors must have the same name as the class.
- ☐ Constructors don't have a return type. If you see code with a return type, it's a method with the same name as the class, it's not a constructor.
- ☐ Typical constructor execution occurs as follows:
 - ☐ The constructor calls its superclass constructor, which calls its superclass constructor, and so on all the way up to the Object constructor.
 - ☐ The Object constructor executes and then returns to the calling constructor, which runs to completion and then returns to its calling constructor, and so on back down to the completion of the constructor of the actual instance being created.
- ☐ Constructors can use any access modifier (even `private`!).
- ☐ The compiler will create a default constructor if you don't create any constructors in your class.
- ☐ The default constructor is a no-arg constructor with a no-arg call to `super()`.
- ☐ The first statement of every constructor must be a call to either `this()` (an overloaded constructor) or `super()`.
- ☐ The compiler will add a call to `super()` unless you have already put in a call to `this()` or `super()`.
- ☐ Instance members are accessible only after the super constructor runs.
- ☐ Abstract classes have constructors that are called when a concrete subclass is instantiated.
- ☐ Interfaces do not have constructors.
- ☐ If your superclass does not have a no-arg constructor, you must create a constructor and insert a call to `super()` with arguments matching those of the superclass constructor.
- ☐ Constructors are never inherited, thus they cannot be overridden.
- ☐ A constructor can be directly invoked only by another constructor (using a call to `super()` or `this()`).
- ☐ Issues with calls to `this()`
 - ☐ May appear only as the first statement in a constructor.
 - ☐ The argument list determines which overloaded constructor is called.

- ❑ Constructors can call constructors, and so on, but sooner or later one of them better call `super()` or the stack will explode.
- ❑ Calls to `this()` and `super()` cannot be in the same constructor. You can have one or the other, but never both.

Statics (Objective 1.3)

- ❑ Use `static` methods to implement behaviors that are not affected by the state of any instances.
- ❑ Use `static` variables to hold data that is class specific as opposed to instance specific—there will be only one copy of a `static` variable.
- ❑ All `static` members belong to the class, not to any instance.
- ❑ A `static` method can't access an instance variable directly.
- ❑ Use the dot operator to access `static` members, but remember that using a reference variable with the dot operator is really a syntax trick, and the compiler will substitute the class name for the reference variable, for instance:

```
d.doStuff();
```

becomes:

```
Dog.doStuff();
```

- ❑ `static` methods can't be overridden, but they can be redefined.

Coupling and Cohesion (Objective 5.1)

- ❑ Coupling refers to the degree to which one class knows about or uses members of another class.
- ❑ Loose coupling is the desirable state of having classes that are well encapsulated, minimize references to each other, and limit the breadth of API usage.
- ❑ Tight coupling is the undesirable state of having classes that break the rules of loose coupling.
- ❑ Cohesion refers to the degree in which a class has a single, well-defined role or responsibility.
- ❑ High cohesion is the desirable state of a class whose members support a single, well-focused role or responsibility.
- ❑ Low cohesion is the undesirable state of a class whose members support multiple, unfocused roles or responsibilities.

SELF TEST

1. Given:

```
public abstract interface Frobnicate { public void twiddle(String s); }
```

Which is a correct class? (Choose all that apply.)

- A.

```
public abstract class Frob implements Frobnicate {  
    public abstract void twiddle(String s) { }  
}
```
- B.

```
public abstract class Frob implements Frobnicate { }
```
- C.

```
public class Frob extends Frobnicate {  
    public void twiddle(Integer i) { }  
}
```
- D.

```
public class Frob implements Frobnicate {  
    public void twiddle(Integer i) { }  
}
```
- E.

```
public class Frob implements Frobnicate {  
    public void twiddle(String i) { }  
    public void twiddle(Integer s) { }  
}
```

2. Given:

```
class Top {  
    public Top(String s) { System.out.print("B"); }  
}  
public class Bottom2 extends Top {  
    public Bottom2(String s) { System.out.print("D"); }  
    public static void main(String [] args) {  
        new Bottom2("C");  
        System.out.println(" ");  
    }  
}
```

What is the result?

- A. BD
- B. DB
- C. BDC
- D. DBC
- E. Compilation fails